



IA PARA VALORACIÓN DE OPCIONES: MODELOS DE VOLATILIDAD EXPLICABLES

PEDRO GALLEGO LÓPEZ Y DANIEL RECUERO CORDOBÉS

Trabajo Final de Máster

Máster en Inteligencia Artificial y Computación Cuántica
Aplicada a los Mercados Financieros

Dirección

Enrique Castellanos

INSTITUTO BME

BOLSAS Y MERCADOS ESPAÑOLES

Madrid, 18 de mayo de 2026



IA para valoración de opciones: modelos de volatilidad explicables

Pedro Gallego López
Daniel Recuero Cordobés

Palabras clave:

Volatilidad implícita, opciones IBEX, Black-76, aprendizaje automático, IA explicable, SHAP, dashboard, API, cloud

Resumen

Este proyecto tiene como objetivo desarrollar modelos de volatilidad implícita para opciones sobre el IBEX mediante técnicas de inteligencia artificial e integrarlos en un marco de explicabilidad riguroso. La motivación central es combinar la capacidad de los modelos de IA para capturar relaciones no lineales complejas en la superficie de volatilidad con los requisitos de interpretabilidad y auditabilidad propios de la modelización financiera tradicional.

A partir de operaciones reales, contratos, futuros subyacentes y series de tipos EONIA/€STR, se construye una base de volatilidad implícita mediante validaciones financieras e inversión numérica del modelo Black-76. Sobre esta base se entrenan varias familias de modelos de regresión: regresión lineal, Random Forest, XGBoost, redes neuronales y una arquitectura *quantum inspired*, inspirada en conceptos de computación cuántica. La evaluación no se limita al error predictivo, sino que incorpora estabilidad temporal, control de sobreajuste, coherencia financiera de la superficie y análisis por tramos de moneyness y vencimiento.

La contribución principal consiste en transformar modelos predictivos en sistemas explicables, auditables y consultables mediante un dashboard y un servicio API. La explicabilidad se aborda con SHAP global y local, superficies de volatilidad, curvas ICE y ALE, árboles surrogate, regresión simbólica, vecinos históricos y diagnósticos de residuales. Esta combinación permite justificar las predicciones con un lenguaje cercano al de los modelos tradicionales, sin renunciar a la capacidad de aproximación de la IA. El resultado es una arquitectura reproducible orientada a la validación de modelos, la auditoría interna y el uso operativo en entornos financieros.

Adicionalmente, se ha implementado una API que permite el despliegue y consumo de estos modelos en distintos contextos de uso.

AI for option pricing: explainable volatility models

Pedro Gallego López
Daniel Recuero Cordobés

Keywords:

Implied volatility, IBEX options, Black-76, machine learning, explainable AI, SHAP, dashboard, API, cloud

Abstract

The main objective of this project is to develop implied-volatility models for IBEX options using artificial-intelligence techniques and to integrate them into a rigorous explainability framework. The central motivation is to combine the ability of AI models to capture complex nonlinear relationships across the volatility surface with the interpretability and auditability requirements of traditional financial modelling.

Starting from real trades, contracts, underlying futures and EONIA/€STR rate series, the project builds an implied-volatility database through financial validations and numerical inversion of the Black-76 pricing model. Several regression families are then trained, including linear regression, Random Forest, XGBoost, neural networks and a quantum-inspired architecture, based on quantum computing concepts as its name suggests. The evaluation is not restricted to predictive error; it also considers temporal stability, overfitting control, financial coherence of the surface, and performance across moneyness and maturity segments.

The main contribution is to turn predictive models into explainable, auditable and queryable systems through a dashboard and an API service. Explainability is addressed through global and local SHAP, volatility surfaces, ICE and ALE curves, surrogate trees, symbolic regression, historical neighbours and residual diagnostics. This combination makes it possible to justify predictions in a language close to traditional financial models, while preserving the approximation power of AI. The resulting architecture is reproducible and oriented towards model validation, internal audit and operational use in financial environments.

Additionally, an API has been implemented to support the deployment and consumption of these models across different usage contexts.

ÍNDICE GENERAL

1. INTRODUCCIÓN	7
1.1. Planteamiento del problema	7
1.2. Relevancia financiera y profesional	8
1.3. Antecedentes	9
1.4. Objetivos	10
1.5. Contribución del trabajo	10
1.6. Estructura de la memoria	12
2. MARCO TEÓRICO	13
2.1. Opciones, futuros y volatilidad implícita	13
2.2. Smile, moneyness y estructura temporal	14
2.3. Aprendizaje automático para volatilidad	15
2.4. Explicabilidad en modelos financieros	16
2.5. Métricas de error, estabilidad y fidelidad	16
2.6. Gobierno de modelos y preguntas de validación	17
3. METODOLOGÍA	19
3.1. Diseño general del estudio	19
3.2. Datos y muestra analizada	20
3.3. Validaciones y tratamiento financiero	25
3.4. Variables y feature engineering	25
3.5. Familias de modelos	27
3.6. Splits temporales y selección	28
3.7. Reentrenamiento y progressive ATM	29
3.8. Construcción del dashboard	31
3.9. API, almacenamiento y cloud	33
3.10. reproducibilidad	35
3.11. Arquitectura del repositorio	35
3.12. Riesgos metodológicos y controles	37
4. RESULTADOS Y DISCUSIÓN	38
4.1. Resultados	38
4.1.1. Comparativa general de modelos	38
4.1.2. Entrenamiento estándar frente a progressive ATM	38
4.1.3. Modelo de referencia	39
4.1.4. Diagnóstico de rendimiento	39
4.1.5. Behaviour And Surface	41
4.1.6. Explicabilidad global	42
4.1.7. Modelos equivalentes: árboles y regresión simbólica	43
4.1.8. Explicabilidad local y soporte histórico	45

4.2.	Aplicaciones operativas y validación independiente	47
4.2.1.	Validación independiente de modelos internos	48
4.2.2.	Interpolación y suavizado: alcance real del enfoque	48
4.2.3.	Detección de anomalías de precio y errores de mercado	48
4.2.4.	Valoración de ilíquidas, control de mesa de negociación, auditoría y cobertura	49
4.3.	Discusión	49
4.3.1.	Lectura financiera	50
4.3.2.	Limitaciones	51
5.	CONCLUSIONES	52
A.	ESQUEMA DETALLADO DE DATOS Y ETL	54
B.	METODOLOGÍA DE DESARROLLO, VALIDACIÓN Y DESPLIEGUE	58
C.	FUNDAMENTOS DE SHAP	61
C.1.	Modelo aditivo de explicación	61
C.2.	Valores de Shapley	62
C.3.	Función de valor en explicabilidad de modelos	63
C.4.	Propiedades de los valores SHAP	64
C.5.	Interpretación local y global	65
C.6.	Aproximación mediante permutaciones	66
C.7.	Variables de fondo y valor base	67
C.8.	Limitaciones interpretativas	67
C.9.	Uso en este proyecto	68
D.	CURVAS ICE	69
E.	CURVAS ALE	73
F.	MODELO QUANTUM INSPIRED	78
F.1.	Arquitectura general: flujo de datos desde entrada a predicción	78
F.2.	Etapas preparatorias y transformación estructural	79
F.2.1.	Normalización e Embedding denso	79
F.2.2.	Reshape a tensor de alta dimensionalidad	79
F.3.	El núcleo Quantum Inspired: Descomposición Tensor-Train	80
F.3.1.	La cadena de contacto (Contact Chain) y propagación de dependencias	80
F.3.2.	Implementación: estructura concreta de los núcleos	81
F.3.3.	Contracción: cómo se produce una salida TT	81
F.3.4.	Ejemplo concreto: relación entre dimensiones de entrada, rangos y salidas	82
F.4.	Etapas finales: Agregación y salida	83
F.5.	Justificación del enfoque Quantum Inspired	83
F.6.	Ventajas frente a una red densa estándar	84
G.	FUNDAMENTOS DE REGRESIÓN SIMBÓLICA	86
G.1.	Motivación	86
G.2.	Espacio de búsqueda de expresiones	87
G.3.	Objetivo de optimización	88

G.4. Búsqueda evolutiva	89
G.5. Frontera precisión-complejidad	89
G.6. Regresión simbólica como modelo sustituto	90
G.7. Configuración utilizada en el proyecto	90
G.8. Evaluación de fidelidad	92
G.9. Interpretación de la ecuación simbólica	93
G.10. Limitaciones	93
G.11. Relación con la explicabilidad del proyecto	94
H. ENTRENAMIENTO PROGRESSIVE ATM	95
I. ANÁLISIS DE SENSIBILIDAD POR MONEYNES Y VENCIMIENTO	96

INTRODUCCIÓN

1.1 PLANTEAMIENTO DEL PROBLEMA

La volatilidad implícita es un concepto central en mercados de derivados. En una opción negociada, el precio observable resume expectativas, oferta y demanda, liquidez, microestructura y percepción de riesgo. Sin embargo, para comparar contratos con strikes y vencimientos distintos, los profesionales no suelen razonar únicamente en precios: transforman esos precios en volatilidades implícitas y analizan smiles, skews y estructuras temporales [Hull, 2022, Derman and Miller, 2016].

El problema de este trabajo surge de una doble exigencia en finanzas cuantitativas. Por un lado, los modelos de aprendizaje automático pueden aproximar relaciones no lineales entre strike, subyacente, vencimiento, tipo de interés y volatilidad. Por otro lado, en un entorno financiero no basta con obtener una predicción numéricamente competitiva. El modelo debe poder auditarse: hay que saber dónde falla, qué variables explican sus predicciones, si respeta patrones razonables de la superficie y si una predicción concreta está respaldada por datos históricos similares. Esta exigencia es coherente con la literatura reciente sobre explicabilidad y gobierno de modelos en finanzas, que enfatiza transparencia, interpretabilidad, documentación y revisión independiente cuando se usan modelos complejos en procesos críticos [Solaimani and Long, 2025, Perez-Cruz et al., 2025].

En este contexto, el proyecto construye modelos de volatilidad explicable, consultable y útil vía API. La explicabilidad se entiende en dos niveles. A nivel de ingeniería, sirve para detectar sesgos, errores de datos, incoherencias, discontinuidades y comportamientos no deseados. A nivel financiero y profesional, permite justificar decisiones ante revisiones internas, auditorías, model validation o análisis de riesgo.

Operativamente, el problema se concreta en cuatro preguntas.

- Cómo transformar operaciones reales de mercado en una base de volatilidad implícita que sea consistente, trazable y reutilizable.

- Cómo aproximar la superficie $\sigma(K, T)$ con modelos suficientemente flexibles sin introducir sesgos temporales o dependencias espurias.
- Cómo operacionalizar ese modelo de forma utilizable, de manera que no quede encerrado en notebooks o tablas estáticas.
- Cómo saber cuándo una predicción es robusta y cuándo conviene tratarla con cautela porque está en una región poco soportada o mal explicada.

La memoria se construye alrededor de esa cadena completa. No se limita a entrenar un predictor, sino que conecta datos de mercado, inversión de Black-76, selección de familias, explicabilidad global y local, diagnóstico por zonas de la superficie y despliegue. Ese enfoque es coherente con el tipo de uso que tendría el sistema en un entorno profesional: consulta repetida, revisión crítica y necesidad de justificar el resultado más allá de una métrica agregada. La construcción detallada de datos y ETL se amplía en el apéndice A; la parte de explicabilidad se desarrolla con más detalle en los apéndices C, apéndice E y apéndice G; y la dimensión de despliegue y trazabilidad de ingeniería se complementa en el apéndice B.

Estas cuatro preguntas se responden de forma progresiva en el [chapter 3](#) (construcción y validación del pipeline), en el [chapter 4](#) (evidencia empírica y lectura operativa) y en el [chapter 5](#) (síntesis final y alcance práctico).

1.2 RELEVANCIA FINANCIERA Y PROFESIONAL

La volatilidad implícita condensa información de mercado y se utiliza en valoración, cobertura, gestión de riesgos y análisis de escenarios. En opciones sobre índices, la forma de la superficie de volatilidad puede reflejar demanda de protección, asimetría percibida, presión de liquidez o cambios de régimen. Un modelo que suaviza en exceso los extremos de moneyness, que comprime volatilidades extremas o que presenta saltos artificiales en vencimientos cortos puede producir señales engañosas aunque su RMSE agregado sea aceptable.

La literatura clásica de derivados establece que la volatilidad no debe interpretarse como un simple parámetro estadístico fijo, sino como una magnitud inferida del precio bajo un modelo de valoración [Black, 1976, Hull, 2022]. La literatura específica sobre smile muestra que los mercados organizan la volatilidad en torno a moneyness y vencimiento, no solo en torno a niveles absolutos de strike [Derman and Miller, 2016]. Por ello, el proyecto se diseña desde la geometría financiera de la superficie: el aprendizaje automático se usa para aproximar una función de volatilidad, pero la evaluación exige interpretar su forma.

La relevancia práctica aparece en varios frentes.

- **Valoración y marking:** una estimación coherente de volatilidad implícita permite comparar contratos y revisar precios fuera de rango.

- **Gestión de riesgos:** la sensibilidad del error por moneyness y vencimiento importa tanto como el error medio total.
- **Gobierno de modelos:** el área usuaria necesita entender si el modelo reacciona con lógica financiera y no solo si ajusta bien una muestra histórica.
- **Infraestructura analítica:** si el modelo no puede consultarse, versionarse y explicarse, su valor operativo cae de forma drástica.

Desde esa perspectiva, un buen resultado no es únicamente “predecir bien”. También implica poder distinguir entre una predicción bien soportada, una extrapolación razonable y una salida que requiere revisión adicional. Ese matiz es el que justifica que la memoria dedique espacio tanto a finanzas como a IA y a arquitectura de servicio.

1.3 ANTECEDENTES

La valoración de opciones sobre futuros se apoya tradicionalmente en Black-76 [Black, 1976]. En ese marco, el precio de una opción europea depende del futuro subyacente, el strike, el vencimiento, el tipo de interés y la volatilidad. Cuando el precio de mercado se observa, la volatilidad implícita se obtiene resolviendo numéricamente la ecuación inversa. En la literatura reciente, las redes neuronales se han utilizado tanto para acelerar la valoración y el cálculo de volatilidades implícitas [Liu et al., 2019] como para estudiar movimientos de la superficie de volatilidad en función de moneyness, vencimiento y entorno de volatilidad [Cao et al., 2019].

En paralelo, el uso de modelos de aprendizaje automático para superficies financieras responde a la necesidad de capturar no linealidades, interacciones y patrones empíricos que no quedan completamente descritos por modelos paramétricos simples. Sin embargo, la adopción profesional de modelos flexibles exige herramientas de interpretación. SHAP proporciona una semántica aditiva para atribuir predicciones a variables [Lundberg and Lee, 2017]; ICE y ALE ayudan a estudiar respuestas funcionales [Goldstein et al., 2015b, Apley and Zhu, 2020]; los árboles surrogate y la regresión simbólica aproximan modelos complejos mediante objetos interpretables [Cranmer, 2023].

Los antecedentes relevantes convergen en tres líneas.

- **Línea financiera:** la inversión de modelos como Black-76 convierte precios observados en volatilidades comparables y lleva el problema al terreno de la superficie.
- **Línea empírica:** los smiles y las estructuras temporales muestran que la volatilidad de mercado no es constante y que la relación con strike y vencimiento es no lineal.

- **Línea metodológica:** los modelos de caja negra ganan flexibilidad, pero obligan a complementar la predicción con mecanismos de explicación y validación.

Este trabajo se sitúa precisamente en la intersección de esas tres líneas. Toma la formulación clásica de derivados como base para construir la variable objetivo, utiliza familias de aprendizaje automático sobre datos tabulares para aproximar la función de volatilidad y añade una capa de explicabilidad y servicio para hacer la solución defendible en un contexto profesional.

1.4 OBJETIVOS

El objetivo general es desarrollar una solución reproducible para estimar, explicar y consultar volatilidad implícita de opciones del IBEX. Este objetivo se concreta en los siguientes objetivos específicos:

1. Construir una base de volatilidad implícita a partir de operaciones, contratos, futuros subyacentes y curvas de tipos, con validaciones de calidad y trazabilidad.
2. Diseñar un conjunto de variables financieras coherentes con la geometría de la superficie de volatilidad.
3. Entrenar y comparar varias familias de modelos de regresión bajo una metodología temporal que reduzca lookahead bias y data snooping.
4. Generar componentes de explicabilidad global, local, funcional y de diagnóstico.
5. Implementar una API y un dashboard que permitan consultar predicciones, explicar muestras concretas y revisar el comportamiento del modelo.
6. Preparar infraestructura cloud reproducible para almacenar modelos, paquetes explicativos y metadatos de servicio.

Tomados en conjunto, estos objetivos cubren los tres planos que estructuran la memoria. El primero es **financiero**, porque la variable objetivo y las transformaciones deben respetar la lógica de opciones sobre futuros. El segundo es **de IA y Data Science**, porque la comparación entre familias exige un diseño temporal riguroso, métricas correctas y lectura crítica del error. El tercero es **de ingeniería**, porque el resultado final no es un gráfico aislado, sino un sistema consultable y versionable.

1.5 CONTRIBUCIÓN DEL TRABAJO

La principal contribución del trabajo es integrar en un único flujo operativo elementos que habitualmente se tratan por separado: construcción de datos de mercado, inversión de Black-76, entrenamiento de modelos, explicabilidad, diagnóstico finan-

ciero, API y despliegue cloud. La Figura 1 resume esta cadena con las mismas dependencias que la documentación técnica.

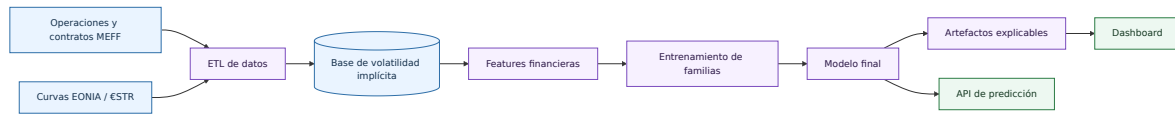


Figura 1: Visión general del proyecto.

La aportación académica se sitúa en la metodología de modelización y validación; la aportación profesional, en la capacidad de convertir un modelo de volatilidad en una herramienta inspeccionable y desplegable.

De forma más concreta, la contribución del trabajo puede resumirse así:

- **Contribución financiera:** construir una base de volatilidad implícita a partir de datos reales enlazando operaciones de opciones, futuros y curvas de tipos.
- **Contribución metodológica:** comparar familias heterogéneas con el mismo protocolo temporal, las mismas métricas y el mismo criterio de selección.
- **Contribución de explicabilidad:** combinar atribuciones SHAP, curvas ICE y ALE, modelos equivalentes, superficies y vecinos históricos en un único flujo de lectura.
- **Contribución de ingeniería:** dejar el modelo convertido en un paquete auto-contenido consumible por dashboard y API, con almacenamiento reproducible y despliegue desacoplado.

Desde el punto de vista de negocio en derivados, el valor aplicado se concreta en ocho usos directos:

- **Valoración de opciones ilíquidas o poco negociadas:** la predicción se apoya en estructura de superficie y soporte histórico de vecinos, no solo en una cotización puntual.
- **Interpolación y suavizado de superficies de volatilidad:** el dashboard permite revisar continuidad por moneyness y vencimiento y detectar escalones no deseados.
- **Detección de precios anómalos o errores de mercado:** los residuales por zonas de superficie ayudan a localizar observaciones fuera de patrón.
- **Validación independiente de modelos internos:** el pipeline puede usarse como contraste externo frente a pricers de mesa o librerías internas.
- **Control de riesgos de mesa de derivados:** el error deja de leerse solo en promedio y se monitoriza por tramos relevantes de moneyness y vencimiento.
- **Explicación ante auditoría interna o model validation:** cada predicción puede justificarse con SHAP, curvas funcionales y evidencia de soporte histórico.
- **Análisis de sensibilidad por moneyness y vencimiento:** superficies locales, ICE y ALE aportan lectura funcional de cómo reacciona el modelo.

- **Soporte a decisiones de cobertura:** la lectura conjunta de smile, term structure y diagnóstico local ofrece contexto operativo para coberturas.

Esa combinación es la que hace que el proyecto no sea solo un ejercicio de entrenamiento de modelos. El valor añadido está en que cada capa técnica responde a una pregunta concreta y útil: cómo se construye la volatilidad, cómo se estima, cómo se valida, cómo se explica y cómo se despliega.

1.6 ESTRUCTURA DE LA MEMORIA

El [chapter 2](#) presenta los fundamentos financieros, de aprendizaje automático, explicabilidad e infraestructura. El [chapter 3](#) describe la construcción de datos, variables, modelos, validación, dashboard, API y cloud. El [chapter 4](#) analiza las métricas persistentes y la lectura que debe hacerse desde las vistas del dashboard. El [chapter 5](#) resume conclusiones y líneas futuras. Los apéndices desarrollan con mayor profundidad la estructura de datos y ETL (apéndice [A](#)), la metodología de desarrollo y despliegue (apéndice [B](#)), SHAP (apéndice [C](#)), ALE (apéndice [E](#)), el modelo Quantum Inspired (apéndice [F](#)), la regresión simbólica (apéndice [G](#)), el entrenamiento progressive ATM (apéndice [H](#)) y el análisis de sensibilidad por moneyness y vencimiento (apéndice [I](#)).

El orden de lectura responde al flujo real del proyecto.

1. En la **Introducción** se delimita el problema y se justifica por qué la explicabilidad es parte del resultado, no un añadido posterior.
2. En el **Marco Teórico** se fijan las piezas conceptuales necesarias para interpretar Black-76, smiles, familias de modelos, métricas y gobierno de modelos.
3. En la **Metodología** se documenta la implementación efectiva: ETL, features, validación temporal, reentrenamiento, generación del paquete de dashboard y despliegue.
4. En **Resultados y Discusión** se conectan métricas, superficies, explicaciones y limitaciones con una lectura financiera y técnica unificada.
5. En los **Apéndices** se amplían los mecanismos de explicabilidad y entrenamiento para que la memoria principal conserve foco sin perder profundidad técnica.

MARCO TEÓRICO

2.1 OPCIONES, FUTUROS Y VOLATILIDAD IMPLÍCITA

Una opción financiera otorga a su comprador el derecho, pero no la obligación, de comprar o vender un activo subyacente a un precio de ejercicio K en una fecha futura. En el caso de opciones sobre futuros, el subyacente relevante para valoración es el precio del futuro F . La literatura de derivados enfatiza que el valor de una opción depende de la distribución esperada del subyacente y, de forma operativa, de la volatilidad usada en el modelo de valoración [Hull, 2022].

En el modelo Black-76, una call europea sobre un futuro se valora como:

$$C = e^{-rT} (FN(d_1) - KN(d_2)), \quad (1)$$

donde

$$d_1 = \frac{\ln(F/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T}. \quad (2)$$

Para una put:

$$P = e^{-rT} (KN(-d_2) - FN(-d_1)). \quad (3)$$

En estas expresiones, r es el tipo de interés, T el tiempo a vencimiento en años, σ la volatilidad y $N(\cdot)$ la función de distribución normal estándar. La volatilidad implícita σ_{imp} es el valor de σ que iguala el precio teórico al precio observado en mercado:

$$P_{mercado} = P_{Black76}(F, K, T, r, \sigma_{imp}). \quad (4)$$

La ecuación (4) no se puede resolver en forma cerrada en general; se obtiene mediante métodos numéricos. En el proyecto se emplea bisección dentro de límites configurados, descartando filas no resolubles o no finitas.

Para el proyecto, esta formulación tiene varias implicaciones directas.

- La calidad de la volatilidad implícita depende de la calidad del precio de opción y del emparejamiento con el futuro subyacente.
- El tiempo a vencimiento no es un mero metadato: entra de forma no lineal a través de $\sqrt{T}$, por lo que errores en fechas o vencimientos contaminan la variable objetivo.
- El tipo de interés afecta tanto al descuento como a la construcción de variables derivadas basadas en el forward ajustado.
- Calls y puts comparten el mismo concepto de volatilidad implícita, pero pueden situarse en zonas distintas de la superficie y, por tanto, inducir respuestas diferentes en el modelo.

Por eso la memoria no trata la volatilidad implícita como una etiqueta estadística cualquiera. Antes de entrenar ningún modelo, la variable objetivo debe quedar bien definida desde el punto de vista financiero y bajo un procedimiento numérico reproducible.

2.2 SMILE, MONEYNES Y ESTRUCTURA TEMPORAL

Si la volatilidad fuese constante, opciones con distinto strike y vencimiento compartirían una misma σ . En mercados reales aparece una superficie de volatilidad:

$$\sigma_{imp} = \sigma(K, T), \quad (5)$$

o, de forma más comparable entre niveles de mercado:

$$\sigma_{imp} = \sigma(m, T), \quad m = \frac{F}{K}. \quad (6)$$

El moneyness m indica la posición relativa del subyacente frente al strike. Su transformación logarítmica,

$$\ell = \log(F/K), \quad (7)$$

es especialmente útil porque centra la región ATM en $\ell = 0$ y trata de forma más simétrica las zonas a ambos lados del strike. La literatura sobre smile subraya que la curvatura y pendiente de la volatilidad implícita contienen información sobre

expectativas de mercado, asimetría y demanda de cobertura [Derman and Miller, 2016]. Trabajos más recientes muestran además que los movimientos de la superficie pueden modelarse explícitamente a partir del retorno del subyacente, el moneyness y la vida restante mediante redes neuronales [Cao et al., 2019].

En la práctica, esta superficie se lee a través de varios patrones.

- **Smile o skew:** describe cómo cambia la volatilidad implícita al alejarse de ATM.
- **Estructura temporal:** muestra cómo cambia la volatilidad con el vencimiento para una región dada de moneyness.
- **Interacción entre ambas:** la curvatura del smile no tiene por qué ser la misma en corto plazo que en vencimientos largos.

Esta lectura geométrica justifica el diseño posterior de variables y visualizaciones. Si el fenómeno financiero se organiza en torno a moneyness y vencimiento, es razonable que el feature engineering, la validación y los gráficos de comportamiento del modelo se construyan también alrededor de esas magnitudes.

2.3 APRENDIZAJE AUTOMÁTICO PARA VOLATILIDAD

El problema de modelización se formula como una regresión supervisada:

$$\hat{\sigma} = f(X), \quad (8)$$

donde X contiene variables financieras observables y transformadas. La literatura reciente ha explorado redes neuronales no solo para aproximar precios de opciones, sino también para acelerar el cálculo de volatilidades implícitas y tareas de calibración asociadas [Liu et al., 2019]. El proyecto compara familias con diferentes niveles de flexibilidad:

- Regresión lineal, como baseline interpretable y control de complejidad.
- Random Forest, basado en agregación de árboles para capturar interacciones no lineales [Breiman, 2001].
- XGBoost, como boosting regularizado con early stopping y buen rendimiento en datos tabulares [Chen and Guestrin, 2016].
- Redes neuronales densas, como aproximadores flexibles de funciones no lineales [Hornik et al., 1989].
- Una red neuronal clásica *quantum inspired*, que introduce una estructura tensorial inspirada en ideas de representación compacta y redes tensoriales para aprendizaje supervisado [Stoudenmire and Schwab, 2016, Novikov et al., 2015]. No se presenta como computación cuántica real, sino como una variante arquitectónica implementada en código. Su desarrollo conceptual y arquitectónico se amplía en el apéndice F.

La comparación no puede basarse solo en error medio. En datos financieros temporales, una evaluación mal planteada puede introducir fuga de información. Por ello se emplean splits temporales, control de contratos compartidos y k-folds temporales, tal como se desarrolla en el [section 3.6](#).

2.4 EXPLICABILIDAD EN MODELOS FINANCIEROS

La explicabilidad cumple una función técnica y una función de gobierno de modelos. Técnicamente, permite detectar variables dominantes, discontinuidades, extrapolaciones y errores locales. Profesionalmente, facilita responder por qué una predicción se considera razonable y en qué condiciones no debería confiarse en ella. En el ámbito financiero, esta necesidad se ha reforzado con trabajos recientes que estructuran la explicabilidad alrededor de transparencia, interpretabilidad y accountability, y que además la conectan con documentación, validación y revisión independiente [[Solaimani and Long, 2025](#), [Perez-Cruz et al., 2025](#)].

SHAP descompone la predicción de un modelo en un valor base y contribuciones por variable:

$$\hat{\sigma}(x) = \phi_0 + \sum_{j=1}^p \phi_j(x), \quad (9)$$

donde ϕ_j mide la contribución atribuida a la característica j [[Lundberg and Lee, 2017](#)]. En este proyecto se complementa con ICE, ALE, árboles surrogate, regresión simbólica y vecinos históricos. La razón es que ninguna herramienta por sí sola responde todas las preguntas relevantes: SHAP explica contribuciones, ICE muestra trayectorias individuales, ALE estima efectos acumulados locales, los modelos surrogate dan reglas aproximadas y los vecinos informan sobre soporte local. Los fundamentos formales de SHAP se desarrollan en el apéndice [C](#), el detalle operativo de ALE en el apéndice [E](#) y la lectura de la regresión simbólica como surrogate en el apéndice [G](#).

2.5 MÉTRICAS DE ERROR, ESTABILIDAD Y FIDELIDAD

La evaluación de un modelo de volatilidad requiere métricas complementarias. El error absoluto medio,

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (10)$$

ofrece una lectura directa en unidades de volatilidad. Es robusto y fácil de comunicar, pero no penaliza de forma diferencial los errores grandes. Por ese motivo se utiliza también RMSE:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (11)$$

El RMSE es especialmente relevante en model validation porque amplifica errores extremos. En una superficie de volatilidad, esos errores pueden aparecer en vencimientos cortos, extremos de moneyness o zonas con menor liquidez. El coeficiente de determinación,

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad (12)$$

resume proporción de variabilidad explicada, aunque debe interpretarse con prudencia en datos financieros heterogéneos.

Junto a estas métricas frente a mercado, el dashboard usa métricas de fidelidad para modelos explicables equivalentes. La fidelidad compara el surrogate con el modelo principal, no con la volatilidad real. Esta distinción es importante: un árbol surrogate puede ser muy fiel a un modelo sesgado, y un modelo principal puede tener buen RMSE pero ser difícil de resumir con reglas simples. Por tanto, la evaluación final debe combinar tres planos: precisión frente a mercado, estabilidad del proceso de selección y fidelidad de las explicaciones.

2.6 GOBIERNO DE MODELOS Y PREGUNTAS DE VALIDACIÓN

En un entorno financiero, un modelo de volatilidad debería poder responder a preguntas de model validation. Algunas son cuantitativas: cuál es el error fuera de muestra, si existe gap entre entrenamiento y validación, si el error cambia por vencimiento o moneyness, y si el resultado es estable entre folds. Otras son cualitativas pero igualmente relevantes: si el modelo reacciona de forma razonable ante cambios de strike, si produce superficies suaves donde el mercado debería ser continuo, si una predicción manual se encuentra dentro de una región observada o si el predictor depende excesivamente de una variable que no debería dominar.

El diseño del proyecto convierte esas preguntas en vistas concretas del dashboard.

- *La pestaña de diagnóstico* responde dónde falla.
- *La pestaña de superficie* responde cómo se comporta como función financiera.

- *La explicabilidad global* responde qué variables tienen mayor influencia en las predicciones del modelo.
- *La pestaña explicabilidad local* responde por qué una muestra concreta obtiene un valor determinado y si existen observaciones históricas cercanas.

Esto evita que una única visualización condicione toda la interpretación. En un entorno de derivados, la confianza en una predicción surge de la coherencia entre varias vistas, no de una sola gráfica llamativa o de una única métrica sintética.

METODOLOGÍA

3.1 DISEÑO GENERAL DEL ESTUDIO

El estudio tiene carácter empírico y aplicado. Parte de datos de mercado, construye una variable objetivo financiera mediante inversión de Black-76, entrena modelos de regresión y evalúa tanto rendimiento predictivo como coherencia explicativa. La Figura 2 resume el flujo metodológico. La ampliación de trazabilidad estructural del ETL se recoge en el apéndice A y la de ingeniería de desarrollo y despliegue en el apéndice B.

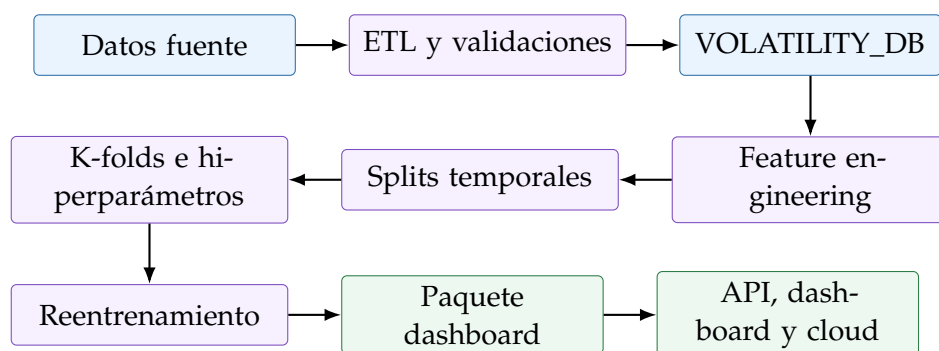


Figura 2: Flujo metodológico del proyecto.

El flujo no es lineal solo en apariencia. Cada bloque deja salidas persistidas y comprobables que sirven de entrada al siguiente, de modo que la metodología puede auditarse por etapas. Primero se consolida la base financiera; después se fija un espacio de variables coherente con la superficie; a continuación se comparan familias bajo un protocolo temporal común; y, por último, se transforma el modelo seleccionado en un sistema consultable. Este diseño responde a cuatro principios metodológicos.

- **Separación de responsabilidades:** ETL, modelización, explicabilidad y servicio no se mezclan en un único bloque opaco.
- **Trazabilidad de extremo a extremo:** cada predicción puede rastrearse hasta datos, configuración y versión de modelo.

- **Comparabilidad entre familias:** todas las alternativas compiten bajo las mismas reglas de split, métricas y reentrenamiento.
- **Lectura profesional del resultado:** el producto final no es solo una tabla de métricas, sino un modelo interpretable y servible.

Esta estructura es importante porque condiciona la credibilidad de los resultados. Si el entrenamiento fuese correcto pero las explicaciones o el servicio no reutilizasen exactamente la misma lógica de features, la calidad del sistema quedaría comprometida aunque las métricas numéricas fueran buenas.

3.2 DATOS Y MUESTRA ANALIZADA

El pipeline de datos transforma ficheros de contratos, operaciones y tipos en una base final de volatilidad implícita. Hay tres fuentes principales:

- **Contratos:** aportan información de código, strike y vencimiento.
- **Operaciones:** aportan precio, cantidad, hora y contrato.
- **Series de EONIA/€STR:** permiten calcular el tipo compuesto aplicable hasta vencimiento.

En el cálculo de tipos se asume una aproximación operativa: no se dispone de un calendario TARGET2 explícito dentro del pipeline, por lo que el conteo de días hábiles se implementa con `pd.bdate_range` (días laborables estándar). La composición se define mediante una regla explícita: se recorre cada día hábil del intervalo, se toma r_i (retrocediendo al último dato disponible si ese día no existe en la serie) y se multiplica $1 + r_i n_i / 360$, con $n_i = 3$ cuando el día hábil es lunes y $n_i = 1$ en el resto. Después se anualiza como

$$\left(\prod_i \left(1 + \frac{r_i n_i}{360} \right) - 1 \right) \cdot \frac{360}{d_c}.$$

La guía de referencia para la lógica de tipos compuestos es la del BCE para €STR: https://www.ecb.europa.eu/stats/euro-short-term-rates/interest_rate_benchmarks/WG_euro_risk-free_rates/shared/pdf/ecb.Compounded_euro_short-term_rate_calculation_rules.en.pdf. En esta memoria se documenta de forma explícita que la implementación actual reproduce la convención aquí especi-

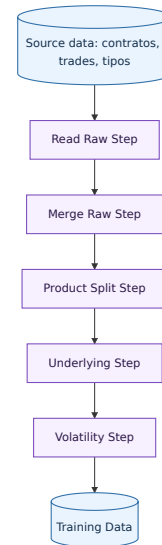


Figura 3: Secuencia principal de procesamiento de datos.

cada (incluido el peso 3 en lunes) y debe interpretarse como aproximación operativa mientras no se incorpore un calendario TARGET2 completo en la construcción de Rate.

La cadena ETL se materializa por pasos, como muestran las Figuras 3 y 4. Esta organización permite cachear salidas intermedias, validar contratos de datos y aislar responsabilidades: los *StepLoaders* orquestan lectura, cache y validación, mientras que los *builders* construyen la salida material de cada paso.

La Figura 5 resume esta separación entre clases *loader* y clases *builder*. El detalle extenso de las estructuras de datos que va produciendo cada etapa se recoge además en el apéndice A, para no sobrecargar el cuerpo principal con un diagrama demasiado largo.

Los datos se filtran al dominio de opciones IBEX mensuales y futuros mensuales asociados. Las opciones semanales y contratos fuera de dominio se excluyen para mantener coherencia contractual.

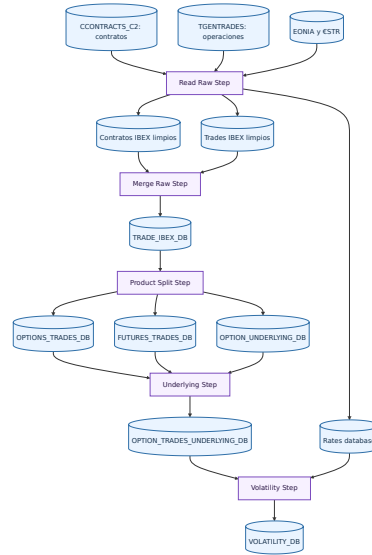


Figura 4: Pipeline completo de datos hasta VOLATILITY_DB.

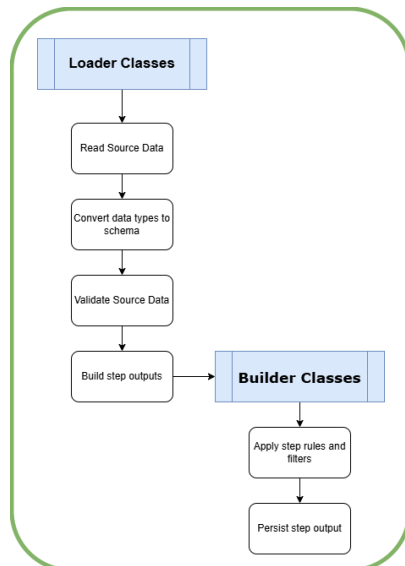


Figura 5: Esquema de responsabilidades entre loaders y builders en el ETL.

La salida final contiene fecha y hora de ejecución, código de opción, tipo call/put, precio negociado, strike, futuro subyacente, precio del subyacente, desfase temporal opción-subyacente (minutos), tiempo a vencimiento, tipo compuesto y volatilidad implícita.

El **primer paso**, Read Raw Step, normaliza ficheros de mercado con o sin cabecera, elimina columnas auxiliares no pertenecientes al esquema final, convierte fechas, horas y numéricos, filtra prefijos IBEX y construye una serie homogénea de tipos. La base de tipos combina EONIA ajustado por el spread configurado antes de la fecha de corte y €STR desde esa fecha. Figuras 6a, 6b.

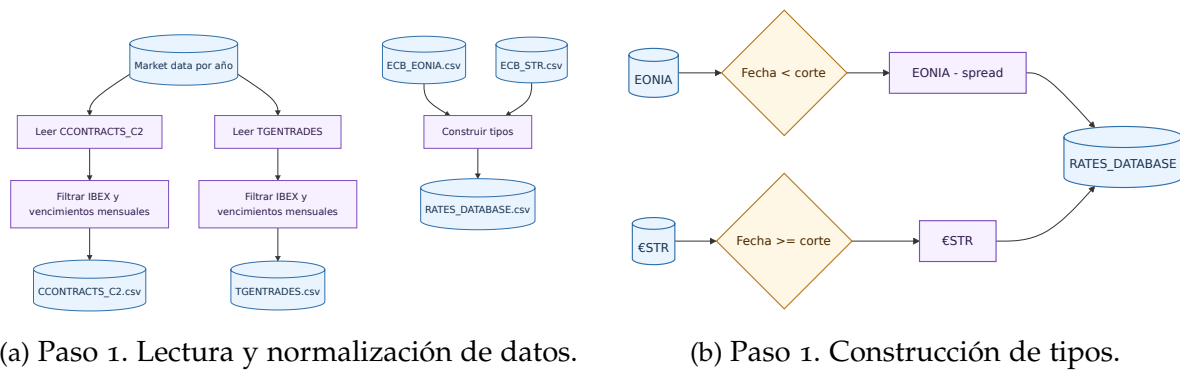


Figura 7: Paso 2. Unión por código de contrato y año de sesión.

El **segundo paso**, Merge Raw Step, une operaciones y contratos por ContractCode y año de sesión, imputa vencimientos con la regla del tercer viernes e imputa strikes desde el tramo configurado del código cuando la fuente no lo trae completo. Las Figuras 7 y 8a resumen visualmente esta fase.

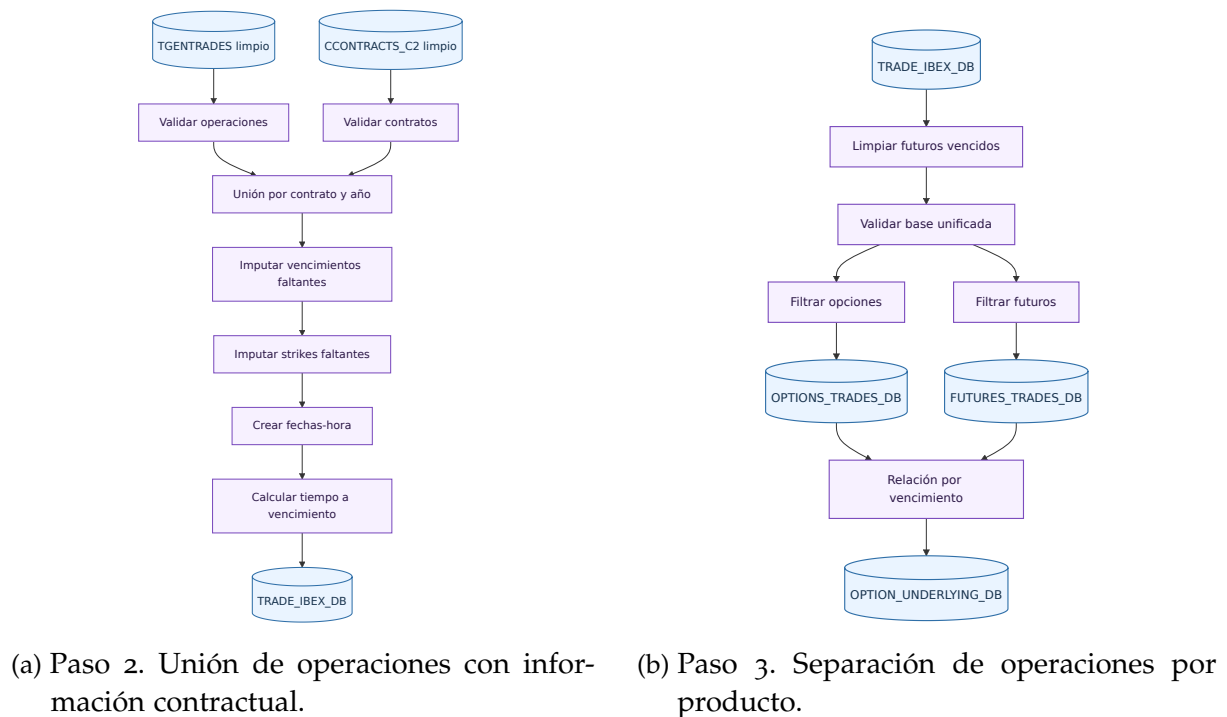


Figura 8: Unión de datos contractuales y separación por producto.

El **tercer paso**, Product Split Step, divide la base unificada en opciones, futuros y relación opción-futuro por vencimiento.

El **cuarto**, Underlying Step, garantiza el emparejamiento temporal sin lookahead y resuelve el precio de futuro subyacente mediante una unión temporal *as-of*: para cada opción selecciona el último trade del futuro con instante de ejecución menor o igual al de la opción. Esta decisión evita lookahead dentro del propio ETL, como se ilustra en la Figura 9.

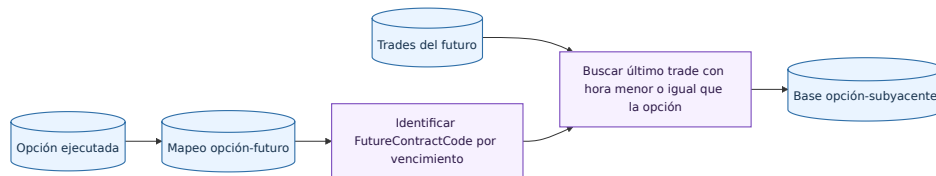


Figura 9: Paso 4. Asignación de futuro subyacente a cada opción.

El **quinto**, Volatility Step, filtra operaciones sin subyacente fiable o con desfase temporal excesivo, calcula el tipo compuesto hasta vencimiento y resuelve la volatilidad implícita por Black-76 inverso (Figuras 11 y 10).

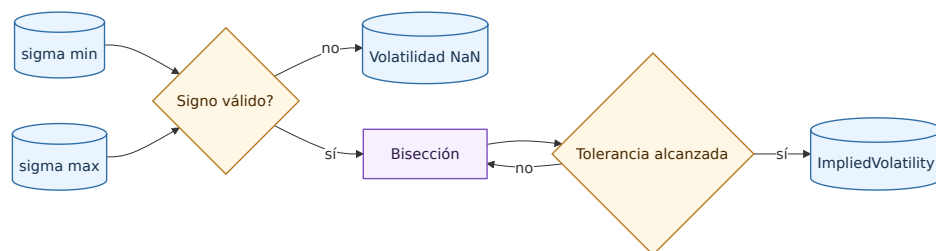


Figura 10: Paso 5. Solver por bisección de la volatilidad implícita.

En este **último paso** se aplican reglas de calidad relevantes para evitar sesgos en entrenamiento: se excluyen filas sin subyacente válido, filas con desfase opción-subyacente superior al umbral configurado de 180 minutos, y filas cuya volatilidad implícita no puede resolverse de forma válida (por ejemplo, sin cambio de signo del solver o con soluciones no finitas). Estas decisiones son coherentes con el análisis exploratorio y con las validaciones financieras del pipeline.

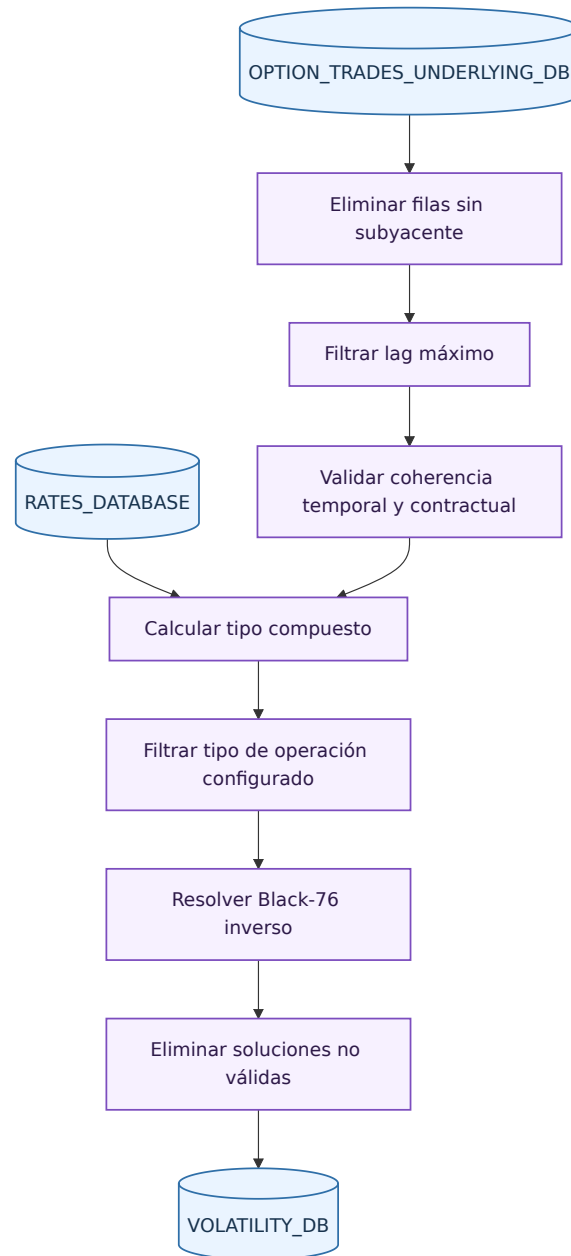


Figura 11: Paso 5. Cálculo de volatilidad implícita.

3.3 VALIDACIONES Y TRATAMIENTO FINANCIERO

El ETL aplica validaciones transversales: precios positivos, cantidades positivas, ausencia de claves duplicadas, control de missing values, vencimientos coherentes, strikes compatibles con el código de contrato y subyacente temporalmente anterior a la operación de la opción. Esta última condición es crítica para evitar utilizar información futura.

Para evitar ambigüedades, en esta memoria se distinguen dos conceptos de lag: el desfase opción-subyacente del ETL (en minutos) y el lag temporal del split de entrenamiento (en número de fechas) usado más adelante en la validación de modelos.

El cálculo de volatilidad implícita se realiza en el último paso. Se eliminan operaciones sin subyacente fiable, se calcula el tipo compuesto hasta vencimiento y se resuelve la ecuación (4). Si no existe cambio de signo en el intervalo de búsqueda o la solución es no finita, la observación se marca como no resoluble y no alimenta el entrenamiento. La Figura 12 resume estas validaciones sobre el contrato y su interpretación.

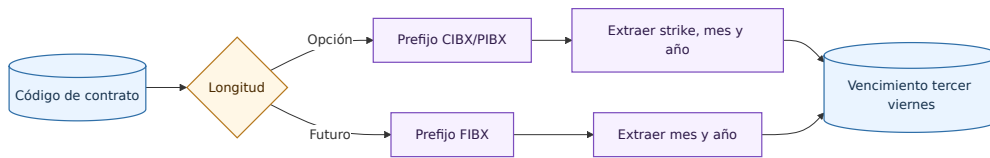


Figura 12: Validación e interpretación del código de contrato.

3.4 VARIABLES Y FEATURE ENGINEERING

Las entradas raw visibles son `OptionType`, `StrikePrice`, `UnderlyingPrice`, `TimeToExpiration` y `Rate`. Estas variables son suficientes para reconstruir el vector de entrenamiento y también son comprensibles para un usuario financiero que consulte la API.

No se aplica una fase de selección de variables ex post (ni filtros automáticos de descarte por importancia) porque el foco metodológico es simular una operación directa con un conjunto de entradas financieras estable, reproducible y utilizable tal cual en API/dashboard. La comparación entre familias se hace, por diseño, sobre el mismo espacio de variables para aislar el efecto de la arquitectura y no de una poda específica de features.

El vector transformado se diseña para representar la geometría de la superficie. La Tabla 1 resume las variables derivadas.

Feature	Fórmula	Motivo
TTEYears	$T = T_{\text{días}}/365$	Vencimiento en años
sqrtTTEYears	$\sqrt{T}$	Escala temporal de Black-76
logMoneyness	$\log(F/K)$	Posición relativa frente al strike
logMoneynessSq	$\log(F/K)^2$	Curvatura del smile
logMoneynessXSqrtTTE	$\log(F/K)\sqrt{T}$	Interacción smile-vencimiento
logForwardMoneyness	$\log(Fe^{rT}/K)$	Ajuste por tipo y vencimiento
rate	r	Sensibilidad directa a tipos
isCall, isPut	Indicadores	Tipo de opción

Cuadro 1: Features financieras utilizadas por los modelos.

Esta elección evita que el modelo dependa de niveles absolutos de strike que pueden cambiar con el régimen de mercado. El moneyness permite comparar contratos negociados en momentos distintos del IBEX, mientras que el vencimiento y sus transformaciones capturan estructura temporal. Las Figuras 13 y 14 muestran la construcción y motivación financiera de estas variables.

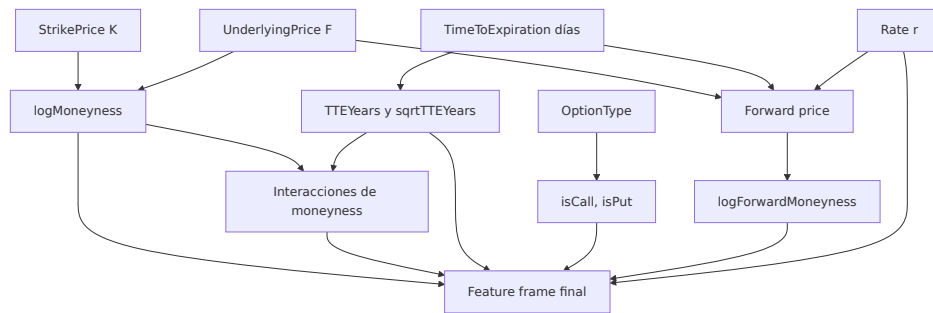


Figura 13: Construcción de features financieras.

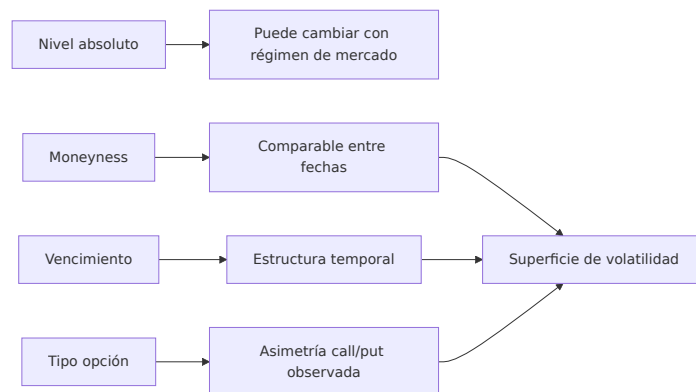


Figura 14: Motivación financiera de las features.

3.5 FAMILIAS DE MODELOS

El repositorio implementa una abstracción común para familias de modelos. Cada familia declara nombre, parámetros fijos, espacio de búsqueda, instanciación, entrenamiento y persistencia. La Tabla 2 resume su papel en el estudio.

Familia	Papel metodológico
Regresión lineal	Baseline interpretable para evaluar cuánto aporta la no linealidad.
Random Forest	Modelo tabular robusto ante interacciones y no linealidades sin escalado.
XGBoost	Boosting regularizado con early stopping interno y alta capacidad predictiva.
Red neuronal secuencial	Aproximador flexible con escalado de variables numéricas y early stopping sobre validación interna.
Quantum Inspired NN	Variante clásica de red neuronal con estructura tensorial inspirada en ideas cuánticas; se evalúa como arquitectura implementada, no como computación cuántica física.

Cuadro 2: Familias de modelos entrenadas en el proyecto.

Todas las familias heredan una interfaz común: declaran parámetros fijos, espacio de búsqueda, instanciación, entrenamiento, persistencia y número de configuraciones exploradas. Esta homogeneidad permite comparar un baseline lineal, modelos de árboles, boosting y arquitecturas neuronales con el mismo protocolo de selección. En la familia Quantum Inspired, el cálculo sigue siendo clásico: las features se proyectan a un embedding, se reordenan como tensor y se contraen mediante una capa tensor-train antes de una capa densa final. El concepto y su implementación práctica se detallan en el apéndice F.

La elección de familias responde a una lógica comparativa.

- La **regresión lineal** fija una referencia interpretable y permite medir cuánto valor añade la no linealidad una vez creadas las features correctas.
- **Random Forest** y **XGBoost** representan dos formas distintas de explotar interacciones tabulares complejas sin exigir el mismo tipo de preprocesado que las redes.
- Las **redes neuronales** sirven para comprobar si una arquitectura más flexible mejora realmente el problema o solo incrementa el coste y la complejidad de explicación.
- La variante **Quantum Inspired** introduce una hipótesis arquitectónica diferenciada manteniendo computación clásica, lo que permite valorar su aportación real frente a alternativas más estándar.

La interfaz común del repositorio hace además que la comparación sea metodológicamente limpia. Todas las familias pasan por los mismos splits temporales, las mismas métricas y el mismo proceso de reentrenamiento y conversión a paquete de dashboard. Eso evita que una familia “gane” por haber sido evaluada con un protocolo distinto del resto.

3.6 SPLITS TEMPORALES Y SELECCIÓN

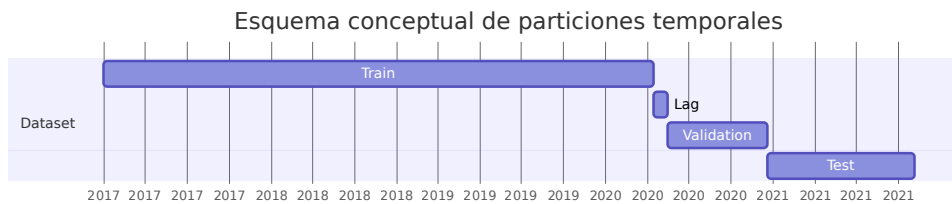


Figura 15: Esquema conceptual de particiones temporales.

La validación sigue tres principios.

1. Respeto del orden temporal: train precede a validation y test.
2. Puede existir un lag de fechas entre bloques para reducir contaminación temporal.
3. Exclusividad de contratos: si el mismo contrato aparece en más de un split, se conserva en un único bloque según volumetría y prioridad.

Esta última regla es relevante porque una opción puede negociarse durante varios días. Si un contrato aparece en entrenamiento y validación, el modelo podría aprender rasgos idiosincráticos del contrato y ofrecer una estimación optimista. Las Figuras 15, 16, 17 y 18 resumen la lógica temporal, la separación con lag y el control de contratos compartidos.

La búsqueda de hiperparámetros se realiza con k-folds temporales. Estos k-folds se construyen exclusivamente dentro del bloque de entrenamiento, de modo que validation y test externos quedan fuera del proceso de búsqueda. Para cada candidato se calculan MAE, RMSE y R^2 en train y validation.

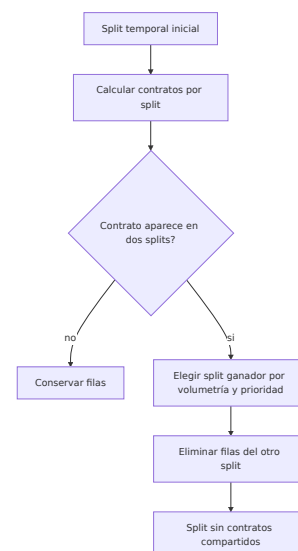


Figura 16: Control de contratos compartidos entre splits.

La selección usa una métrica compuesta:

$$CE_1 = RMSE_{val} + \alpha \cdot std(RMSE_{val}) + \beta \cdot \max(0, RMSE_{val} - RMSE_{train}). \quad (13)$$

Esta métrica penaliza error de validación, inestabilidad entre folds y gap de sobreajuste. Según la configuración del repositorio, $\alpha = 0,25$ y $\beta = 0,75$.

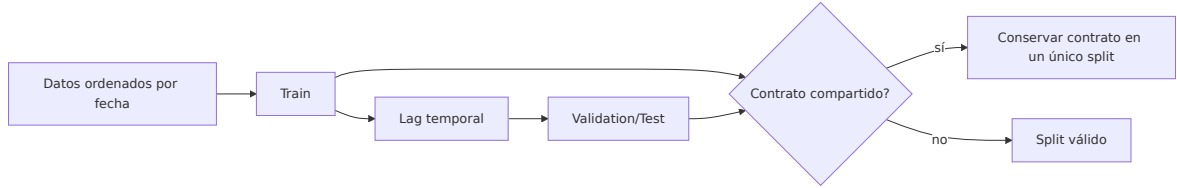


Figura 17: Control temporal y exclusividad de contratos.

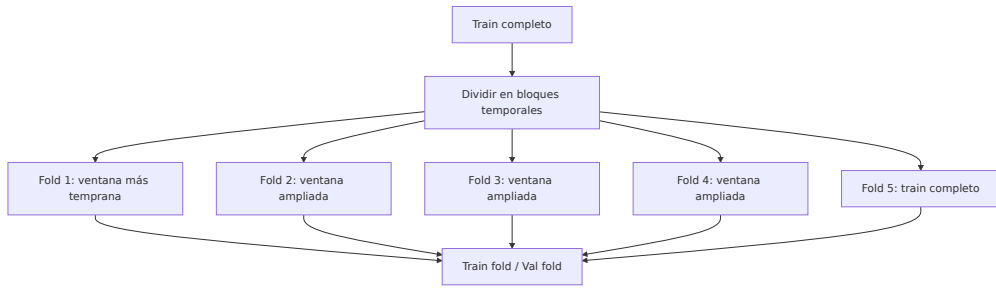


Figura 18: Construcción de k-folds temporales.

3.7 REENTRENAMIENTO Y PROGRESSIVE ATM

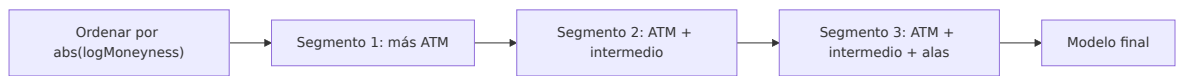


Figura 19: Entrenamiento progressive ATM por segmentos.

Después de seleccionar hiperparámetros, el modelo se reentrena en dos fases: train/validation y final test. En la fase final se ajusta con trainval y se evalúa contra test. La métrica de reentrenamiento incorpora el rendimiento observado en selección:

$$CE_2 = RMSE_{val} + \gamma \cdot CE_1 + \beta \cdot \max(0, RMSE_{val} - RMSE_{train}), \quad (14)$$

con $\gamma = 0,25$ y $\beta = 0,75$ en la configuración actual.

Es importante precisar que CE_2 se utiliza para monitorizar consistencia en la fase train/validation. La evaluación definitiva de generalización del modelo se realiza siempre sobre el bloque test fuera de muestra.

El proyecto incluye una variante progressive ATM. Ordena observaciones por $|\log(F/K)|$, de más cercano al ATM a más alejado, y divide el entrenamiento en segmentos. La idea financiera es aprender primero la zona central, habitualmente más líquida y estable, y después incorporar zonas más alejadas. En modelos no iterativos se implementa mediante pesos; en XGBoost y redes se implementa mediante fases acumulativas. Por su carácter experimental, esta variante se menciona aquí y se desarrolla en el apéndice H. Las Figuras 19 y 20 muestran el reentrenamiento del mejor candidato y los segmentos ATM.

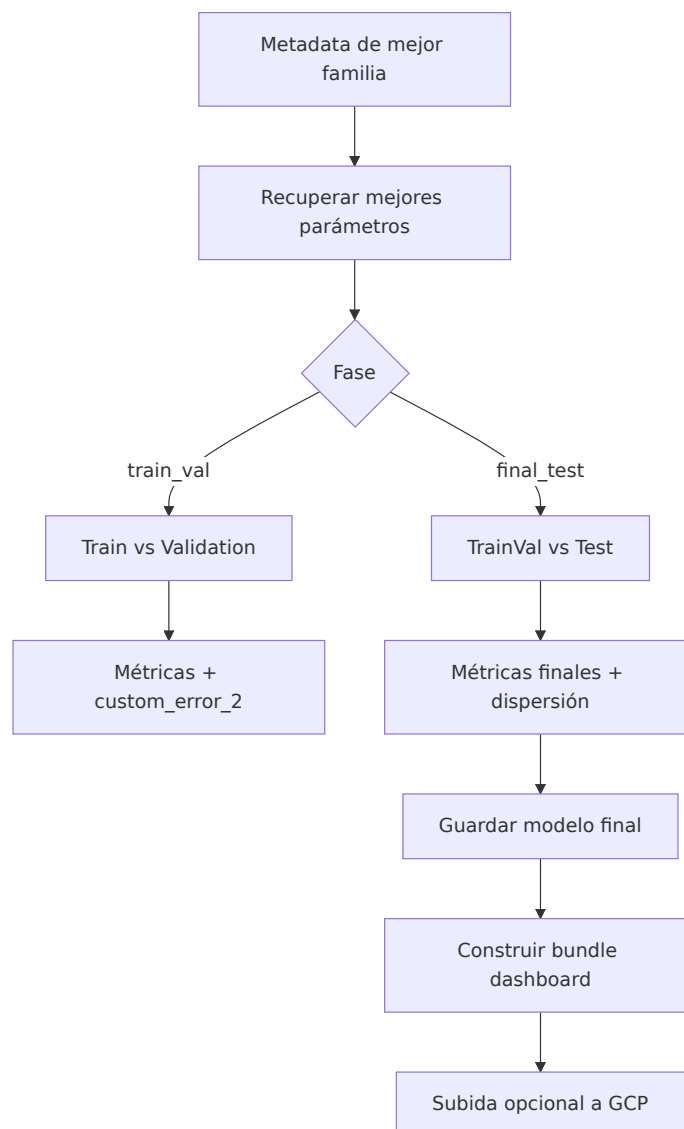


Figura 20: Reentrenamiento del mejor candidato.

3.8 CONSTRUCCIÓN DEL DASHBOARD

El dashboard no entrena modelos. Consume paquetes autocontenidos generados desde modelos finales (ver Figura 21). Cada paquete contiene dataset con predicciones, residuales y error absoluto, muestras para explicabilidad local, puntos de referencia de superficies, SHAP global y local, árboles surrogate, regresor simbólico, vecinos, superficies, ICE, ALE y diagnóstico agregado. La base teórica y operativa de SHAP, ALE y regresión simbólica se desarrolla respectivamente en los apéndice C, apéndice E y apéndice G.

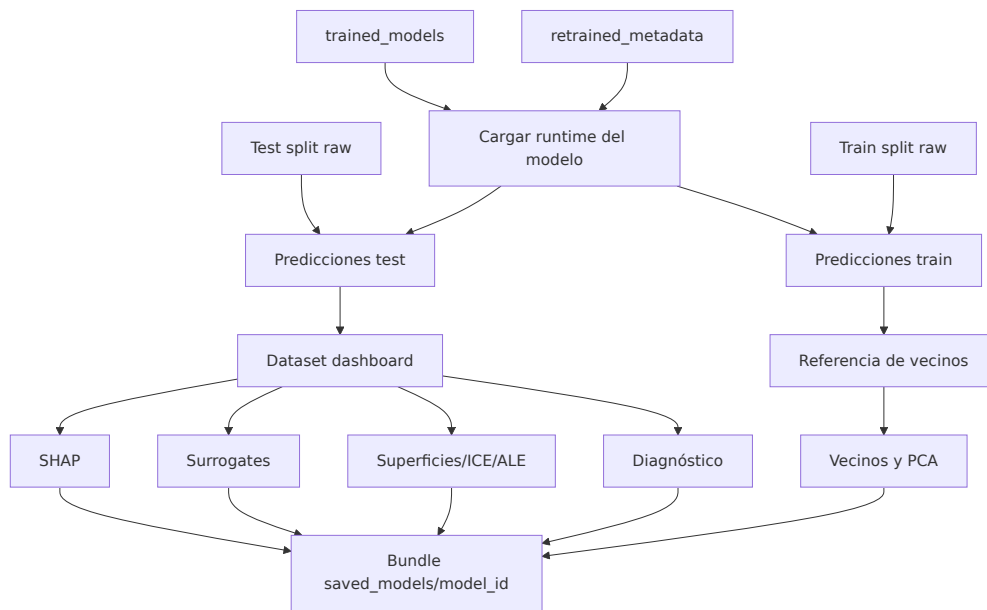


Figura 21: Conversión de modelo final a paquete visualizable.

La transformación de un modelo final a este paquete no modifica el predictor: carga el runtime, aplica el mismo feature engineering sobre train y test, calcula artefactos explicativos y persiste una carpeta autocontenida en `src/dashboard/saved_models/`.

La aplicación se estructura en cuatro pestañas, resumidas en la Tabla 3. Esta organización responde a preguntas distintas y evita confundir error predictivo con explicación.

En particular, la lectura SHAP se divide en dos niveles complementarios: **SHAP global** en la pestaña *Global Explainability* (drivers y patrones agregados) y **SHAP local** en la pestaña *Sample Explainability*, donde se muestra el *Local SHAP Waterfall* para justificar una predicción concreta. La formalización de ambas lecturas se recoge en el apéndice C.

Pestaña	Pregunta principal
Behaviour And Surface	¿Cómo responde el modelo ante cambios de money-ness, vencimiento y otras variables financieras?
Global Explainability	¿Qué variables y patrones dominan las predicciones globales?
Sample Explainability	¿Por qué una muestra concreta recibe esa volatilidad y tiene soporte histórico?
Diagnosis	¿Dónde acierta y dónde falla el modelo en la superficie?

Cuadro 3: Pestañas del dashboard y función analítica.

Las Figuras 22, 23, 24, 25 y 26 muestran estos flujos y vistas; en particular, la vista local de *Sample Explainability* es la puerta de entrada al *Local SHAP Waterfall* y al contexto de vecinos históricos.

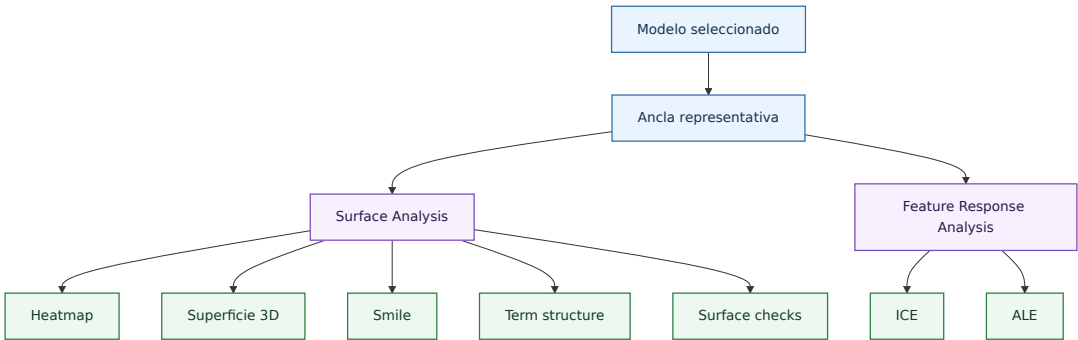


Figura 22: Flujo de la pestaña Behaviour And Surface.

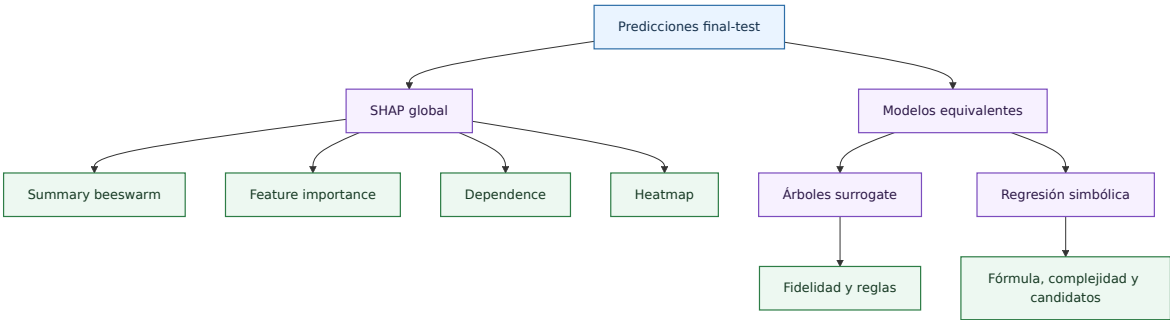


Figura 23: Flujo de la pestaña Global Explainability.

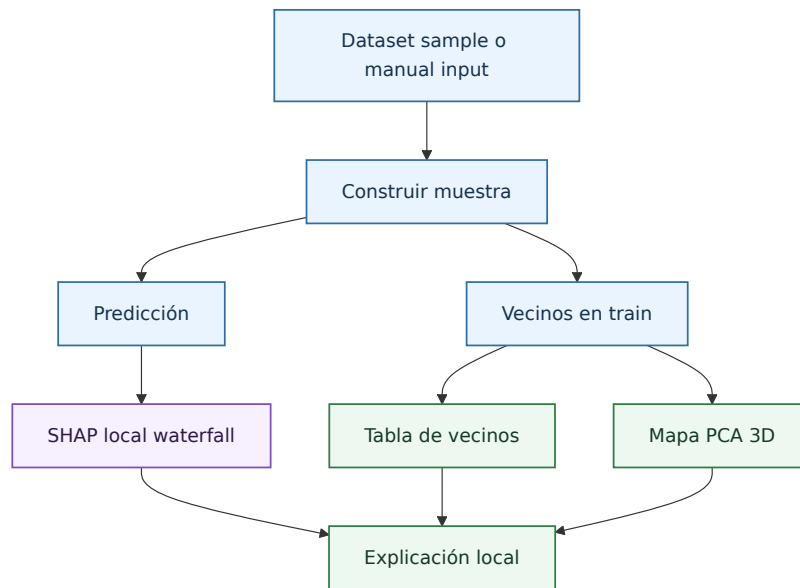


Figura 24: Flujo de la pestaña Sample Explainability.

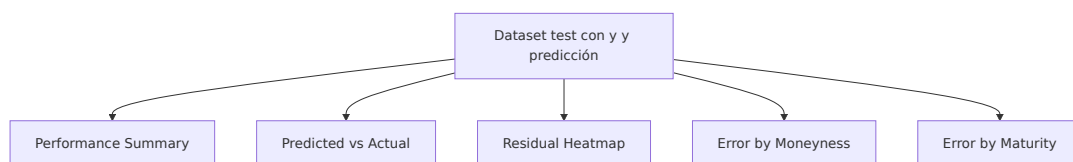


Figura 25: Vistas de diagnóstico del modelo final.

3.9 API, ALMACENAMIENTO Y CLOUD

La API se implementa con FastAPI como un servicio *ad hoc* independiente del dashboard. Puede servir cualquier modelo preparado bajo este flujo de entrenamiento, sin depender de la capa visual. En el despliegue objetivo de este trabajo, tanto la API como el dashboard se ejecutan sobre la plataforma Google Cloud Platform (GCP), concretamente en servicios de Cloud Run. El código expone endpoints de salud, predicción y explicabilidad local:

- /health
- /run_model/predict/
- /run_model/sample_explainability/

La petición incluye modelo, tipo de opción, strike, subyacente, tiempo a vencimiento, tipo de interés y, opcionalmente, código de contrato y volatilidad real.

La respuesta de predicción incluye el modelo, la volatilidad estimada y la entrada normalizada. La respuesta de explicabilidad local añade waterfall SHAP, payload de

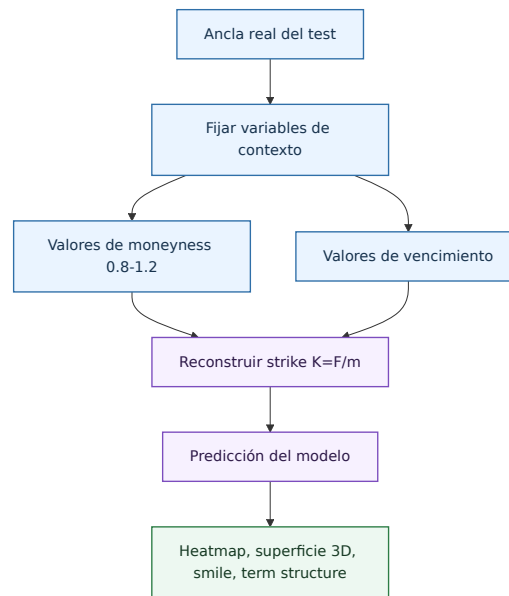


Figura 26: Construcción de superficies locales de volatilidad.

contribuciones, vecinos históricos y distancias. API y dashboard comparten modelos y paquetes autocontenidos de dashboard, lo que reduce el riesgo de explicar una versión distinta de la usada para predecir. La operativa de workflows, automatización y despliegue reproducible que soporta esta separación se desarrolla con más detalle en el apéndice B.

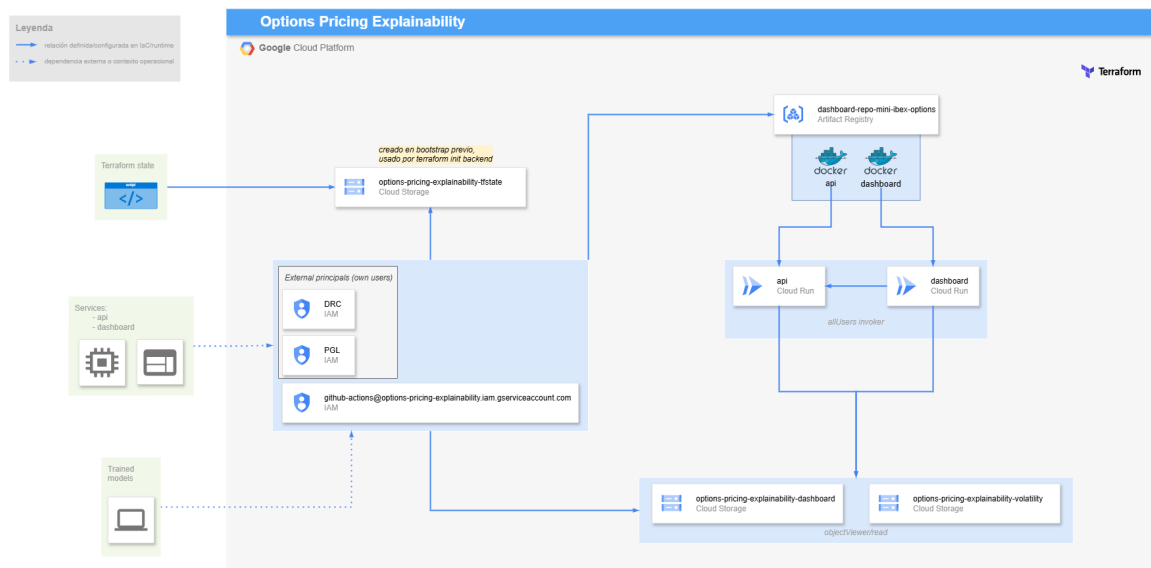


Figura 27: Arquitectura cloud del sistema: almacenamiento, elementos y servicios de API/dashboard.

El backend de almacenamiento puede ser local o GCP. En el entorno de despliegue del proyecto se utiliza GCP: Cloud Storage para modelos y paquetes de dashboard,

Artifact Registry para imágenes de contenedor, Cloud Run para API y dashboard, e IAM para permisos. Terraform define los recursos y permite reproducir la infraestructura. La parte crítica es mantener sincronizados modelo, scaler, metadatos y paquete explicativo; por eso el reentrenamiento final dispara su construcción después de guardar el modelo. La Figura 27 resume este esquema de despliegue, dejando explícita la separación entre almacenamiento, artefactos de modelo y servicios desplegados.

3.10 REPRODUCIBILIDAD

El trabajo se ha realizado siguiendo todos los pasos descritos en este documento. De la misma manera, siguiendo todos los pasos descritos en este documento junto con los notebooks de ayuda del repositorio y la [documentación de github \(epiblasfunds.github.io/options-pricing/\)](https://epiblasfunds.github.io/options-pricing/) es posible reproducir el trabajo si contamos con los datos fuente necesarios.

Lo necesario para reproducir el estudio es llevar a cabo los procesos de:

1. Procesado de datos.
2. Entrenamiento de modelos.
3. Conversión de modelos entrenados al formato adecuado para presentar en el dashboard.
4. Tener cuenta de GCP y poder desplegar el dashboard y la API.

Es importante remarcar que aunque la configuración sea la misma es posible que los resultados de entrenamiento varíen un poco y esto provoque métricas, fórmulas de regresión simbólica y árboles de decisión ligeramente distintos.

3.11 ARQUITECTURA DEL REPOSITORIO

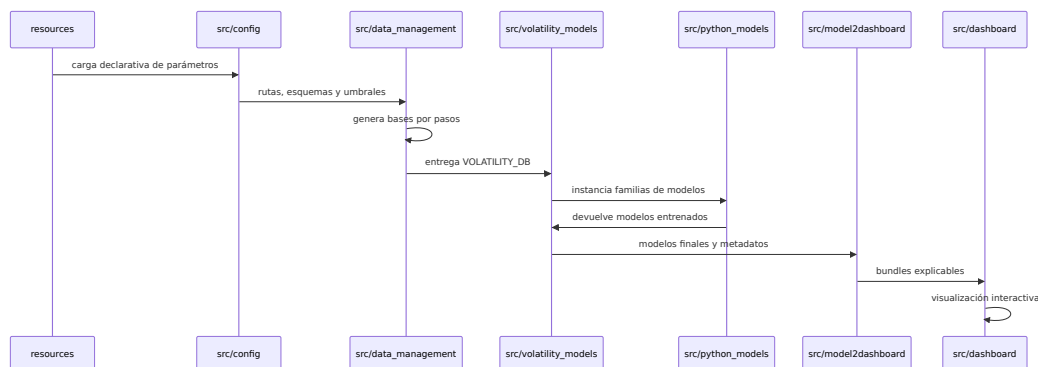


Figura 28: Secuencia de ejecución entre paquetes.

La arquitectura del repositorio sigue una separación por funcionalidades: cada paquete tiene un cometido identificable y las salidas persistidas pasan de una etapa a otra sin mezclar entrenamiento, explicación y servicio. La Figura 28 muestra la secuencia de ejecución, la cadena de salidas y las dependencias entre paquetes.

Paquete o carpeta	Responsabilidad principal
src/config	Carga de configuración, rutas absolutas, logging y objetos de configuración por dominio.
src/enums	Definición centralizada de columnas, tipos de contrato y tipos de opción.
src/exceptions	Excepciones de dominio usadas en validaciones de datos y control de errores del ETL.
src/data_management	ETL de datos: loaders, builders, validaciones y utilidades de contrato.
src/volatility_models	Preparación de datos de entrenamiento, splits, k-folds, selección y reentrenamiento.
src/python_models	Abstracciones de familias de modelos y estructuras persistentes de dashboard.
src/model2dashboard	Conversión de modelos finales en paquetes explicativos autocontenidos.
src/dashboard	Aplicación Dash, servicios, callbacks, gráficos y utilidades de validación.
src/api	API FastAPI para predicción y explicabilidad local.
resources	Configuración declarativa (datos, modelos y cliente-servidor) consumida por el runtime.
dockerfiles	Definiciones de imagen para API, dashboard, LaTeX y utilidades de documentación.
scripts	Automatizaciones auxiliares de soporte operativo y generación de artefactos.
terraform	Infraestructura cloud reproducible: buckets, registro de imágenes, Cloud Run e IAM.

Cuadro 4: Arquitectura lógica del repositorio.

La Tabla 4 muestra que el dashboard no reentrena modelos y la API no reconstruye explicaciones desde cero salvo en casos de explicabilidad local manual. Esta separación reduce el acoplamiento y facilita detectar errores. Si una predicción de API no coincide con la vista del dashboard, el problema puede acotarse a carga del paquete autocontenido, transformación de features o versión de modelo.

Vista así, la arquitectura no es solo una organización de carpetas. Es una forma de aislar responsabilidades, simplificar depuración y poder justificar cada paso del flujo de principio a fin.

Desde esa misma lógica, la arquitectura se apoya en un flujo de trabajo de ingeniería más próximo al de un proyecto real que al de un experimento ad hoc: ramas de trabajo, commits trazables, validaciones antes de integrar y control explícito de cambios sobre código, configuración, documentación y despliegue. Esa capa metodológica de desarrollo, validación y despliegue se amplía en el apéndice [B](#).

Todos estos paquetes están cubiertos por más del 80 % en cobertura de tests unitarios, haciendo que las funcionalidades sean robustas para futuras evoluciones del código, haciéndolo más escalable. A esta escalabilidad ayuda también el chequeo de formato Flake8. Para obligar a respetar tanto el formato Flake8 como el paso de los tests, se han desarrollado workflows de GitHub para que GitHub Actions ejecute en pipelines para comprobar ambas condiciones, tal como se resume de forma específica en el apéndice [B](#).

3.12 RIESGOS METODOLÓGICOS Y CONTROLES

Los principales riesgos metodológicos se anticipan desde el diseño.

1. Lookahead bias: controlado mediante orden temporal y subyacentes anteriores a la opción.
2. Data snooping: mitigado separando búsqueda de hiperparámetros, reentrenamiento y evaluación final.
3. Memorización de contratos: tratado con exclusividad de contratos entre splits.
4. Extrapolación en zonas poco cubiertas, diagnosticada con vecinos históricos y heatmaps de error.
5. Confundir explicabilidad del modelo con verdad de mercado: por ello los surrogates se presentan con métricas de fidelidad y los resultados se contrastan frente a residuales y superficies.

RESULTADOS Y DISCUSIÓN

4.1 RESULTADOS

4.1.1 Comparativa general de modelos

Modelo	Entrenamiento	MAE test	RMSE test	R^2 test
Regresión lineal	estándar	0.067585	0.183824	0.096059
Random Forest	estándar	0.054366	0.088403	0.790941
XGBoost	estándar	0.054604	0.091735	0.774885
Red neuronal secuencial	estándar	0.055425	0.113506	0.655354
Quantum Inspired NN	estándar	0.056056	0.143025	0.452787

Cuadro 5: Comparación final de modelos.

La Tabla 5 muestra que las familias de árboles dominan el test. Random Forest estándar obtiene el menor RMSE y el mayor R^2 ; XGBoost estándar queda cerca, pero con RMSE ligeramente superior. Las redes neuronales no superan a las familias de árboles en esta muestra, aunque la variante Quantum Inspired mejora claramente al aplicar progressive ATM frente a su versión estándar.

4.1.2 Entrenamiento estándar frente a progressive ATM

El entrenamiento progressive ATM no produce una mejora universal. En Random Forest, que ya muestra un comportamiento muy estable en la comparación, el cambio es casi neutro; en XGBoost y en la red secuencial empeora el RMSE test; en Quantum Inspired sí actúa como regularizador útil. Esto encaja con la interpretación metodológica: ordenar por cercanía a ATM puede estabilizar arquitecturas iterativas más sensibles, pero también puede sesgar el aprendizaje si los extremos de moneyness contienen información necesaria para test. Por tanto, progressive ATM debe tratarse como variante experimental y no como decisión automática.

Familia	RMSE	RMSE ATM	Δ	Lectura
Regresión lineal	0.183824	0.187747	+0.003923	Empeora levemente.
Random Forest	0.088403	0.088533	+0.000130	Prácticamente neutro.
XGBoost	0.091735	0.113606	+0.021871	Degrada test.
Red neuronal secuencial	0.113506	0.120161	+0.006655	Degrada test.
Quantum Inspired NN	0.143025	0.117404	-0.025621	Mejora clara.

Cuadro 6: Impacto agregado de progressive ATM frente a entrenamiento estándar.

La evidencia sugiere una lectura por familias.

- En **Random Forest**, la proximidad de resultados indica que el bosque ya capta bien la señal central sin necesitar entrenamiento progresivo ATM.
- En **XGBoost** y en la **red secuencial**, el deterioro sugiere que priorizar ATM puede dejar peor representadas zonas relevantes para la generalización final.
- En **Quantum Inspired**, la mejora apunta a que la estrategia sí puede actuar como regularizador estructural cuando la arquitectura es más sensible al orden de aprendizaje.

La conclusión práctica es sobria: progressive ATM es útil como hipótesis de trabajo y como herramienta de análisis, pero su activación debe justificarse por evidencia empírica en cada familia concreta. Se puede ver más en el apéndice [H](#).

4.1.3 Modelo de referencia

El modelo de referencia es el XGBoost:

- Desempeño en test muy notable quedando muy cercano al mejor desempeño entre todas las familias.
- Los modelos de regresión simbólica y árboles de decisión consiguen explicar el modelo mejor que al resto de familias: $RMSE = 0,0378$, $MAE = 0,0219$, $R^2 = 0,9264$. La formulación e interpretación de la regresión simbólica como modelo sustituto se desarrollan en el apéndice [G](#).

4.1.4 Diagnóstico de rendimiento

Las vistas del dashboard son necesarias para localizar dónde se concentra el error. El scatter *predicted vs actual* debe confirmar si los extremos altos y bajos se acercan

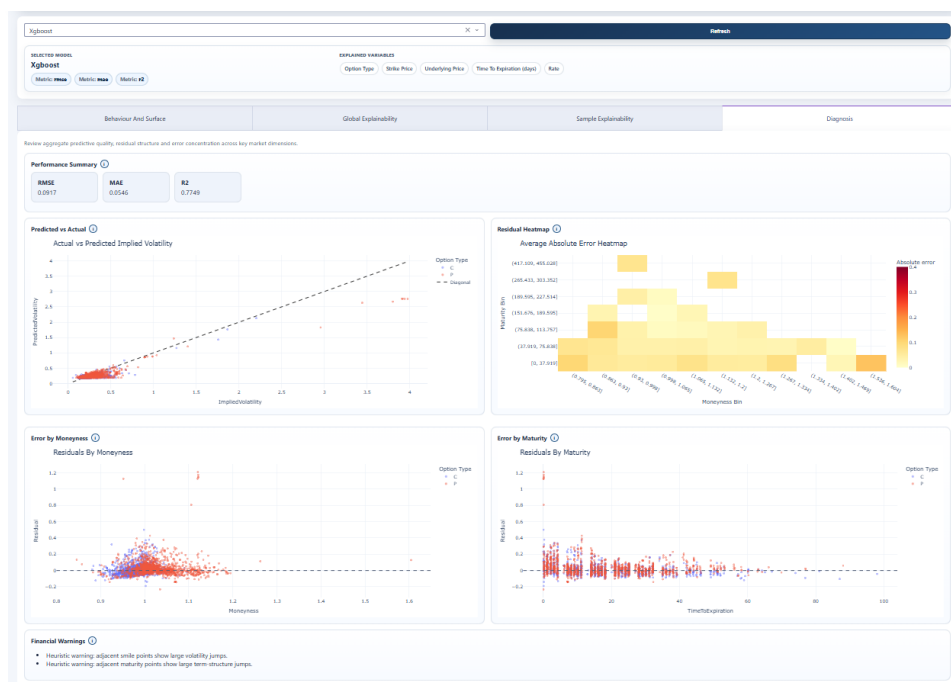


Figura 29: *Diagnosis en XGBoost.*

a la media; el residual heatmap debe separar error por moneyness y vencimiento; y las cajas *Error by Moneyness* y *Error by Maturity* deben indicar si los extremos de moneyness o los vencimientos cortos concentran el fallo. La conclusión accionable es que el modelo es competitivo agregadamente, pero su uso debe acompañarse de diagnóstico por zonas de la superficie cuando se consulten contratos poco líquidos o alejados de ATM.

La Figura 29 sintetiza este diagnóstico y permite verificar visualmente si existe compresión hacia la media en los extremos de la superficie.

4.1.4.1 *Análisis del impacto del entrenamiento Progressive ATM*

El proyecto incluyó una variante experimental denominada Progressive ATM (o *Entrenamiento Progresivo Ordenado por ATM*): un enfoque de aprendizaje que reordena cronológicamente el entrenamiento para presentar primero observaciones cercanas al ATM (moneyness bajo, $|\log(F/K)| < 0,05$), donde se concentra la máxima liquidez y menor ruido, para después incorporar progresivamente observaciones hacia extremos de moneyness y vencimientos cortos. La motivación financiera es doble:

1. La zona ATM en opciones sobre índices suele ser la más líquida y menos afectada por sesgos de selección.

2. Entrenar primero en datos menos ruidosos podría regularizar arquitecturas sensibles al orden de presentación, como redes neuronales secuenciales y modelos cuánticos inspirados.

Esta lectura se complementa con el Apéndice [H](#).

Metodología de evaluación. Se reentrenaron independientemente los cinco modelos de las familias seleccionadas bajo esta variante, conservando la validación cruzada 5-fold y los mismos hiperparámetros que en el entrenamiento estándar. Se compararon las métricas test de RMSE, MAE y R^2 entre ambas versiones (estándar vs. ATM) para cada familia. El análisis se complementó con inspección visual de residual heatmaps en el dashboard para verificar si la reordenación producía cambios en la distribución espacial del error (por moneyness y vencimiento).

Síntesis interpretativa del impacto ATM por arquitectura. Los resultados revelan un patrón sistemático: la técnica Progressive ATM no produce un modelo mejor de forma universal, sino que su efectividad depende críticamente de la arquitectura subyacente.

- Modelos ya óptimos (Random Forest, y en menor medida XGBoost base): El entrenamiento progresivo ATM no mejora; en XGBoost incluso resulta perjudicial. La razón es que estos modelos ya poseen mecanismos de regularización o agregación intrínsecos (bootstrap, boosting) que no se benefician de reordenamientos externos. Forzar un orden artificial puede introducir sesgo de inicialización que degrada capacidad global de la arquitectura, particularmente en modelos iterativos como boosting.
- Modelos de baja capacidad o convergencia frágil (Sequential NN, Quantum Inspired): El entrenamiento progresivo ATM produce mejoras tangibles. Estos modelos carecen de regularización inherente y son sensibles a inicialización y orden de muestras. Entrenar primero en región ATM establece trayectoria de aprendizaje más robusta, evitando convergencia a mínimos pobres que es frecuente cuando se enfrentan datos heterogéneos desde el inicio.
- Modelos lineales (Linear Regression): El orden es irrelevante puesto que la hipótesis es global. No hay beneficio de regularización local.

4.1.5 *Behaviour And Surface*

El modelo final debe leerse como función financiera, no solo como tabla de errores. Las superficies generadas alrededor de puntos de referencia del test permiten inspeccionar $\hat{\sigma}(m, T)$, smiles y estructuras temporales.

La lectura recomendada es usar al menos tres puntos de referencia: una observación ATM líquida, una observación en extremos de moneyness y una observación de vencimiento corto. Si el dashboard muestra saltos alrededor de un punto, la ex-

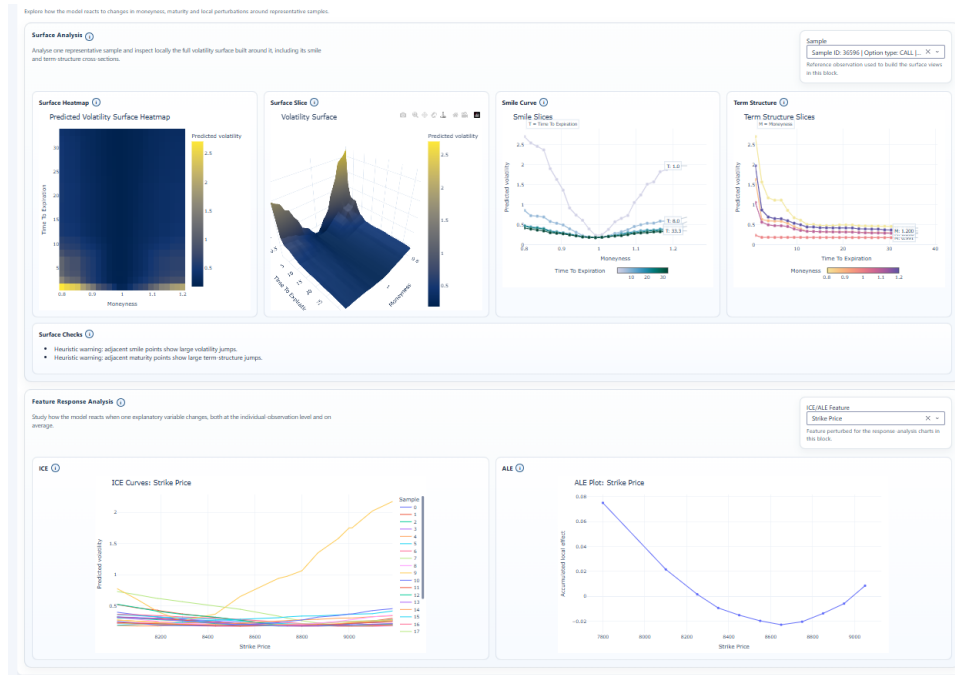


Figura 30: *Behaviour And Surface* de XGBoost.

plicación debe cruzarse con vecinos y residual heatmap antes de atribuirlo a señal financiera. Si el smile es suave y el error local bajo, la predicción es más defendible.

La Figura 30 muestra que las curvas smile y term-structure de XGBoost mantienen forma financiera interpretable (mínimo en entorno ATM y descenso de volatilidad al aumentar vencimiento para moneyness fijo).

4.1.6 Explicabilidad global

La explicación global debe comprobar que los drivers principales son coherentes con la geometría de volatilidad: moneyness, vencimiento, subyacente, strike, tipo y tipo de opción. SHAP se calcula sobre variables raw visibles y el runtime reconstruye las features transformadas, por lo que una contribución de StrikePrice o UnderlyingPrice debe interpretarse en relación con moneyness y no como efecto causal aislado. La base formal de esta descomposición y sus limitaciones interpretativas se desarrollan en el apéndice C.

La lectura correcta combina ranking, signo local y dependence plots. Si una variable domina el ranking pero tiene colores mezclados o contribuciones con mucha dispersión, el modelo está usando interacciones. Esa conclusión es razonable en opciones, porque strike, subyacente y vencimiento no son independientes. Si aparecen saltos en dependence plots, deben contrastarse con las superficies y con diagnóstico por zonas de la superficie.

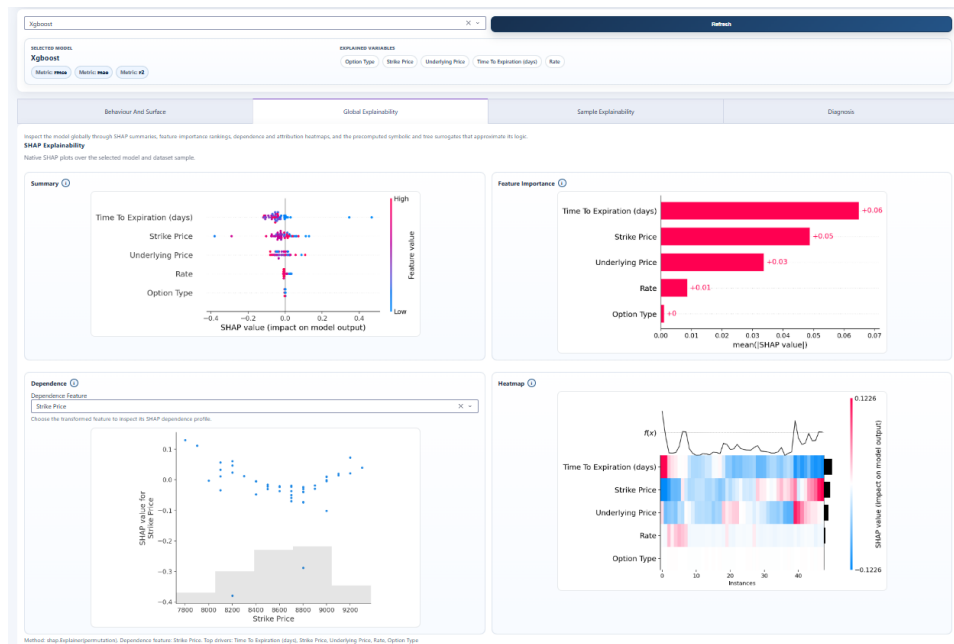


Figura 31: SHAP global XGBoost.

En las cinco variantes revisadas, y para lo que se visualiza en dashboard, la jerarquía global aparece estable: Time To Expiration (days) como driver principal y Strike Price como segundo, seguidas por Underlying Price; Rate y Option Type tienen peso menor. La Figura 31 hace visible esta regularidad, que coincide con la lectura financiera esperada de superficie. El punto importante es que la consistencia entre modelos se observa en el orden de importancia, no necesariamente en la magnitud de impacto: Quantum Inspired y Sequential NN muestran dispersión SHAP más amplia para algunos rangos, coherente con su mayor sensibilidad local en otras pestañas.

4.1.7 Modelos equivalentes: árboles y regresión simbólica

Modelo	RMSE simbólico	MAE simbólico	R^2 simbólico
Regresión lineal	0.0244	0.0135	0.7585
Quantum Inspired NN	0.1120	0.0277	0.5518
Random Forest	339.9547	3.5845	-5,173,729.5872
Sequential NN	0.0376	0.0316	0.8880
XGBoost	0.0378	0.0219	0.9264

Cuadro 7: Fidelidad de regresión simbólica reportada en dashboard por familia de modelo.

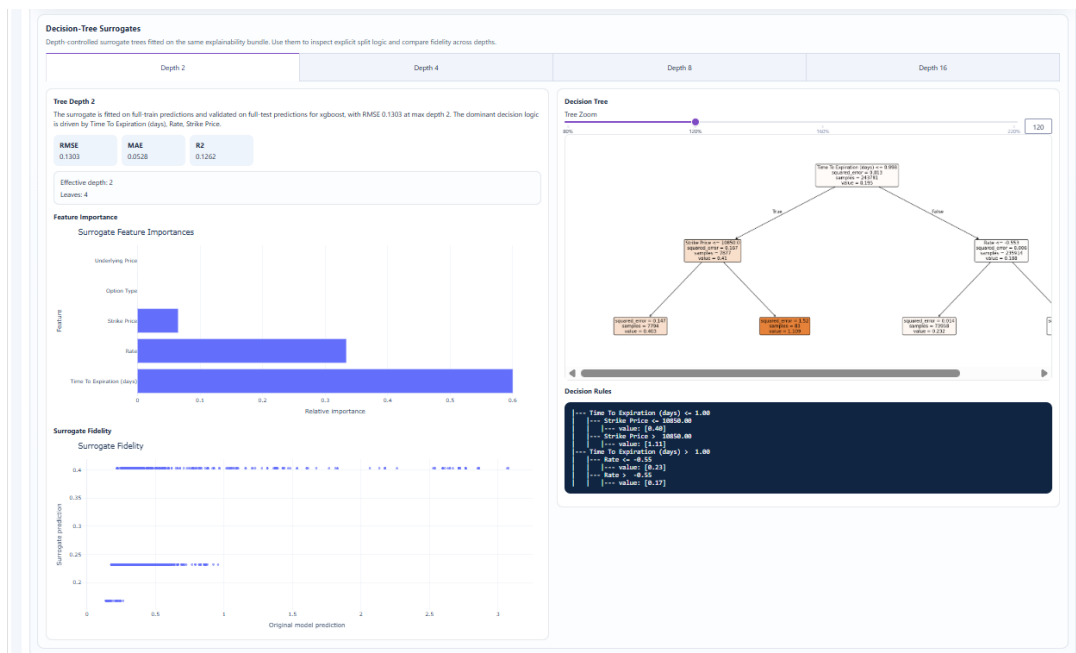


Figura 32: Árboles surrogate por profundidad en XGBoost.

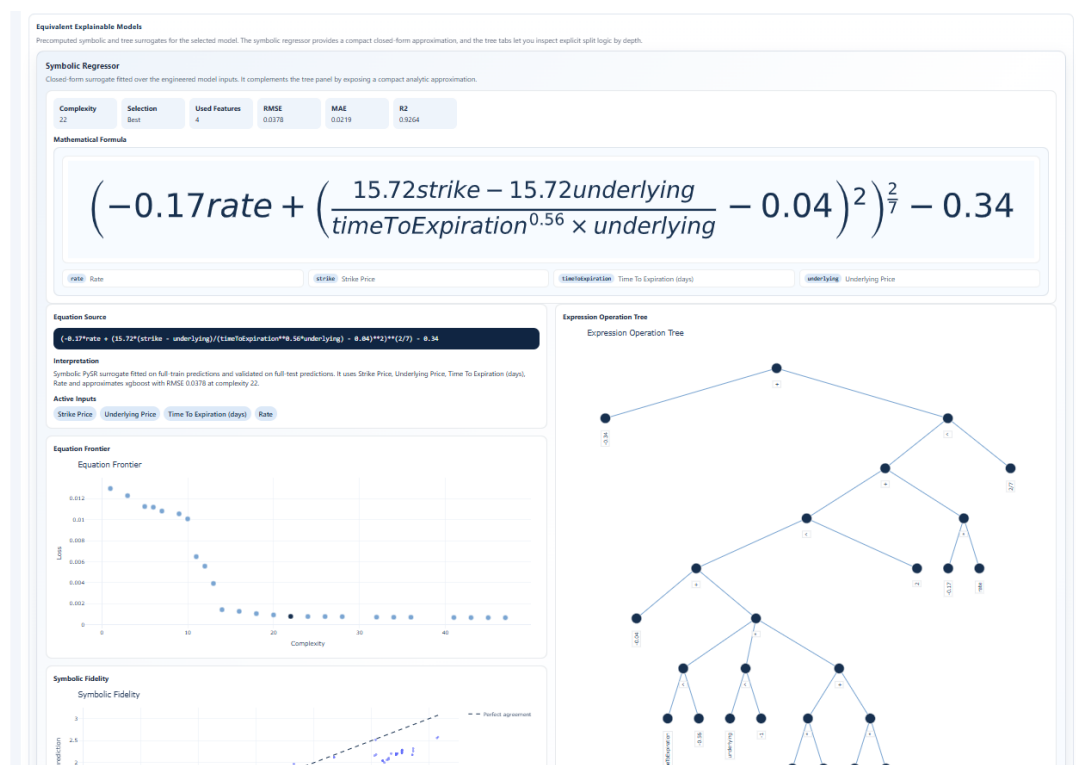


Figura 33: Regresión simbólica del XGBoost.

Los modelos equivalentes explican el predictor, no la verdad de mercado. La regresión simbólica debe leerse aquí exactamente en ese sentido de surrogate y fidelidad, como se detalla en el apéndice G.

La fidelidad de los árboles es limitada: ni siquiera profundidades altas reproducen bien el comportamiento completo del XGBoost. Esto no invalida el modelo principal, pero sí limita el uso de reglas simples como explicación global. La regresión simbólica (ver Tabla 8) generada para este paquete muestra métricas no utilizables para defensa del Random Forest ni del Quantum Inspired NN, con RMSE de fidelidad extremadamente alto; por tanto, el modelo más fiable que obtenemos es el XGBoost.

La Figura 32 y la Figura 33, junto con la Tabla 8 y la Tabla 7, permiten una conclusión transversal: la facilidad para explicar un modelo mediante reglas o fórmulas compactas no coincide necesariamente con su capacidad predictiva final en test.

Profundidad	RMSE fidelidad	MAE fidelidad	R^2 fidelidad
2	0.1303	0.0528	0.1262
4	0.1266	0.0504	0.1750
8	0.1325	0.0566	0.0963
16	0.1290	0.0434	0.1438

Cuadro 8: Fidelidad de árboles surrogate frente al XGBoost.

4.1.8 Explicabilidad local y soporte histórico

El análisis local debe seleccionar una muestra representativa y una muestra problemática. Para cada una, el waterfall SHAP explica la predicción como suma de valor base y contribuciones. Los vecinos históricos, calculados contra train en el espacio transformado y estandarizado, indican si la muestra está dentro de una región conocida. En el paquete se guardan hasta 200 vecinos por muestra, usando una referencia de entrenamiento de hasta 20.000 observaciones. La lógica matemática de SHAP y la semántica de valor base y contribuciones locales se desarrollan en el apéndice C.

Las Figuras 34 y 35 recogen esta lectura en modo *dataset* para la misma observación, separando la explicación local y el contexto de soporte histórico.

La interpretación profesional debe combinar ambas piezas. Si una muestra tiene waterfall coherente y vecinos cercanos con volatilidades similares, la explicación es sólida. Si el waterfall parece razonable pero los vecinos están lejos, el modelo puede estar extrapolando; en ese caso, la predicción debe revisarse con diagnóstico por zonas de la superficie y superficie local. Así una explicación local es más defendible cuando se cumplen tres condiciones:

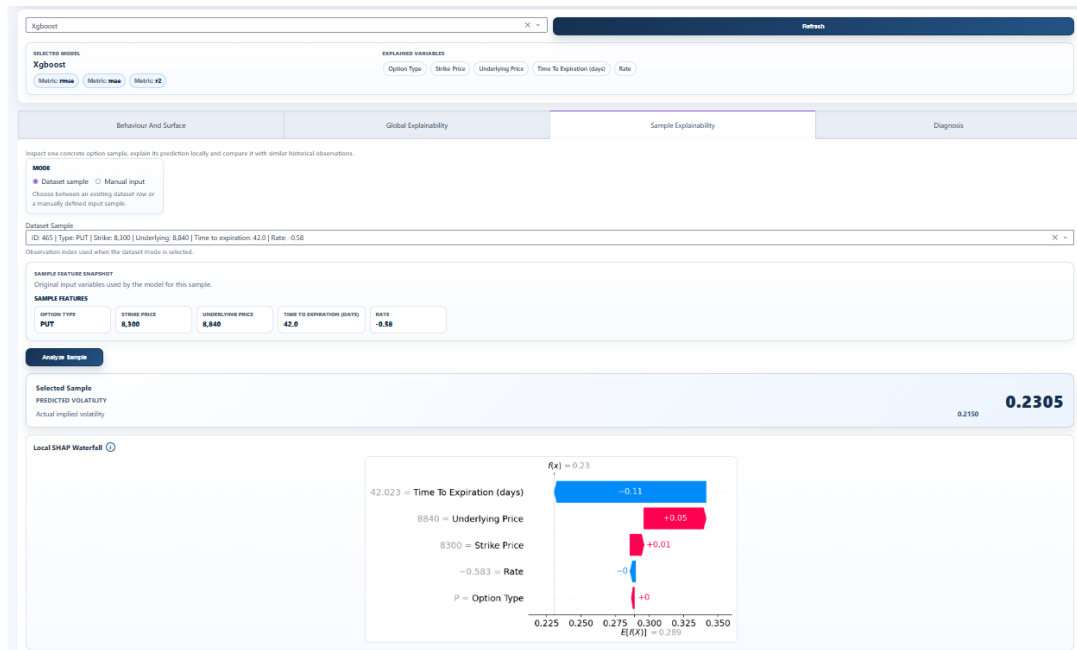


Figura 34: (Parte 1) Explicabilidad local en modo *dataset* para la misma muestra (ID 465) con XGBoost.



Figura 35: (Parte 2) Explicabilidad local en modo *dataset* para la misma muestra (ID 465) con XGBoost.

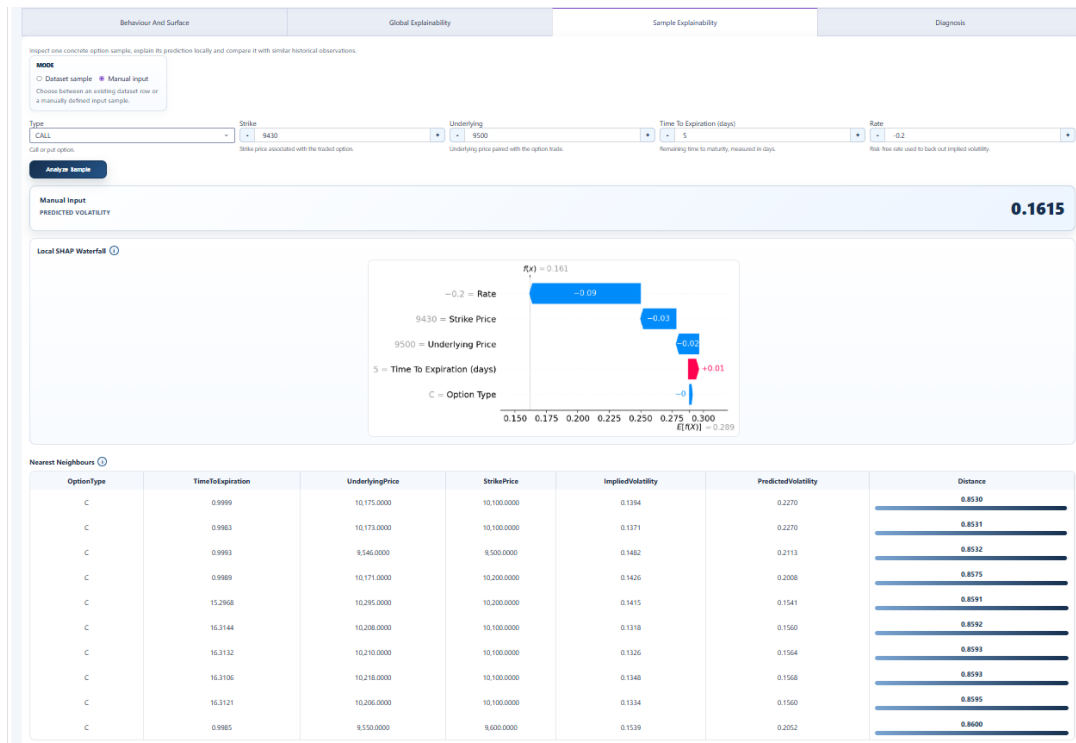


Figura 36: Explicabilidad local en modo *manual input* (stress corto vencimiento): contraste de coherencia local y soporte histórico en ambos finalistas.

La Figura 36 ilustra este contraste en modo *manual input* bajo un escenario de estrés en vencimiento corto.

- El signo y magnitud de las contribuciones SHAP son compatibles con la intuición financiera del caso.
- Los vecinos cercanos muestran contratos, moneyness y vencimientos realmente parecidos.
- El error por zonas del modelo no es anómalamente alto en esa área.

Cuando falla alguna de esas tres condiciones, la lectura debe cambiar: la explicación sigue diciendo cómo razona el modelo, pero deja de ser una justificación sólida de que la predicción sea fiable en términos de mercado.

4.2 APLICACIONES OPERATIVAS Y VALIDACIÓN INDEPENDIENTE

Este apartado conecta los resultados del dashboard con usos profesionales concretos en valoración, control de riesgos y gobierno de modelos. El objetivo no es extrapolar más allá de la evidencia disponible, sino traducir lo observado en reglas de uso defendibles para un entorno de mesa de negociación y model validation. El detalle visual de sensibilidad por regiones se amplía en el Apéndice I.

4.2.1 Validación independiente de modelos internos

El modelo final puede utilizarse como referencia independiente para contrastar superficies internas de valoración. En la práctica, la validación no se limita a comparar un único precio, sino que contrasta consistencia por regiones de moneyness y vencimiento. Un desacople localizado entre el pricer interno y la referencia empírica puede señalar problemas de parametrización, de datos de entrada o de régimen de mercado no recogido por el enfoque clásico.

- **Nivel agregado:** contraste de RMSE/MAE y de dispersión global frente a un benchmark interno.
- **Nivel por superficie:** contraste de residual heatmap y cajas de error por moneyness/vencimiento para detectar discrepancias localizadas.
- **Nivel de caso:** uso de SHAP local y vecinos para justificar por qué una diferencia concreta es esperable o requiere revisión.

Este enfoque es especialmente útil cuando el área de validación necesita una segunda opinión cuantitativa que no comparta la misma familia paramétrica que el modelo interno de producción.

4.2.2 Interpolación y suavizado: alcance real del enfoque

La superficie se aprende de forma empírica a partir de observaciones de mercado mediante modelos supervisados. Por tanto, la “interpolación” se produce de forma implícita a través de la función aprendida $\hat{\sigma}(m, T, \dots)$ y se evalúa visualmente con las vistas de *Behaviour And Surface* y *Diagnosis*. El desglose más específico de sensibilidad por regiones de moneyness y vencimiento se recoge en el apéndice I.

La suavidad no se da por supuesta: se verifica. Si aparecen escalones locales o concentraciones de error en zonas concretas, el resultado se interpreta como limitación del modelo en esa región, no como propiedad financiera del mercado. Esta distinción es clave para no confundir capacidad predictiva local con una estructura de superficie económicamente robusta.

4.2.3 Detección de anomalías de precio y errores de mercado

La detección de anomalías se construye en dos capas complementarias. La primera está en el ETL, con controles de calidad (precios y cantidades válidas, coherencia temporal opción-subyacente, resolución numérica estable de volatilidad implícita). La segunda está en diagnóstico posterior, donde los residuales por zona de superficie permiten localizar trades atípicos en extremos de moneyness o vencimientos cortos.

El detalle estructural del ETL está desarrollado en el apéndice [A](#) y la lectura regional del error en el apéndice [I](#).

En términos operativos, una anomalía no se define solo por error alto puntual, sino por contexto: error alto junto con baja densidad de vecinos, geometría local inestable o incoherencia entre explicación local y patrón global. Esta lectura mejora la utilidad del modelo como filtro de revisión manual.

4.2.4 Valoración de ilíquidas, control de mesa de negociación, auditoría y cobertura

Las aplicaciones más exigentes se apoyan en la misma secuencia de lectura: diagnóstico por zonas, contraste funcional de superficie y justificación local. Con ese flujo, el modelo aporta soporte en cuatro frentes conectados:

- **Valoración de opciones ilíquidas o poco negociadas:** cuando no hay suficiente señal transaccional directa, la predicción basada en superficie y vecinos históricos ofrece una referencia empírica utilizable, con cautela reforzada en extremos.
- **Control de riesgos de mesa de derivados:** la lectura por moneyness y vencimiento permite identificar dónde el error puede contaminar griegas, escenarios o límites de riesgo más que la media global.
- **Explicación ante auditoría interna/model validation:** SHAP global/local, trazabilidad de pipeline y evidencias por superficie permiten defender decisiones de modelo con base reproducible.
- **Soporte a decisiones de cobertura:** la geometría smile/term-structure más el diagnóstico local aportan contexto para priorizar coberturas en zonas de mayor sensibilidad o menor soporte histórico.

4.3 DISCUSIÓN

La pregunta planteada en [section 1.1](#) puede responderse afirmativamente con matices. Sí es posible construir un modelo útil y consultable para volatilidad implícita de opciones del IBEX. El matiz es que la explicación no puede descansar en un único resumen global: la fidelidad de modelos equivalentes simples es limitada, y la defensa técnica debe combinar SHAP, superficies, ICE/ALE, vecinos y diagnóstico. Los fundamentos de SHAP, ALE y regresión simbólica que sostienen esta lectura se amplían en los apéndice [C](#), apéndice [E](#) y apéndice [G](#).

4.3.1 *Lectura financiera*

Financieramente, el proyecto trata la volatilidad como superficie y no como serie unidimensional. El buen RMSE agregado es relevante, pero el dato más importante para gestión de riesgos es dónde se produce el error. La compresión de volatilidades extremas sugiere que deben revisarse los extremos de moneyness y los vencimientos cortos con más cuidado. Este comportamiento es plausible en derivados: esas zonas suelen tener menos liquidez, más ruido de microestructura y mayor sensibilidad al desfase entre opción y futuro subyacente.

- El modelo final es útil para lectura agregada y consulta, pero no debe tratarse como atajo automático del juicio financiero en zonas extremas.
- La geometría de la superficie sigue siendo el criterio principal de plausibilidad: smiles, term structures y suavidad local importan tanto como el error medio.
- La lectura de vecinos y soporte histórico es especialmente valiosa cuando se analiza un contrato poco líquido o lejano de ATM.

En otras palabras, el trabajo refuerza una idea clásica en derivados: el resultado útil no es un único número, sino una predicción situada dentro de una estructura de superficie, de un contexto de mercado y de una evidencia histórica comparable.

También conviene subrayar una implicación práctica. Cuando el modelo se usa como apoyo a valoración o revisión manual, la pregunta correcta no es solo si “acierta de media”, sino si conserva lógica financiera en la zona concreta que se está consultando. Esa exigencia explica por qué el documento dedica espacio a smiles, extremos de moneyness, vencimientos cortos y soporte local, y no solo a tablas globales de error. Esta lectura aplicada se concreta en la Sección 4.2 y se refuerza con el apéndice de sensibilidad por regiones de superficie (I).

Aportación frente a enfoques clásicos.

La comparación con enfoques clásicos debe plantearse como complementariedad y mejora operativa, no como sustitución automática. Los enfoques paramétricos aportan estructura financiera explícita y son valiosos para gobernanza. La propuesta de este trabajo añade valor en escenarios donde esa estructura no basta por sí sola:

- **Mayor flexibilidad empírica:** captura interacciones no lineales entre moneyness, vencimiento, subyacente y tipo con menor rigidez funcional.
- **Mejor lectura por zonas:** añade diagnóstico de error por región de superficie, clave para contratos ilíquidos o extremos de moneyness.
- **Validación cruzada de pricing:** permite contrastar salidas de modelos clásicos internos con una referencia empírica independiente.

4.3.2 Limitaciones

Las limitaciones se agrupan en cuatro bloques.

1. La volatilidad implícita depende de la calidad del enlace opción-subyacente y del solver Black-76.
2. El conjunto de datos es heterogéneo en liquidez y cobertura de money-ness/vencimiento, por lo que los extremos deben diagnosticarse localmente. No toda región del espacio de entrada tiene la misma densidad histórica ni la misma calidad de soporte.
3. La explicabilidad aproxima comportamientos del modelo y no sustituye validación económica. Un modelo explicable sigue pudiendo estar equivocado si aprende sesgos sistemáticos de los datos.
4. El despliegue cloud exige disciplina de versionado para evitar inconsistencias entre modelos, paquetes de dashboard y metadatos.

CONCLUSIONES

Este proyecto desarrolla una cadena integral para estimar volatilidad implícita en opciones del IBEX y convertir el resultado en un sistema explicable y operable. El trabajo parte de datos de mercado, construye una base de volatilidad mediante inversión de Black-76, entrena familias de modelos bajo validación temporal y genera explicaciones y diagnósticos. Ese recorrido queda documentado de forma principal en el [chapter 3](#) y se evalúa empíricamente en el [chapter 4](#).

La primera conclusión es que la calidad de un modelo de volatilidad no puede evaluarse solo con métricas agregadas. En derivados, la forma de la superficie, el comportamiento por moneyness y vencimiento y la localización del error son elementos centrales. Por ello el dashboard no se plantea como una capa estética, sino como parte del mecanismo de validación. Esta lectura por regiones se desarrolla de forma específica en el apéndice [I](#).

La segunda conclusión es que la explicabilidad aporta valor en dos planos. En ingeniería, ayuda a detectar fallos, fugas, sesgos y discontinuidades. En finanzas, permite justificar predicciones y analizar si el modelo se comporta de forma compatible con intuición económica y revisión profesional. Los desarrollos técnicos que sostienen esa capa explicativa aparecen en los apéndice [C](#), apéndice [E](#) y apéndice [G](#).

La tercera conclusión es que la arquitectura de software condiciona la utilidad del modelo. Al separar ETL, entrenamiento, paquetes explicativos, API, dashboard y almacenamiento, el proyecto facilita reproducibilidad, mantenimiento y despliegue. Esta separación también reduce el riesgo de que las explicaciones se calculen sobre versiones distintas de las usadas en predicción. La estructura del ETL se detalla en el apéndice [A](#) y la metodología de desarrollo, validación y despliegue en el apéndice [B](#).

La cuarta conclusión es que un entrenamiento progresivo ATM (entrenar en un inicio con datos más ATM y conforme avance el entrenamiento ir incorporando datos menos ATM) no debe presentarse como mejora universal. Su utilidad depende de si estabiliza la zona ATM sin degradar los extremos de moneyness ni los vencimientos menos representados, como se desarrolla en el apéndice [H](#).

Desde la perspectiva financiera aplicada, la contribución diferencial frente a enfoques clásicos es doble.

1. Aporta una referencia empírica flexible para valoración y validación en zonas con menor liquidez o cobertura irregular.
2. Incorpora mecanismos de explicación y control (SHAP, ICE/ALE, vecinos y diagnóstico por superficie) que permiten trasladar el modelo a situaciones operativas reconocibles: contrastar precios internos, revisar anomalías, priorizar coberturas y leer el riesgo por regiones de moneyness y vencimiento. Los desarrollos técnicos correspondientes se amplían en los apéndice [C](#), apéndice [E](#) y apéndice [I](#).

El trabajo es a su vez una herramienta de apoyo a valoración, validación independiente y control de riesgos, con una trazabilidad explícita entre resultados, discusión y desarrollo anexo.

Por ello, el avance de este trabajo no debe leerse solo como una mejora de RMSE. Debe leerse como mejora de **capacidad operativa**: detectar anomalías, justificar decisiones, analizar sensibilidad por moneyness/vencimiento y usar el modelo como herramienta de control de riesgos de mesa.

Líneas Futuras

Las líneas futuras planteadas:

- Complementar la explicabilidad ya incorporada con técnicas no incluidas en este trabajo, como Integrated Gradients o LIME, para contrastar mejor la lectura local.
- Explorar modelos secuenciales como CNN 1D, LSTM o GRU si la muestra se reformula como ventanas temporales por contrato.
- Comparar el enfoque actual con modelos híbridos que impongan estructura financiera explícita sin perder flexibilidad empírica.



ESQUEMA DETALLADO DE DATOS Y ETL

Este apéndice reúne el detalle estructural de las tablas intermedias y finales generadas por el pipeline de datos. Su objetivo no es introducir lógica nueva, sino dejar trazabilidad precisa sobre qué base crea cada etapa del ETL y cuál es el detalle de sus campos en Read Raw Step, Merge Raw Step, Product Split Step, Underlying Step y Volatility Step; las Figuras 37 a 41 presentan, por etapa, la base generada y la estructura de sus campos.

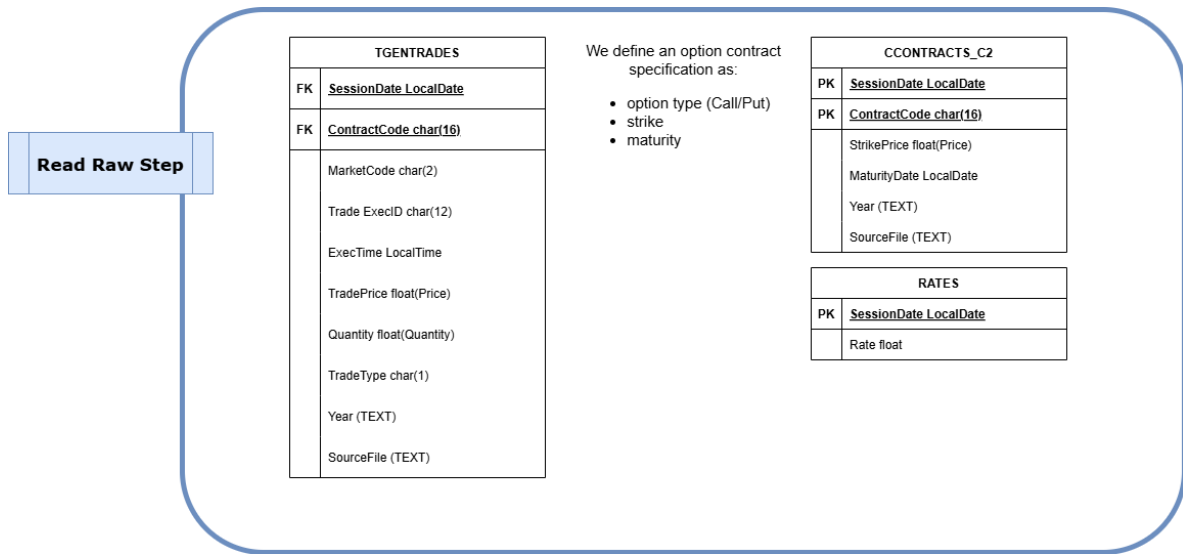


Figura 37: Detalle estructural del ETL: Read Raw Step.

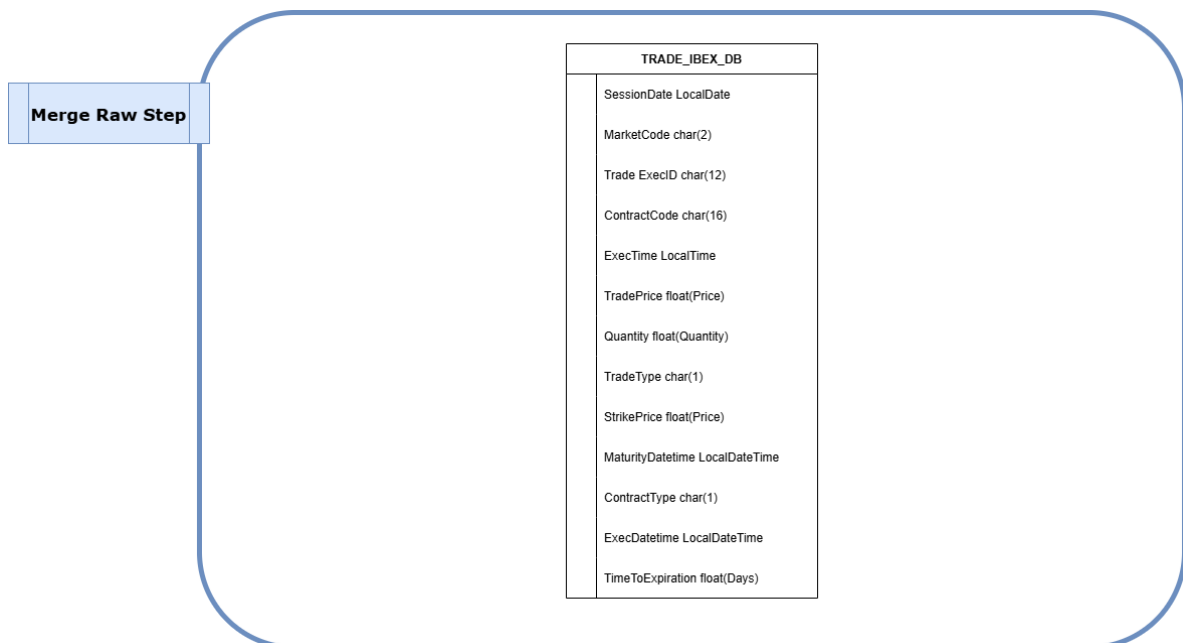


Figura 38: Detalle estructural del ETL: Merge Raw Step.

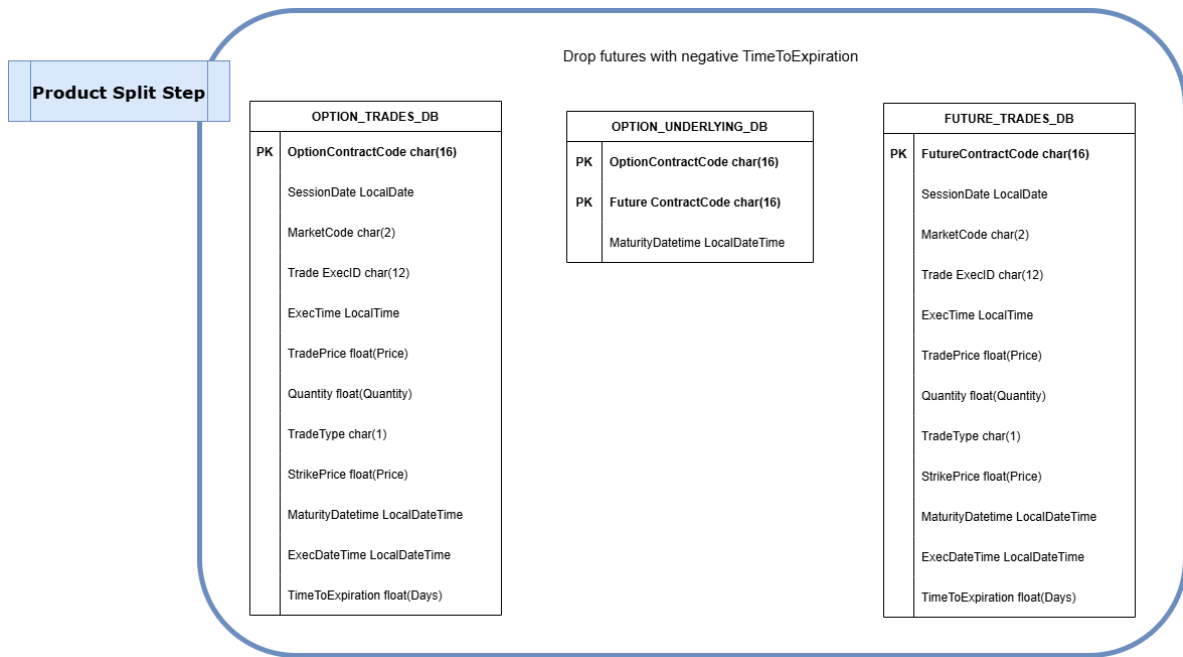


Figura 39: Detalle estructural del ETL: Product Split Step.

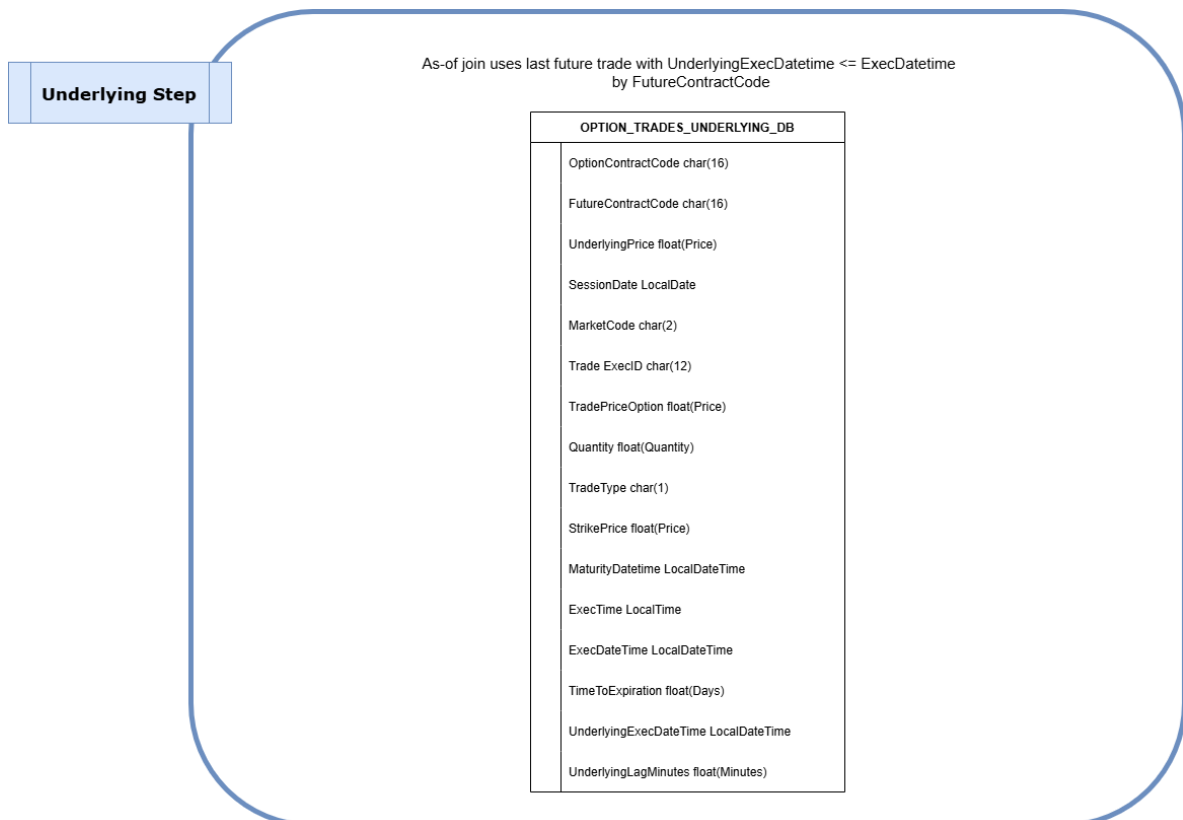


Figura 40: Detalle estructural del ETL: Underlying Step.

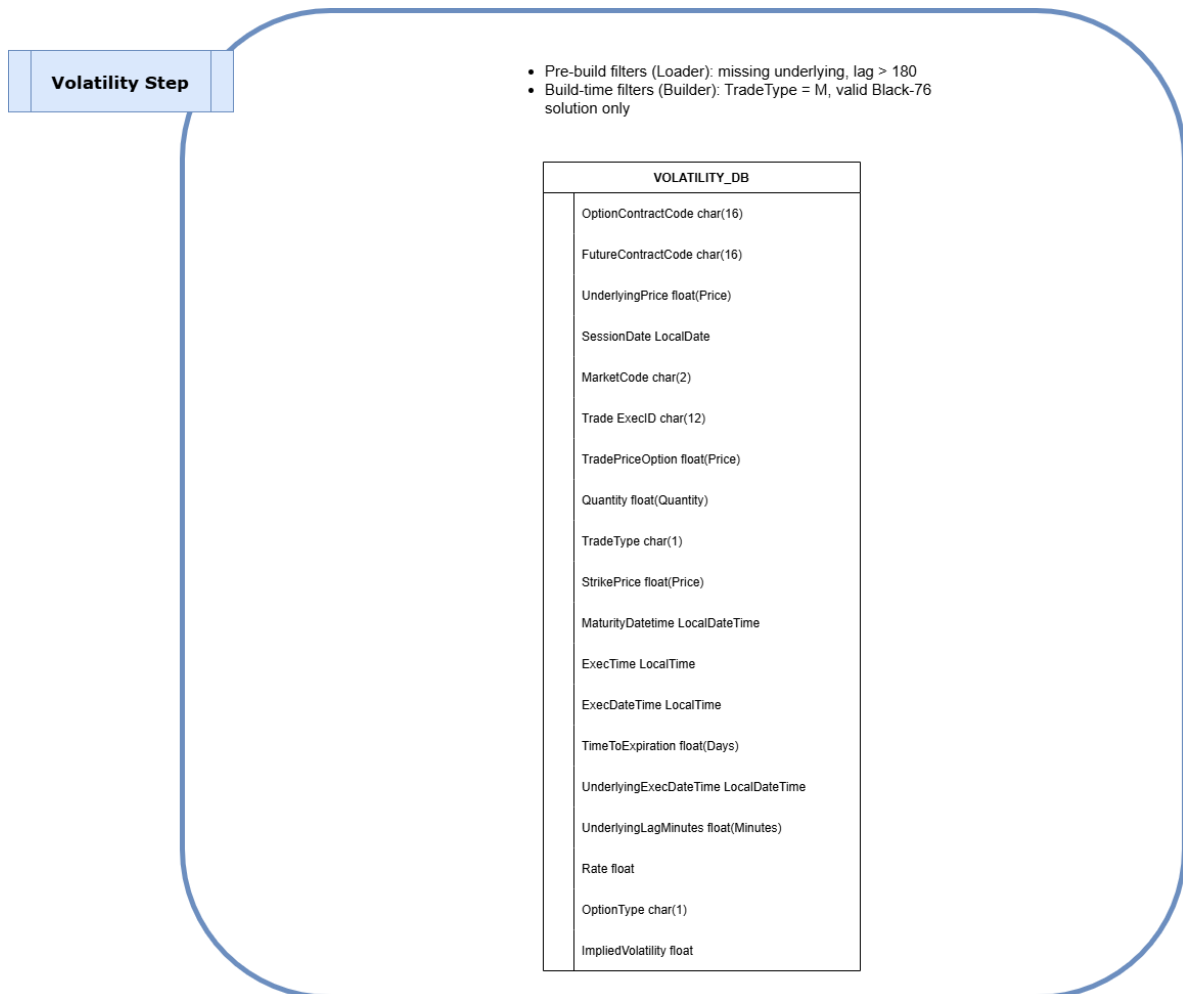


Figura 41: Detalle estructural del ETL: Volatility Step.

METODOLOGÍA DE DESARROLLO, VALIDACIÓN Y DESPLIEGUE

Una parte relevante del valor metodológico del proyecto es que no se ha construido como un prototipo monolítico difícil de auditar, sino como un repositorio vivo gestionado mediante prácticas habituales de ingeniería de software. El desarrollo se ha realizado sobre Git y GitHub, con versionado explícito del código, la configuración, la documentación, los artefactos de infraestructura y los componentes desplegables del sistema. Esta organización permite reconstruir la evolución del proyecto y asociar cada resultado técnico a una versión concreta del repositorio.

El flujo de trabajo se apoya en ramas de desarrollo, integración mediante *pull requests* y automatización con GitHub Actions. De este modo, los cambios no se incorporan únicamente por modificación directa del código, sino a través de un proceso controlado donde cada entrega puede ser validada antes de integrarse en la rama principal. Esta práctica resulta especialmente importante en un proyecto que combina extracción y tratamiento de datos financieros, entrenamiento de modelos, explicabilidad, dashboard, API e infraestructura cloud, ya que una modificación local en cualquiera de estos bloques puede afectar silenciosamente al comportamiento global del sistema.

Para reforzar este control, el repositorio incorpora un workflow específico de validación sobre *pull requests*. Dicho workflow instala el entorno Python del proyecto, ejecuta comprobaciones estáticas con `flake8` y lanza la batería de tests mediante `pytest`. Esto permite detectar errores de estilo, problemas de integración y regresiones funcionales antes de aceptar cambios en la rama principal. Además, el proyecto mantiene una cobertura de tests superior al 80 %, lo que aporta una garantía adicional sobre la estabilidad de los módulos principales y reduce el riesgo de introducir modificaciones no controladas en piezas críticas del sistema.

La cobertura de tests no debe interpretarse como una garantía absoluta de corrección, pero sí como una evidencia de madurez metodológica. En este trabajo, las pruebas automatizadas cubren componentes relevantes del repositorio y se complementan con validaciones manuales sobre los artefactos generados, los flujos de datos,

el dashboard y la API. Esta combinación aproxima el desarrollo del proyecto al ciclo de vida de un proyecto real: implementar una mejora, verificar su comportamiento, integrarla de forma controlada y conservar una traza reproducible de los cambios realizados.

Junto al workflow de validación, el proyecto incluye automatización específica para la documentación. El workflow de despliegue de documentación construye el sitio generado con MkDocs y lo publica en GitHub Pages a partir de la configuración definida en `../doc/mkdocs.yml`. Esto permite que la documentación técnica evolucione junto con el código y que los cambios incorporados al repositorio puedan reflejarse de manera sistemática en una versión navegable del proyecto. La documentación deja así de ser un artefacto estático y separado, y pasa a formar parte del propio proceso de integración.

El repositorio también automatiza el despliegue de infraestructura cloud mediante Terraform. El workflow de infraestructura carga la configuración del proyecto desde `resources/cloud_config.json`, prepara las variables necesarias para Terraform, autentica contra Google Cloud, inicializa el backend remoto de estado en Google Cloud Storage y ejecuta las fases de `plan` y `apply`. Este proceso permite definir la infraestructura como código, mantener trazabilidad sobre los recursos cloud utilizados y reducir la dependencia de configuraciones manuales no documentadas. Además, el uso de un backend remoto para el estado de Terraform facilita la consistencia entre ejecuciones y evita que el estado de la infraestructura dependa de una máquina local concreta.

Una vez preparada la infraestructura base, el sistema incorpora un workflow reutilizable para construir y desplegar los componentes ejecutables del proyecto: el dashboard y la API. Este workflow construye las imágenes Docker correspondientes a partir de los `Dockerfile` definidos para cada servicio, las publica en Artifact Registry y aplica la configuración necesaria para su despliegue en Cloud Run mediante Terraform. De esta forma, el paso desde código fuente hasta servicio desplegable queda formalizado en una cadena reproducible: construir imagen, publicar artefacto, planificar infraestructura y aplicar cambios.

Esta separación entre workflows tiene una finalidad metodológica clara. El workflow de *pull request* protege la calidad del código antes de la integración; el workflow de documentación mantiene actualizada la documentación técnica; el workflow de infraestructura gestiona los recursos cloud de base; y el workflow de despliegue de dashboard y API materializa los servicios finales. Cada automatización cubre una responsabilidad distinta, evitando mezclar validación, documentación, provisión de infraestructura y despliegue aplicativo en un único proceso opaco.

En este contexto, operaciones como `commit`, `pull`, `push`, *pull request*, `plan` o `apply` no son únicamente acciones operativas. Funcionan como mecanismos de trazabilidad metodológica. Un resultado experimental o una funcionalidad del sistema no

queda defendida solo por su salida final, sino por la versión concreta del código, la configuración, los tests y la infraestructura que la acompañan. Esta trazabilidad es especialmente relevante en un proyecto financiero, donde pequeñas variaciones en el pipeline de datos, en la configuración de entrenamiento, en los modelos o en la capa de explicación pueden modificar los resultados obtenidos.

Por tanto, el uso de Git, GitHub Actions, tests automatizados, control estático de código, documentación desplegable, Docker, Terraform y Google Cloud no constituye un elemento accesorio de implementación. Forma parte de la metodología del trabajo: asegura reproducibilidad, facilita el control de cambios, reduce regresiones, documenta la evolución del sistema y permite justificar el resultado final no solo por lo que predice o visualiza, sino por cómo se ha construido, validado, integrado y desplegado.

FUNDAMENTOS DE SHAP

SHAP, acrónimo de *SHapley Additive exPlanations*, es una metodología de explicabilidad basada en los valores de Shapley de la teoría de juegos cooperativos. Su objetivo es asignar a cada variable de entrada una contribución numérica a una predicción concreta del modelo. En lugar de interpretar el modelo de forma global mediante sus parámetros internos, SHAP interpreta localmente una predicción como la suma de un valor base y una serie de aportaciones individuales asociadas a las variables explicativas.

La idea central es considerar una predicción del modelo como el resultado de una “coalición” de variables. Algunas variables están presentes y aportan información; otras se consideran ausentes y su efecto se sustituye por una referencia esperada. El problema consiste entonces en repartir de forma justa la diferencia entre la predicción concreta y el valor medio de referencia entre las variables que intervienen en la predicción.

C.1 MODELO ADITIVO DE EXPLICACIÓN

SHAP pertenece a la familia de métodos de explicación aditiva. Para una observación x , la predicción del modelo $f(x)$ se aproxima o descompone como

$$f(x) = \phi_0 + \sum_{j=1}^p \phi_j(x), \quad (15)$$

donde p es el número de variables explicativas, ϕ_0 representa el valor base de la predicción y $\phi_j(x)$ es la contribución atribuida a la variable j para la observación x . En muchas aplicaciones, ϕ_0 se interpreta como el valor esperado del modelo sobre una distribución de referencia:

$$\phi_0 = \mathbb{E}[f(X)]. \quad (16)$$

Por tanto, cada valor SHAP mide cuánto desplaza una variable la predicción respecto a ese valor base. Si $\phi_j(x) > 0$, la variable j empuja la predicción por encima de la referencia. Si $\phi_j(x) < 0$, la empuja por debajo. La suma de todas las contribuciones reconstruye la predicción del modelo para la observación analizada.

En el contexto de este proyecto, donde el objetivo es explicar predicciones de volatilidad implícita, la lectura natural es la siguiente: el valor base representa una volatilidad de referencia estimada por el modelo, mientras que cada valor SHAP indica cuánto contribuyen variables como el precio del subyacente, el strike, el tiempo a vencimiento o el tipo de interés a desplazar la predicción final hacia arriba o hacia abajo.

C.2 VALORES DE SHAPLEY

El fundamento matemático de SHAP procede de los valores de Shapley. En teoría de juegos cooperativos, se considera un conjunto de jugadores $N = \{1, \dots, p\}$ y una función de valor $v(S)$, que asigna a cada subconjunto de jugadores $S \subseteq N$ el valor que obtiene esa coalición. El objetivo es repartir el valor total $v(N)$ entre los jugadores de forma coherente con su contribución marginal media.

Trasladado a explicabilidad de modelos, los jugadores son las variables del modelo y la función $v(S)$ mide el valor esperado de la predicción cuando se conoce únicamente el subconjunto de variables S . Para una variable j , su valor de Shapley se define como

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} [v(S \cup \{j\}) - v(S)]. \quad (17)$$

El término

$$v(S \cup \{j\}) - v(S) \quad (18)$$

representa la contribución marginal de la variable j cuando se añade a una coalición S que todavía no la contiene. Como esta contribución puede depender fuertemente de qué otras variables estén ya presentes, el valor de Shapley no toma una única contribución marginal, sino una media ponderada sobre todos los subconjuntos posibles.

El peso

$$\frac{|S|!(p - |S| - 1)!}{p!} \quad (19)$$

tiene una interpretación combinatoria. Representa la proporción de órdenes posibles de entrada de las variables en los que el subconjunto S aparece antes que la variable j . De esta forma, el valor de Shapley puede entenderse como la contribución marginal esperada de una variable cuando las variables se incorporan al modelo en todos los órdenes posibles.

Esta construcción evita depender de un único orden arbitrario de incorporación de variables. Por ejemplo, en un modelo financiero, el efecto atribuido al *strike* puede depender de si el precio del subyacente ya se ha considerado o no. SHAP promedia estas situaciones y asigna a cada variable una contribución que tiene en cuenta todas las posibles coaliciones.

C.3 FUNCIÓN DE VALOR EN EXPLICABILIDAD DE MODELOS

Para aplicar los valores de Shapley a modelos predictivos es necesario definir la función de valor $v(S)$. Una formulación habitual consiste en definir

$$v(S) = \mathbb{E} [f(X) \mid X_S = x_S], \quad (20)$$

donde X_S representa el subconjunto de variables incluidas en S , y x_S son los valores observados de esas variables para la instancia que se quiere explicar.

Esta expresión indica que, cuando solo se conocen las variables de S , la predicción se calcula como el valor esperado del modelo condicionando a que esas variables toman los valores observados. Las variables que no pertenecen a S se integran o sustituyen mediante una distribución de referencia.

En la práctica, esta definición plantea una dificultad importante: no siempre es sencillo estimar la distribución condicional de las variables ausentes. Por ello, diferentes explicadores SHAP utilizan aproximaciones distintas. Algunos métodos asumen independencia entre variables, otros usan muestras de fondo, otros aprovechan la estructura interna del modelo, como ocurre con árboles de decisión, y otros estiman las contribuciones mediante permutaciones.

Esta elección no es un detalle menor. Cuando las variables están correlacionadas, la forma de tratar las variables ausentes afecta a la atribución final. Por ejemplo, en opciones financieras, variables como el precio del subyacente, el *strike* y el *money*ness no son independientes en sentido económico. Por tanto, los valores SHAP deben interpretarse como atribuciones del modelo bajo una definición concreta de ausencia de variables, no como efectos causales puros.

C.4 PROPIEDADES DE LOS VALORES SHAP

Una de las razones por las que SHAP es ampliamente utilizado es que hereda propiedades teóricas deseables de los valores de Shapley. Entre las más relevantes destacan la eficiencia, la simetría, la nulidad y la aditividad.

La propiedad de eficiencia establece que la suma de las contribuciones asignadas a las variables coincide con la diferencia entre la predicción de la observación y el valor base:

$$\sum_{j=1}^p \phi_j(x) = f(x) - \phi_0. \quad (21)$$

Equivalentemente,

$$f(x) = \phi_0 + \sum_{j=1}^p \phi_j(x). \quad (22)$$

Esta propiedad también se conoce en el contexto de SHAP como *local accuracy*. Garantiza que la explicación local no solo ordena variables por importancia, sino que reconstruye numéricamente la predicción explicada.

La propiedad de simetría indica que si dos variables aportan exactamente lo mismo a todas las coaliciones posibles, entonces deben recibir la misma contribución. Formalmente, si para dos variables i y j se cumple que

$$v(S \cup \{i\}) = v(S \cup \{j\}) \quad \text{para todo } S \subseteq N \setminus \{i, j\}, \quad (23)$$

entonces

$$\phi_i = \phi_j. \quad (24)$$

La propiedad de nulidad establece que una variable que no cambia el valor de ninguna coalición debe recibir contribución cero. Si

$$v(S \cup \{j\}) = v(S) \quad \text{para todo } S \subseteq N \setminus \{j\}, \quad (25)$$

entonces

$$\phi_j = 0. \quad (26)$$

Finalmente, la propiedad de aditividad indica que si una función de valor puede escribirse como suma de dos funciones,

$$v(S) = v_1(S) + v_2(S), \quad (27)$$

entonces los valores de Shapley asociados también se suman:

$$\phi_j(v) = \phi_j(v_1) + \phi_j(v_2). \quad (28)$$

Estas propiedades hacen que SHAP proporcione una asignación consistente de contribuciones bajo el marco matemático elegido. No obstante, la consistencia formal de la atribución no debe confundirse con causalidad económica ni con validación del modelo.

C.5 INTERPRETACIÓN LOCAL Y GLOBAL

SHAP es, en primer lugar, una técnica de explicación local. Para una observación concreta x_i , produce un vector de contribuciones

$$\phi(x_i) = (\phi_1(x_i), \dots, \phi_p(x_i)), \quad (29)$$

que explica cómo se pasa del valor base ϕ_0 a la predicción individual $f(x_i)$. Esta lectura es especialmente útil cuando se quiere justificar por qué un contrato concreto recibe una determinada predicción de volatilidad.

A partir de las explicaciones locales también pueden construirse medidas globales. Una medida habitual de importancia global de la variable j es la media del valor absoluto de sus contribuciones SHAP sobre un conjunto de observaciones:

$$I_j = \frac{1}{n} \sum_{i=1}^n |\phi_j(x_i)|. \quad (30)$$

Esta magnitud no indica si una variable aumenta o reduce la predicción en promedio, sino cuánto contribuye, en valor absoluto, a modificar las predicciones del modelo. Por ello, debe interpretarse como una medida de intensidad explicativa, no como un coeficiente de regresión ni como una elasticidad financiera.

Además, el signo y la magnitud de $\phi_j(x_i)$ pueden variar entre observaciones. Una misma variable puede aumentar la predicción en una zona del espacio de datos y reducirla en otra. Esta característica resulta especialmente relevante en modelos no lineales, donde el efecto aprendido por el modelo puede depender del nivel de *monotonicity*, del vencimiento o de la región de mercado considerada.

C.6 APROXIMACIÓN MEDIANTE PERMUTACIONES

El cálculo exacto de los valores de Shapley requiere evaluar todos los subconjuntos posibles de variables. Como existen 2^p subconjuntos, el coste computacional crece exponencialmente con el número de variables. Para modelos con muchas variables o predicciones costosas, el cálculo exacto puede ser inviable.

Una forma de aproximar los valores SHAP consiste en estimar las contribuciones marginales mediante permutaciones. En lugar de enumerar explícitamente todos los subconjuntos, se generan distintos órdenes de entrada de las variables. Para cada permutación, se mide cuánto cambia la predicción al incorporar una variable después de las que la preceden en ese orden. Promediando estas contribuciones marginales sobre varias permutaciones, se obtiene una estimación del valor de Shapley.

Sea π una permutación de las variables y sea P_j^π el conjunto de variables que aparecen antes que j en dicha permutación. La contribución marginal de j bajo el orden π puede escribirse como

$$\Delta_j^\pi = v(P_j^\pi \cup \{j\}) - v(P_j^\pi). \quad (31)$$

El valor de Shapley puede interpretarse entonces como la esperanza de esta contribución marginal sobre todas las permutaciones posibles:

$$\phi_j = \mathbb{E}_\pi [\Delta_j^\pi]. \quad (32)$$

En la práctica, esta esperanza se aproxima usando un número finito de permutaciones. Esta aproximación tiene la ventaja de ser independiente de la familia del modelo, por lo que puede aplicarse de forma homogénea a redes neuronales, modelos de árboles, regresiones u otros estimadores. Su principal inconveniente es que puede ser más costosa computacionalmente que explicadores especializados diseñados para una arquitectura concreta.

En este proyecto se utiliza un explicador de permutación mediante `shap.Explainer(..., algorithm="permutation")`. Esta decisión prioriza una semántica común entre modelos heterogéneos frente a la eficiencia máxima que podría obtenerse con explicadores específicos. El presupuesto mínimo de evaluaciones por observación depende

del número de variables p , y en la implementación empleada se controla mediante el parámetro

$$\text{max_evals} \geq 2p + 1. \quad (33)$$

C.7 VARIABLES DE FONDO Y VALOR BASE

El cálculo de SHAP requiere una referencia frente a la cual medir las contribuciones. Esta referencia se construye mediante un conjunto de fondo o *background dataset*. Dicho conjunto representa la distribución de datos que el explicador utiliza para sustituir o integrar las variables ausentes.

El valor base ϕ_0 depende de esta distribución de fondo. Por ello, cambiar el conjunto de referencia puede cambiar las explicaciones, incluso si el modelo predictivo no cambia. En términos prácticos, explicar una predicción respecto a una muestra representativa de todo el mercado no es necesariamente equivalente a explicarla respecto a una muestra restringida a opciones de corto vencimiento o a una región concreta de moneyness.

En este proyecto, el conjunto de fondo se controla mediante el parámetro `shap_background_size`. Las observaciones usadas para generar explicaciones globales se controlan mediante `shap_explain_size`, mientras que las explicaciones locales de contratos concretos se calculan sobre muestras seleccionadas mediante `sample_option_size`. Esta parametrización permite equilibrar coste computacional, estabilidad de las explicaciones y representatividad de la muestra explicada.

C.8 LIMITACIONES INTERPRETATIVAS

Aunque SHAP proporciona una descomposición matemáticamente rigurosa de la predicción del modelo, su interpretación debe hacerse con cautela.

En primer lugar, SHAP no identifica causalidad. Un valor SHAP positivo para una variable indica que, bajo el modelo entrenado y la definición de referencia utilizada, esa variable contribuye a elevar la predicción respecto al valor base. No implica que modificar causalmente esa variable produciría necesariamente el mismo cambio en el mercado real.

En segundo lugar, SHAP explica el comportamiento del modelo, no la verdad del fenómeno financiero subyacente. Si el modelo ha aprendido relaciones espurias, sesgos de muestra o patrones inestables, SHAP puede describir esas relaciones con precisión sin que por ello sean económicamente válidas.

En tercer lugar, las correlaciones entre variables pueden afectar a la atribución. En problemas financieros, muchas variables no son independientes: el strike, el precio del subyacente, el moneyness y el tiempo a vencimiento pueden presentar dependencias estructurales. Por tanto, la asignación de contribuciones debe entenderse dentro de la aproximación concreta utilizada por el explicador.

Finalmente, una explicación local detallada no equivale a una garantía de calidad predictiva. Un gráfico SHAP puede mostrar de forma clara cómo el modelo reparte una predicción entre variables, pero no demuestra que la predicción sea precisa. La explicabilidad debe combinarse con métricas de error, validación temporal, análisis de robustez y conocimiento financiero del producto.

C.9 USO EN ESTE PROYECTO

En este trabajo, SHAP se utiliza para explicar modelos de predicción de volatilidad implícita desde dos perspectivas complementarias. A nivel local, permite analizar por qué el modelo asigna una determinada volatilidad a un trade concreto. A nivel global, permite identificar qué variables concentran mayor intensidad explicativa en el conjunto de observaciones analizadas.

Las explicaciones se calculan sobre variables visibles financieramente interpretables, como `OptionType`, `StrikePrice`, `UnderlyingPrice`, `TimeToExpiration` y `Rate`. Aunque el modelo pueda utilizar transformaciones internas o variables derivadas, la capa de explicabilidad reconstruye las entradas necesarias para mantener una lectura coherente desde el punto de vista financiero.

Los artefactos visuales del dashboard, como gráficos *beeswarm*, rankings de importancia media absoluta, *dependence plots*, *heatmaps* y gráficos *waterfall*, son distintas formas de representar la misma estructura matemática: la descomposición de una predicción en un valor base y un conjunto de contribuciones aditivas. Así, SHAP no se utiliza como sustituto de la validación del modelo, sino como herramienta complementaria para auditar su comportamiento y mejorar la trazabilidad de sus predicciones.

CURVAS ICE

Las curvas ICE, del inglés *Individual Conditional Expectation*, son una herramienta de explicabilidad utilizada para estudiar cómo cambia la predicción de un modelo cuando se modifica una variable concreta y se mantienen fijas todas las demás. En este trabajo se emplean para analizar el comportamiento del modelo de volatilidad implícita ante cambios contrafactuales en variables financieras relevantes como el precio de ejercicio, el precio del subyacente, el tiempo hasta vencimiento o el tipo de interés.

La idea fundamental es sencilla: se toma una observación real del conjunto de datos, se modifica únicamente una de sus variables y se vuelve a evaluar el modelo. Repitiendo este proceso para varios valores de dicha variable se obtiene una curva individual de respuesta. A diferencia de otras técnicas globales, ICE no resume directamente el efecto medio de una variable, sino que muestra la trayectoria de predicción para cada observación por separado. Esto resulta especialmente útil en modelos de volatilidad, donde el efecto de una variable puede depender fuertemente del contexto del contrato: tipo de opción, vencimiento, nivel del subyacente, strike relativo o zona de la superficie de volatilidad.

Las curvas ICE fueron propuestas por [Goldstein et al. \[2015a\]](#) como una extensión individualizada de los gráficos de dependencia parcial, y se han consolidado como una técnica habitual de interpretabilidad agnóstica al modelo [[Molnar, 2022](#)].

Formalmente, sea f el modelo entrenado de predicción de volatilidad, sea x_i una observación del conjunto de datos y sea j la variable que se quiere analizar. Denotamos por $x_{i,-j}$ al conjunto de variables de la observación i salvo la variable j . Para un valor contrafactual z , la curva ICE de la observación i respecto a la variable j se define como:

$$ICE_i^{(j)}(z) = f(z, x_{i,-j}). \quad (34)$$

Esta expresión representa la predicción que produciría el modelo si la observación i mantuviera todas sus características originales excepto la variable j , que se sustituye

por el valor z . Al evaluar la ecuación anterior sobre una rejilla de valores $z_1, \dots, z_m$, se obtiene una curva:

$$\{f(z_1, x_{i,-j}), f(z_2, x_{i,-j}), \dots, f(z_m, x_{i,-j})\}. \quad (35)$$

Cada curva ICE responde por tanto a una pregunta contrafactual: *qué habría predicho el modelo para este contrato concreto si solo cambiara una determinada variable*. En el contexto de opciones, por ejemplo, una curva ICE para `StrikePrice` muestra cómo cambiaría la volatilidad predicha para un contrato dado si se evaluara ese mismo contrato bajo distintos precios de ejercicio, manteniendo fijos el resto de atributos usados por el modelo.

La Figura 42 resume el flujo de construcción de estas curvas en el dashboard.

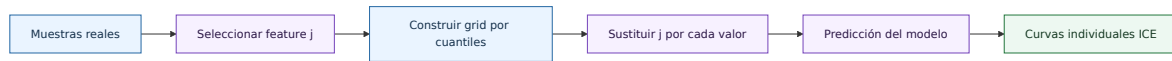


Figura 42: Construcción de curvas ICE por feature.

CONSTRUCCIÓN DE LAS CURVAS EN EL PROYECTO

En el dashboard desarrollado en este trabajo, las curvas ICE se construyen mediante la función `build_ice_frame`. El procedimiento seguido puede resumirse en los siguientes pasos:

1. Se selecciona una muestra de observaciones reales del conjunto de datos usado por el dashboard. Para evitar una visualización excesivamente densa, el número máximo de curvas individuales está controlado por el parámetro `ice_sample_size`, fijado en este proyecto a 24.
2. Para cada variable analizada se construye una rejilla de valores. En lugar de usar una rejilla equiespaciada entre el mínimo y el máximo, se utilizan cuantiles de la distribución observada. Esto concentra los puntos de evaluación en zonas donde existen datos reales y reduce, aunque no elimina, el riesgo de evaluar escenarios extremadamente alejados de la muestra.
3. Para cada observación seleccionada y para cada valor de la rejilla, se crea una copia contrafactual de la observación sustituyendo únicamente la variable analizada. El resto de variables permanece constante.
4. El modelo vuelve a predecir la volatilidad implícita sobre cada una de estas observaciones contrafactuales.
5. Finalmente se guarda una tabla con la variable analizada, el identificador de la muestra, el valor contrafactual y la predicción obtenida. Esta tabla es la que permite representar una curva por observación.

En la configuración utilizada, cada curva se evalúa sobre `curve_points=25` puntos. Las variables incluidas en el análisis ICE son `StrikePrice`, `UnderlyingPrice`, `TimeToExpiration` y `Rate`. La elección de estas variables responde a su interpretación financiera directa: el strike y el subyacente determinan la moneyness de la opción; el tiempo hasta vencimiento condiciona la estructura temporal de la volatilidad; y el tipo de interés afecta al valor teórico de la opción y, de forma indirecta, a la volatilidad implícita estimada.

INTERPRETACIÓN DE LAS CURVAS ICE

La lectura de una curva ICE debe hacerse de forma individual y comparativa. Individualmente, la pendiente de una curva indica cómo reacciona la predicción del modelo para una observación concreta al modificar la variable analizada. Comparativamente, la forma conjunta de todas las curvas permite identificar si el modelo responde de manera homogénea o heterogénea en distintas regiones del espacio de datos.

Si las curvas son aproximadamente paralelas, el modelo está aplicando un efecto similar de la variable analizada sobre la mayoría de observaciones. En ese caso, la variable tiene una influencia relativamente estable y poco dependiente del contexto. Por ejemplo, si las curvas ICE de `Rate` fueran casi paralelas y con pendiente suave, esto sugeriría que el modelo incorpora el efecto del tipo de interés de manera parecida para distintos contratos.

Si, por el contrario, las curvas tienen pendientes muy distintas, se cruzan o muestran comportamientos opuestos, existe heterogeneidad en la respuesta del modelo. Esta heterogeneidad suele ser una señal de interacción entre variables. En opciones financieras, esto es esperable. El efecto de aumentar el `StrikePrice` no puede interpretarse de forma aislada, porque depende del nivel del `UnderlyingPrice`; conjuntamente ambas variables determinan si la opción está dentro, fuera o cerca del dinero. Por tanto, una fuerte divergencia en las curvas ICE de `StrikePrice` puede indicar que el modelo está capturando efectos asociados a la moneyness, aunque esta no aparezca explícitamente como variable analizada.

RELACIÓN ENTRE ICE Y PDP

Las curvas ICE están estrechamente relacionadas con los *Partial Dependence Plots* o PDP. Mientras que ICE muestra una curva por observación, el PDP promedia todas esas curvas para obtener un efecto medio global de la variable analizada:

$$PDP_j(z) = \frac{1}{n} \sum_{i=1}^n f(z, x_{i,-j}). \quad (36)$$

Por tanto, un PDP puede entenderse como el promedio vertical de las curvas ICE en cada valor z . Esta agregación facilita una lectura global, pero también puede ocultar comportamientos relevantes. Si para algunas observaciones la predicción aumenta al incrementar una variable y para otras disminuye, el promedio puede resultar casi plano. En ese caso, el PDP sugeriría erróneamente que la variable tiene poco efecto, cuando en realidad el modelo presenta efectos locales fuertes pero de signo contrario.

Esta diferencia es relevante en este trabajo porque la volatilidad implícita no depende de las variables de forma puramente aditiva. La relación entre strike, subyacente y vencimiento suele ser no lineal y estar condicionada por la zona de la superficie de volatilidad. Por ello, las curvas ICE aportan una visión más detallada que el promedio global: permiten comprobar si el efecto aprendido por el modelo es estable o si depende del tipo de contrato analizado.

RESUMEN

Las curvas ICE permiten inspeccionar cómo responde el modelo de volatilidad ante cambios controlados en variables financieras relevantes. Su principal ventaja es que revelan heterogeneidad e interacciones que pueden quedar ocultas en promedios globales como el PDP. Su principal riesgo es la generación de escenarios contrafactuales poco plausibles. Por ello, en este trabajo se utilizan como una herramienta de diagnóstico visual y explicabilidad local-global, siempre interpretadas junto con el soporte de datos y el comportamiento observado en la superficie de volatilidad.

CURVAS ALE

Las curvas ALE, del inglés *Accumulated Local Effects*, son una herramienta de explicabilidad utilizada para estudiar el efecto medio de una variable sobre la predicción de un modelo. En este trabajo se emplean para analizar cómo cambia la volatilidad implícita predicha cuando varían determinadas características financieras, como el precio de ejercicio, el precio del subyacente, el tiempo hasta vencimiento o el tipo de interés.

La formulación de ALE sigue la propuesta de [Apley and Zhu \[2020\]](#), y su uso se interpreta como una técnica de explicabilidad global basada en efectos locales acumulados, especialmente adecuada cuando las variables predictoras presentan dependencia estadística.

La motivación de ALE es similar a la de otras técnicas de análisis de sensibilidad: se quiere entender qué papel desempeña cada variable en la predicción del modelo. Sin embargo, ALE lo hace de forma especialmente útil cuando las variables explicativas están correlacionadas.

A diferencia de los gráficos PDP, que evalúan una variable sobre una rejilla global y promedian las predicciones resultantes, ALE calcula efectos locales dentro de regiones donde existen observaciones reales. Esto reduce el riesgo de evaluar el modelo en combinaciones artificiales poco representativas del conjunto de entrenamiento. Por tanto, ALE no elimina completamente el problema de las correlaciones entre variables, pero proporciona una estimación más prudente del efecto medio de una variable en zonas con soporte de datos.

Formalmente, sea f el modelo entrenado y sea X_j la variable que se quiere analizar. La curva ALE de la variable j se define como el efecto local medio acumulado:

$$ALE_j(z) = \int_{z_0}^z E \left[\frac{\partial f(X)}{\partial X_j} \mid X_j = s \right] ds - C. \quad (37)$$

Esta expresión puede interpretarse en dos pasos. Primero, se estima cómo cambia localmente la predicción del modelo cuando cambia ligeramente la variable X_j , condicionando a que dicha variable tome valores cercanos a s . Después, esos efectos locales se acumulan desde un punto inicial z_0 hasta el valor z . La constante C centra la curva, normalmente para que su media sea cero. Por ello, un valor ALE positivo no significa que la volatilidad predicha sea positiva, sino que el efecto acumulado de esa variable en esa zona está por encima de su nivel medio de referencia.

CONSTRUCCIÓN POR INTERVALOS

En la práctica, el modelo no se deriva analíticamente ni se calcula la integral anterior de forma exacta. En su lugar, se utiliza una aproximación por intervalos, o *bins*. Para cada variable analizada, se divide su distribución observada en varios intervalos y se mide cuánto cambia la predicción al pasar del borde inferior al borde superior de cada intervalo.

Sea K el número de intervalos y sea $[l_k, u_k]$ el intervalo k -ésimo. Se define I_k como el conjunto de observaciones cuyo valor de la variable j cae dentro de dicho intervalo:

$$I_k = \{i : x_{ij} \in [l_k, u_k]\}. \quad (38)$$

Para cada observación $i \in I_k$, se crean dos versiones contrafactuales locales. En la primera, la variable j se sustituye por el borde inferior l_k ; en la segunda, se sustituye por el borde superior u_k . El resto de variables se mantiene igual. La diferencia entre ambas predicciones aproxima el efecto local de moverse dentro de ese intervalo:

$$f(u_k, x_{i,-j}) - f(l_k, x_{i,-j}). \quad (39)$$

Promediando esta diferencia sobre todas las observaciones del intervalo se obtiene el incremento local medio del bin:

$$\Delta_k = \frac{1}{|I_k|} \sum_{i \in I_k} [f(u_k, x_{i,-j}) - f(l_k, x_{i,-j})]. \quad (40)$$

El valor Δ_k indica si, dentro de ese intervalo observado, pasar del borde inferior al borde superior tiende a aumentar o disminuir la predicción del modelo. Si $\Delta_k > 0$, el modelo predice, en promedio, una mayor volatilidad al avanzar dentro del intervalo. Si $\Delta_k < 0$, el efecto local medio es descendente. Si Δ_k es cercano a cero, el modelo apenas modifica su predicción en esa región.

Una vez calculados los incrementos locales, se acumulan de forma secuencial:

$$ALE_k^u = \sum_{r \leq k} \Delta_r, \quad (41)$$

donde ALE_k^u representa el efecto acumulado sin centrar hasta el intervalo k . Finalmente, la curva se centra restando la media de los valores acumulados:

$$ALE_k = ALE_k^u - \frac{1}{K} \sum_{r=1}^K ALE_r^u. \quad (42)$$

Este centrado hace que la curva tenga interpretación relativa. El nivel cero no representa volatilidad cero ni ausencia absoluta de efecto. Representa el nivel medio de referencia del efecto acumulado de la variable. Por tanto, lo importante no es únicamente si la curva está por encima o por debajo de cero, sino su forma, su pendiente y los cambios de pendiente entre regiones.

IMPLEMENTACIÓN EN EL PROYECTO

En el dashboard de este trabajo, las curvas ALE se calculan mediante la función `build_ale_frame`. Para cada variable de análisis, la implementación toma la distribución observada en el conjunto de datos y construye bordes por cuantiles entre 0.05 y 0.95. En la configuración utilizada se emplean 13 puntos, lo que genera intervalos internos sobre los que se calculan los efectos locales.

El uso de cuantiles tiene una ventaja importante: concentra los intervalos en zonas donde realmente existen observaciones. En vez de dividir el rango de la variable de forma equiespaciada, lo cual podría producir intervalos con muy pocos datos en regiones extremas, los cuantiles distribuyen mejor las observaciones entre los bins. Esto resulta especialmente relevante en datos financieros, donde algunas zonas de la superficie de volatilidad pueden estar mucho más pobladas que otras.

INTERPRETACIÓN DE LA CURVA

La curva ALE se interpreta como un efecto acumulado relativo. Una curva creciente indica que, al aumentar la variable, el modelo tiende a incrementar la volatilidad predicha en las regiones observadas. Una curva decreciente indica el efecto contrario. Una curva aproximadamente plana sugiere que la variable tiene poco efecto marginal medio en las zonas analizadas. Una curva curvada indica que el efecto no es constante: puede ser fuerte en unas regiones y débil en otras.

También conviene prestar atención a tramos irregulares. Cambios bruscos, oscilaciones o dientes pueden deberse a ruido, escasez de observaciones en ciertos bins, sensibilidad excesiva del modelo o cambios reales en la relación aprendida. Por ello, una curva ALE no debe interpretarse únicamente como una verdad estructural del mercado, sino como una descripción del comportamiento del modelo entrenado sobre las regiones representadas en los datos.

RELACIÓN CON ICE Y PDP

ALE, ICE y PDP responden a preguntas relacionadas, pero no equivalentes. ICE muestra cómo cambia la predicción para observaciones individuales cuando se modifica una variable. PDP promedia predicciones contrafactuales globales. ALE, en cambio, estima efectos locales medios dentro de regiones observadas y después los acumula.

Esta diferencia es importante. Si las variables estuvieran poco correlacionadas y el modelo tuviera efectos relativamente homogéneos, PDP y ALE podrían ofrecer conclusiones parecidas. Sin embargo, cuando existen correlaciones fuertes o regiones poco pobladas, PDP puede evaluar combinaciones poco plausibles. Por ejemplo, podría combinar un determinado vencimiento, strike y subyacente de una forma que apenas aparece en el mercado observado. ALE reduce este problema porque compara valores cercanos dentro de intervalos donde sí hay observaciones.

Respecto a ICE, ALE ofrece una lectura más agregada. Si las curvas ICE son muy heterogéneas pero la curva ALE es casi plana, puede significar que existen efectos locales opuestos que se compensan al promediar. Si ICE y ALE apuntan en la misma dirección, la evidencia sobre el efecto de la variable es más estable. Por tanto, en este trabajo ambas herramientas se interpretan de forma complementaria: ICE permite detectar heterogeneidad individual y ALE resume el efecto medio acumulado en zonas observadas.

LIMITACIONES

Aunque ALE es más robusto que PDP frente a combinaciones contrafactuales irreales, no elimina todas las dificultades. En primer lugar, sigue siendo una técnica agregada. Al promediar efectos locales dentro de cada intervalo, puede ocultar diferencias entre subgrupos de observaciones. En segundo lugar, la forma de la curva depende del número de bins y de la distribución de los datos. Pocos intervalos pueden suavizar demasiado el efecto; demasiados intervalos pueden introducir ruido.

En resumen, las curvas ALE permiten estudiar el efecto medio acumulado de una variable sobre la volatilidad implícita predicha, utilizando variaciones locales en regiones observadas. Su principal ventaja es que ofrecen una alternativa más prudente que PDP cuando las variables financieras están correlacionadas. Su principal limitación es que siguen siendo una herramienta agregada y descriptiva del modelo. En este trabajo se utilizan como complemento de ICE, superficies de volatilidad y análisis local para obtener una visión más completa del comportamiento aprendido por los modelos de pricing.

MODELO QUANTUM INSPIRED

Este apéndice describe la arquitectura *Quantum Inspired*, una red neuronal clásica que adopta un paradigma distinto al de matrices densas: utiliza una **factorización tensorial** inspirada en redes tensoriales usadas en física cuántica simulada clásicamente. La idea central es convertir el espacio de características embebidas en un tensor multidimensional y descomponerlo en una cadena de núcleos pequeños interconectados.

Esta estrategia es apropiada para sistemas con muchos grados de libertad (como la superficie de volatilidad) porque combina compresión estructural, regularización implícita y una lectura más trazable de cómo se propagan dependencias entre variables.

F.1 ARQUITECTURA GENERAL: FLUJO DE DATOS DESDE ENTRADA A PREDICCIÓN

La arquitectura transforma un vector de entrada en una predicción escalar de volatilidad siguiendo tres bloques: preparación, núcleo Tensor-Train y agregación final. La Figura 43 resume el flujo.

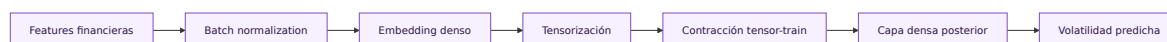


Figura 43: Arquitectura Quantum Inspired clásica basada en tensor-train.

La clave arquitectónica es que **el componente Quantum Inspired es la capa Tensor-Train**, no el conjunto completo. Las etapas anteriores preparan los datos; las posteriores agregan y refinan la salida. De este modo se separa la responsabilidad de cada bloque: preparación (embedding), transformación tensorial (TT) y agregación (post-TT dense).

F.2 ETAPAS PREPARATORIAS Y TRANSFORMACIÓN ESTRUCTURAL

F.2.1 Normalización e Embedding denso

Sea n_{in} el número de variables de entrada. La primera operación es una normalización por lote con transformación $n_{\text{in}} \rightarrow n_{\text{in}}$, que centra y escala cada mini-lote durante entrenamiento.

Después, el embedding denso aplica la transformación $n_{\text{in}} \rightarrow \prod_{k=1}^d n_k$, donde d es el orden del tensor y n_k es la dimensión del eje k del tensor objetivo. Esta proyección es crítica porque:

- **Captura relaciones no lineales:** modela interacciones entre variables desde el inicio.
- **Mejora y enriquecimiento:** la proyección denso-no-lineal reorganiza el espacio de características, destacando direcciones relevantes. Es análogo a un cambio de base adaptativo.
- **Evitación de cercanías espurias:** sin este paso, el reshape directo sería arbitrario. El embedding asegura que la interpretación de índices como “adyacentes” en el tensor tiene sentido matemático real.

El embedding se parametriza mediante:

$$\mathbf{z} = \sigma_{\text{emb}} (\mathbf{W}_{\text{emb}} \mathbf{x} + \mathbf{b}_{\text{emb}}), \quad (43)$$

donde $\mathbf{x} \in \mathbb{R}^{n_{\text{in}}}$ es el vector de entrada y σ_{emb} es la activación Swish. El resultado es $\mathbf{z} \in \mathbb{R}^{\prod n_k}$.

Se utiliza Swish en embedding porque suele mantener un flujo de gradientes más estable que alternativas más abruptas.

F.2.2 Reshape a tensor de alta dimensionalidad

El reshape realiza la transformación $\prod n_k \rightarrow n_1 \times n_2 \times \cdots \times n_d$. Es decir, el vector $\mathbf{z}$ se reinterpreta como un tensor de orden d :

$$\mathcal{Z} \in \mathbb{R}^{n_1 \times n_2 \times \cdots \times n_d}, \quad (44)$$

donde cada dimensión $(n_1, n_2, \dots, n_d)$ representa un grado de libertad abstracto que puede codificar factores de riesgo agrupados.

Este reshape no es un mero cambio de vista: es el pivote conceptual que permite pasar de un vector euclidiano a una estructura multiíndice. Una vez que $\mathbf{z}$ es visto

como $\mathcal{Z}(i_1, i_2, \dots, i_d)$ (con $1 \leq i_k \leq n_k$), se pueden aplicar contracciones tensoriales que de otro modo serían imposibles.

La importancia radica en que **genera un objeto de altísima dimensionalidad implícita**. Si por ejemplo $(n_1, n_2, n_3, n_4) = (2, 2, 2, 2)$, entonces $\prod n_k = 16$ componentes se reinterpretan como un tensor de $2^4 = 16$ entradas. La contracción TT que viene después explotará esta estructura multidimensional para comprimir ese espacio de manera exponencial.

F.3 EL NÚCLEO QUANTUM INSPIRED: DESCOMPOSICIÓN TENSOR-TRAIN

La descomposición tensor-train (TT) es una forma estructurada de representar un tensor multidimensional mediante un producto de matrices pequeñas encadenadas. Para un tensor $\mathcal{Z}(i_1, \dots, i_d)$, la descomposición TT escribe:

$$\mathcal{Z}(i_1, i_2, \dots, i_d) \approx G_1[i_1] \cdot G_2[i_2] \cdot \dots \cdot G_d[i_d], \quad (45)$$

donde cada G_k es un tensor de orden 3 con forma (r_{k-1}, n_k, r_k) :

- n_k es la dimensión del eje k del tensor (determinada por la forma de $\mathcal{Z}$).
- r_{k-1} y r_k son los **rangos tensoriales** internos que conectan el núcleo $k - 1$ con el k .
- Los rangos en las fronteras son forzados a 1: $r_0 = r_d = 1$, asegurando que el producto final es un escalar (o en nuestro caso, una componente de salida).

Para dejar el esquema claro desde el inicio: **cada salida TT tiene su propia cadena de núcleos**. Si la capa TT produce m_{out} salidas, existen m_{out} cadenas independientes, todas aplicadas sobre el mismo tensor de entrada.

Cada núcleo G_k puede pensarse como una célula local que captura cómo la información asociada al eje k interactúa con sus vecinos a través de los rangos. El encadenamiento de núcleos es el **contact chain** (cadena de contacto), mecanismo por el cual la información y las dependencias se propagan de un extremo del tensor al otro.

F.3.1 La cadena de contacto (Contact Chain) y propagación de dependencias

Durante el entrenamiento, todos los núcleos $G_1, G_2, \dots, G_d$ aprenden simultáneamente. Los rangos internos $r_1, r_2, \dots, r_{d-1}$ actúan como conexiones que permiten que gradientes y actualizaciones de parámetros se propaguen:

- Un cambio en el núcleo G_k afecta directamente a r_{k-1} y r_k .

- Estos cambios, a su vez, repercuten en G_{k-1} y G_{k+1} a través de las contracciones tensoriales.
- De este modo, la red de dependencias es densa y altamente acoplada.

Esto contrasta radicalmente con una red densa donde cada capa aprende “de forma más independiente”: aquí, cada núcleo no solo aprende su propia parametrización, sino que está en constante diálogo con sus vecinos a través de la estructura tensorial compartida.

F.3.2 Implementación: estructura concreta de los núcleos

En código, cada núcleo para un output dado es creado como:

```
# Para cada salida out_idx in range(output_dim):
#   Para cada eje k del tensor de entrada:
#       Crear el núcleo con forma (r_k, n_k, r_{k+1})
tt_core = self.add_weight(
    name=f"tt_core_{out_idx}_{core_idx}",
    shape=(
        self.tt_ranks[core_idx],      # r_k
        mode_dim,                    # n_k
        self.tt_ranks[core_idx + 1],  # r_{k+1}
    ),
    initializer=GlorotUniform(), # Inicialización apropiada
    trainable=True,
)
```

Sobre inicialización, en el código actual los **núcleos TT** se inicializan con **Glorot Uniform**. El hiperparámetro `kernel_initializer` (`glorot_uniform/he_uniform`) se aplica a las capas densas de embedding y post-TT, no a los núcleos de `TensorTrainLayer`. En todos los casos, el objetivo es arrancar con una escala de pesos compatible con entrenamiento estable en las primeras épocas.

F.3.3 Contracción: cómo se produce una salida TT

Dado un tensor $\mathcal{Z}(i_1, \dots, i_d)$ y una cadena de núcleos $\{G_1, \dots, G_d\}$, la contracción TT produce una salida mediante:

$$\text{output} = \sum_{i_1=1}^{n_1} \cdots \sum_{i_d=1}^{n_d} G_1[i_1] \cdot G_2[i_2] \cdot \dots \cdot G_d[i_d] \cdot \mathcal{Z}(i_1, \dots, i_d). \quad (46)$$

Conceptualmente, la operación itera a través de los núcleos, contrayendo los índices de cada eje del tensor de entrada y propagando información a través de los rangos internos. Lo crucial es que esta contracción se realiza para *cada* output: si hay m_{out} salidas, existen m_{out} cadenas independientes de núcleos, cada una contraída separadamente. Todas comparten la misma entrada $\mathcal{Z}$, pero aprenden distintos patrones de interacción.

En la ecuación 46, i_k recorre los índices del eje k , $G_k[i_k]$ es el bloque del núcleo k asociado a ese índice, y el producto encadenado acumula la interacción entre ejes y rangos internos.

F.3.4 Ejemplo concreto: relación entre dimensiones de entrada, rangos y salidas

Supongamos:

- Input original: $n_{\text{in}} = 5$ características (precio subyacente, strike, vencimiento, tasa, tipo).
- Tensor target: $(n_1, n_2, n_3, n_4) = (2, 2, 2, 2)$, así $\prod n_k = 16$.
- Embedding: capa densa $5 \rightarrow 16$.
- Reshape: los 16 componentes se organizan como tensor $2 \times 2 \times 2 \times 2$.
- Rangos TT: $(r_0, r_1, r_2, r_3, r_4) = (1, 2, 2, 2, 1)$.
- Salidas TT: $m_{\text{out}} = 8$ (especificado en configuración).

Aquí conviene distinguir dos magnitudes distintas: el valor $16 = \prod n_k$ describe el tamaño del **tensor de entrada** que recibe la capa TT, mientras que $m_{\text{out}} = 8$ describe el número de **cadenas TT independientes** y, por tanto, el número de características de salida producidas por la capa.

Para cada una de las 8 salidas:

- Núcleo 1: forma $(1, 2, 2)$ — conecta “antes” (rango 0, siempre 1) con rango 1.
- Núcleo 2: forma $(2, 2, 2)$ — conecta rango 1 con rango 2.
- Núcleo 3: forma $(2, 2, 2)$ — conecta rango 2 con rango 3.
- Núcleo 4: forma $(2, 2, 1)$ — conecta rango 3 con “después” (rango 4, siempre 1).

Total de parámetros por cadena: $(1 \times 2 \times 2) + (2 \times 2 \times 2) + (2 \times 2 \times 2) + (2 \times 2 \times 1) = 4 + 8 + 8 + 4 = 24$ elementos.

Como la implementación crea **una cadena completa por cada salida** (no una única TT-matrix factorizando simultáneamente entrada y salida), el total en la capa TT es $8 \times 24 = 192$ parámetros. En el embedding, si contamos pesos y sesgo, son $5 \times 16 + 16 = 96$ parámetros. Esto sigue siendo compacto frente a una alternativa densa equivalente.

Se utiliza **Swish** como activación post-TT y en el embedding por su suavidad y por mantener un entrenamiento estable.

F.4 ETAPAS FINALES: AGREGACIÓN Y SALIDA

Tras la contracción TT, tenemos m_{out} características aprendidas. Estas pasan por una capa densa intermedia ($m_{\text{out}} \rightarrow u_{\text{dense}}$), seguida opcionalmente de Dropout, y finalmente por una capa de salida ($u_{\text{dense}} \rightarrow 1$) que genera la predicción escalar de volatilidad.

Esta última etapa ejecuta:

$$\hat{\sigma}(\mathbf{x}) = \mathbf{W}_{\text{out}} \cdot \phi(\mathbf{W}_{\text{post}} \cdot \mathbf{c}_{\text{TT}} + \mathbf{b}_{\text{post}}) + b_{\text{out}}, \quad (47)$$

donde $\mathbf{c}_{\text{TT}}$ son las m_{out} salidas de la contracción TT.

Lectura rápida de los términos: $\mathbf{W}_{\text{post}}$ y $\mathbf{b}_{\text{post}}$ definen la proyección densa intermedia, $\phi(\cdot)$ representa la función de activación de ese bloque (en la configuración base se usa Swish), y $\mathbf{W}_{\text{out}}, b_{\text{out}}$ realizan la proyección final al escalar $\hat{\sigma}(\mathbf{x})$.

El Dropout es una regularización estocástica que desactiva aleatoriamente algunas neuronas durante entrenamiento, reduciendo co-adaptación y mejorando generalización.

F.5 JUSTIFICACIÓN DEL ENFOQUE QUANTUM INSPIRED

La etiqueta “quantum inspired” se refiere a que adoptamos paradigmas de redes tensoriales originarios de la simulación clásica de sistemas cuánticos. En física cuántica, un estado multiqubit se describe mediante un tensor exponencialmente grande (en principio, 2^N componentes para N qubits). Para computación tratable se usan descomposiciones tensoriales como tensor-train (conocida como *matrix product state*) que comprimen esa representación en productos de matrices pequeñas. Esta misma intuición de representación compacta y aprendizaje supervisado con redes tensoriales se ha formalizado en literatura de referencia para ML clásico [Stoudenmire and Schwab, 2016, Novikov et al., 2015].

La conexión con lo “quantum” está en la **estructura matemática** (red tensorial tipo MPS/TT), no en el hardware ni en el algoritmo de entrenamiento: aquí no hay qubits ni circuitos cuánticos, sino optimización clásica por gradiente sobre una parametrización inspirada en esas descomposiciones.

Aquí hacemos lo análogo en contexto clásico (predicción de volatilidad): reconocemos que aunque los datos sugieren un espacio de altísima dimensionalidad, es

potencialmente compresible mediante factorización. Los grados de libertad del sistema (strike, vencimiento, subyacente, tasas) no son independientes; están acoplados por dinámicas de mercado. Capturar esa estructura acoplada mediante núcleos tensoriales es más eficiente que parametrización densa.

El contact chain (cadena de contacto) permite propagación bidireccional de dependencias durante entrenamiento: el modelo aprende no solo factores locales, sino cómo se **coordinan** esos factores en la cadena entera. Esto es esencial para capturar fenómenos como sonrisas de volatilidad.

La inferencia y entrenamiento son completamente clásicos (CPUs/GPUs estándar). No hay computación cuántica real: es puramente una arquitectura neuronal inspirada en estructuras tensoriales de física cuántica, implementada como red clásica bajo el mismo protocolo que otros modelos.

F.6 VENTAJAS FRENTE A UNA RED DENSA ESTÁNDAR

Una red secuencial convencional con capas densas tendría el siguiente flujo:

- Input (5 dimensiones) $\rightarrow$ Dense (32 unidades) $\rightarrow$ Dense (24 unidades) $\rightarrow$ Output (1 dimensión).

Cálculo de parámetros (incluyendo bias): $(5 \times 32 + 32) + (32 \times 24 + 24) + (24 \times 1 + 1) = 192 + 792 + 25 = 1009$ parámetros.

La arquitectura Quantum Inspired, con la misma aproximación de complejidad computacional:

- Embedding: $5 \rightarrow 16$ ($5 \times 16 + 16 = 96$ parámetros).
- Tensor-Train (8 cadenas de salida, cada una con estructura (2,2,2,2) y rangos (1,2,2,2,1)): $8 \times 24 = 192$ parámetros.
- Post-TT Dense: $8 \rightarrow 32 \rightarrow 1$: $(8 \times 32 + 32) + (32 \times 1 + 1) = 288 + 33 = 321$ parámetros.
- Total (sin BatchNorm): $96 + 192 + 321 = 609$ parámetros.

Con configuraciones similares de capacidad, el modelo TT es más compacto. Pero la ventaja no es solo la compresión de parámetros: es la **estructura inductiva** que impone la factorización tensorial. Mientras que una red densa puede representar cualquier función (dada suficiente capacidad), también puede aprender cualquier ruido. La factorización TT, al obligar a factorizar el espacio latente, **induce un sesgo inductivo** hacia soluciones que exhiben estructura de rango bajo, lo cual es a menudo una propiedad de sistemas complejos.

Adicionalmente, la propagación de gradientes en una arquitectura TT es más directa: los rangos internos actúan como canales de comunicación entre núcleos, reduciendo la necesidad de capas densas muy anchas para capturar interacciones complejas.

FUNDAMENTOS DE REGRESIÓN SIMBÓLICA

La regresión simbólica es una técnica de aprendizaje automático cuyo objetivo es encontrar una expresión matemática explícita que aproxime la relación entre un conjunto de variables de entrada y una variable objetivo. A diferencia de una regresión lineal, donde la forma funcional se fija de antemano, o de una red neuronal, donde la relación aprendida queda distribuida entre muchos parámetros, la regresión simbólica busca simultáneamente la estructura de la fórmula y sus coeficientes.

En términos generales, dado un conjunto de observaciones

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad x_i \in \mathbb{R}^p, \quad y_i \in \mathbb{R}, \quad (48)$$

el objetivo consiste en encontrar una función explícita g tal que

$$y_i \approx g(x_i), \quad (49)$$

donde g no pertenece necesariamente a una familia paramétrica cerrada previamente definida. En lugar de asumir, por ejemplo, una forma lineal, polinómica o exponencial concreta, la regresión simbólica explora un espacio de expresiones construido a partir de variables, constantes y operadores matemáticos.

G.1 MOTIVACIÓN

La motivación principal de la regresión simbólica es combinar capacidad predictiva e interpretabilidad. En muchos problemas, especialmente cuando se usan modelos no lineales complejos, el modelo entrenado puede producir buenas predicciones pero ser difícil de interpretar directamente. Una red neuronal, por ejemplo, puede aproximar relaciones muy flexibles, pero no suele proporcionar una ecuación simple que describa su comportamiento.

La regresión simbólica aborda este problema buscando una fórmula cerrada que aproxime la relación aprendida. Esta fórmula puede utilizarse de dos maneras. En primer lugar, como modelo predictivo interpretable si se ajusta directamente sobre la variable objetivo. En segundo lugar, como modelo sustituto o *surrogate* si se ajusta sobre las predicciones de otro modelo más complejo. En este segundo caso, la regresión simbólica no pretende descubrir necesariamente la verdadera ley económica del fenómeno, sino aproximar de forma interpretable el comportamiento de un modelo previamente entrenado.

En este proyecto se utiliza esta segunda lectura. La regresión simbólica se ajusta sobre las predicciones generadas por los modelos de volatilidad implícita. Por tanto, la ecuación obtenida debe interpretarse como una aproximación funcional al modelo original, no como una fórmula financiera exacta ni como una alternativa directa a modelos clásicos de valoración.

G.2 ESPACIO DE BÚSQUEDA DE EXPRESIONES

El elemento central de la regresión simbólica es el espacio de expresiones candidatas. Una expresión simbólica puede construirse combinando variables, constantes y operadores. Por ejemplo, si las variables disponibles son x_1 , x_2 y x_3 , y los operadores permitidos son suma, resta, producto y división, algunas expresiones candidatas podrían ser

$$g_1(x) = x_1 + x_2, \quad (50)$$

$$g_2(x) = \frac{x_1}{x_2 + c}, \quad (51)$$

o

$$g_3(x) = c_1 x_1^2 + c_2 x_3, \quad (52)$$

donde c , c_1 y c_2 son constantes que también deben estimarse.

Este espacio de búsqueda es mucho más amplio que el de una regresión tradicional. No solo hay que encontrar los valores de los coeficientes, sino también decidir qué variables aparecen, qué operadores se usan, cómo se combinan y cuál es la profundidad o complejidad de la expresión. Por ello, la regresión simbólica suele formularse como un problema de optimización sobre estructuras.

Una forma habitual de representar las fórmulas candidatas es mediante árboles de expresión. En un árbol de expresión, las hojas representan variables o constantes, y los nodos internos representan operadores. Por ejemplo, la expresión

$$g(x) = \frac{x_1 + x_2}{x_3} \quad (53)$$

puede interpretarse como un árbol cuya raíz es una división, cuyo numerador es una suma y cuyo denominador es la variable x_3 . Esta representación permite aplicar operadores de búsqueda, mutación y recombinación sobre fórmulas completas.

G.3 OBJETIVO DE OPTIMIZACIÓN

La regresión simbólica no busca únicamente minimizar el error predictivo. También busca que la expresión resultante sea suficientemente simple para poder ser interpretada. Por ello, el problema se plantea habitualmente como un compromiso entre precisión y complejidad.

Sea g una expresión candidata. Una forma general de definir el objetivo es

$$\mathcal{L}(g) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, g(x_i)) + \lambda C(g), \quad (54)$$

donde ℓ es una función de pérdida, por ejemplo el error cuadrático, $C(g)$ mide la complejidad de la expresión y λ controla la penalización por complejidad.

Si se utiliza error cuadrático medio, la parte de ajuste puede escribirse como

$$\text{MSE}(g) = \frac{1}{n} \sum_{i=1}^n (y_i - g(x_i))^2. \quad (55)$$

El término de complejidad puede depender del número de nodos del árbol, de la profundidad de la expresión, del número de operadores utilizados o de restricciones específicas sobre determinadas operaciones. En la práctica, esta penalización evita que el algoritmo elija fórmulas excesivamente largas que ajustan bien los datos pero son difíciles de interpretar y tienen peor capacidad de generalización.

El parámetro λ refleja el principio de parsimonia: entre dos expresiones con precisión similar, debe preferirse la más simple. Este principio es especialmente importante en un proyecto de explicabilidad, donde una fórmula demasiado compleja puede perder gran parte de su utilidad interpretativa.

G.4 BÚSQUEDA EVOLUTIVA

El espacio de posibles expresiones crece de forma combinatoria con el número de variables, operadores y niveles de profundidad permitidos. Por ello, no suele ser posible enumerar todas las fórmulas candidatas. En su lugar, muchos métodos de regresión simbólica emplean algoritmos evolutivos o estrategias de búsqueda heurística.

El procedimiento general puede resumirse en los siguientes pasos. Primero, se genera una población inicial de fórmulas candidatas. Después, cada fórmula se evalúa de acuerdo con su error y su complejidad. Las expresiones con mejor comportamiento tienen mayor probabilidad de conservarse, modificarse o recombinarse para generar nuevas expresiones. Este proceso se repite durante muchas iteraciones, explorando progresivamente regiones prometedoras del espacio de búsqueda.

De forma esquemática, si $\mathcal{G}$ representa el conjunto de expresiones posibles, la regresión simbólica busca

$$g^* = \arg \min_{g \in \mathcal{G}} \left[\frac{1}{n} \sum_{i=1}^n \ell(y_i, g(x_i)) + \lambda C(g) \right]. \quad (56)$$

La dificultad reside en que $\mathcal{G}$ no es un espacio vectorial simple, sino un conjunto discreto y estructurado de árboles de expresión. Por tanto, la búsqueda combina exploración de estructuras, ajuste de constantes y selección de modelos.

G.5 FRONTERA PRECISIÓN-COMPLEJIDAD

Una característica importante de la regresión simbólica es que no produce necesariamente una única fórmula relevante. Normalmente genera una familia de ecuaciones candidatas con distintos niveles de complejidad y precisión. Algunas fórmulas son muy simples pero aproximan peor los datos. Otras son más complejas y reducen el error, pero resultan menos interpretables.

Esto permite analizar una frontera precisión-complejidad. Para cada expresión candidata g_k , pueden compararse dos magnitudes:

$$\text{Error}(g_k) \quad \text{y} \quad C(g_k). \quad (57)$$

La selección final no debe basarse exclusivamente en el menor error. En contextos de explicabilidad, puede ser preferible una ecuación ligeramente menos precisa pero mucho más clara. Por ejemplo, una fórmula con tres términos interpretables puede

aportar más valor analítico que una expresión muy extensa con una mejora marginal en RMSE.

En este proyecto, el modelo simbólico seleccionado se evalúa mediante métricas como RMSE, MAE y R^2 , pero también se conserva información sobre la complejidad de la ecuación y sobre un conjunto de ecuaciones candidatas. Esta decisión permite no reducir la regresión simbólica a una competición puramente predictiva, sino tratarla como una herramienta de análisis del compromiso entre fidelidad e interpretabilidad.

G.6 REGRESIÓN SIMBÓLICA COMO MODELO SUSTITUTO

En el contexto del proyecto, la regresión simbólica se utiliza como modelo sustituto de modelos de predicción de volatilidad implícita. Sea f el modelo original, por ejemplo una red neuronal o cualquier otro estimador entrenado, y sea g la expresión simbólica buscada. El objetivo no es ajustar g directamente contra la volatilidad observada, sino aproximar las predicciones del modelo original:

$$g(x_i) \approx f(x_i). \quad (58)$$

Por tanto, la pérdida se calcula frente a las predicciones del modelo base:

$$\mathcal{L}_{surrogate}(g) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - g(x_i))^2 + \lambda C(g). \quad (59)$$

Esta formulación tiene una interpretación clara. Si g aproxima bien a f , entonces la ecuación simbólica proporciona una descripción interpretable del comportamiento aprendido por el modelo original. Sin embargo, la fidelidad de g respecto a f no implica necesariamente que f sea correcto respecto al mercado. Una ecuación simbólica puede explicar muy bien un modelo mal calibrado. Por ello, la regresión simbólica debe interpretarse junto con las métricas predictivas del modelo original y con validaciones financieras adicionales.

G.7 CONFIGURACIÓN UTILIZADA EN EL PROYECTO

La implementación del proyecto utiliza PySRRegressor, una herramienta de regresión simbólica basada en búsqueda evolutiva. El modelo simbólico se entrena sobre las predicciones generadas por el modelo original en el conjunto de entrenamiento y se valida sobre las predicciones del modelo original en el conjunto de test. De este modo, la evaluación mide la fidelidad del sustituto simbólico respecto al modelo base fuera de la muestra usada para ajustar la expresión.

El flujo utilizado puede resumirse en cuatro fases.

1. Construir un marco de variables de explicabilidad a partir de los datos de referencia.
2. Codificar dichas variables mediante el encoder de explicabilidad, manteniendo un conjunto de variables financieramente interpretables.
3. Ajusta el regresor simbólico sobre las predicciones del modelo base.
4. Evalúa la ecuación obtenida sobre el conjunto de test, calculando métricas de fidelidad y almacenando tanto la mejor ecuación como un conjunto de ecuaciones candidatas.

Las variables utilizadas en la capa simbólica coinciden con las variables de explicabilidad del proyecto. Esta decisión favorece que la fórmula resultante sea legible desde un punto de vista financiero. En lugar de ajustar una expresión sobre representaciones internas difíciles de interpretar, la regresión simbólica se aplica sobre variables visibles y relacionadas con el contrato, como el tipo de opción, el precio de ejercicio, el precio del subyacente, el tiempo a vencimiento y el tipo de interés.

La configuración permite los operadores binarios

$$+, -, *, /, \wedge, \quad (60)$$

y los operadores unarios

$$\text{square}(x) = x^2, \quad \text{cube}(x) = x^3. \quad (61)$$

Además, se restringe el operador potencia mediante restricciones específicas para evitar expresiones demasiado inestables o difíciles de interpretar. Estas restricciones son relevantes porque operadores como la división o la potencia pueden producir fórmulas con gran capacidad de ajuste, pero también con comportamientos extremos fuera de la región de datos observada.

La búsqueda se controla mediante parámetros como el número de iteraciones, el número de poblaciones, el tamaño de población, el número de ciclos por iteración, la penalización de parsimonia, el tamaño máximo de expresión, la profundidad máxima y el tiempo máximo de ejecución. En particular, el proyecto utiliza un tamaño de muestra simbólica controlado por `symbolic_sample_size`, un número elevado de iteraciones `symbolic_niterations`, varias poblaciones en paralelo y un límite temporal de ejecución definido por `symbolic_timeout_seconds`. Estos parámetros reflejan el compromiso entre explorar suficientemente el espacio de fórmulas y mantener un coste computacional razonable dentro del dashboard.

G.8 EVALUACIÓN DE FIDELIDAD

Como el modelo simbólico actúa como sustituto, la evaluación principal mide la proximidad entre las predicciones del modelo original y las predicciones de la ecuación simbólica. Si $\hat{y}_i^f = f(x_i)$ representa la predicción del modelo base y $\hat{y}_i^g = g(x_i)$ la predicción simbólica, el residual de fidelidad se define como

$$r_i = \hat{y}_i^f - \hat{y}_i^g. \quad (62)$$

A partir de estos residuos se calculan métricas como RMSE, MAE y R^2 . El RMSE de fidelidad puede escribirse como

$$\text{RMSE}_{fid} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i^f - \hat{y}_i^g)^2}. \quad (63)$$

El MAE de fidelidad se define como

$$\text{MAE}_{fid} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i^f - \hat{y}_i^g|. \quad (64)$$

Estas métricas no comparan directamente la ecuación simbólica con la volatilidad observada, sino con el modelo que se quiere explicar. Por tanto, un bajo error de fidelidad indica que la ecuación simbólica reproduce bien el comportamiento del modelo base en la muestra evaluada.

El coeficiente R^2 de fidelidad mide qué proporción de la variabilidad de las predicciones del modelo base queda explicada por la fórmula simbólica:

$$R_{fid}^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i^f - \hat{y}_i^g)^2}{\sum_{i=1}^n (\hat{y}_i^f - \bar{\hat{y}}^f)^2}. \quad (65)$$

Un valor de R_{fid}^2 próximo a uno indica alta fidelidad global, mientras que un valor bajo indica que la fórmula simbólica no captura adecuadamente la variabilidad del modelo original.

G.9 INTERPRETACIÓN DE LA ECUACIÓN SIMBÓLICA

La ecuación simbólica final debe interpretarse como una aproximación analítica al comportamiento del modelo en la región de datos utilizada para entrenar y validar el sustituto. Su utilidad reside en que permite inspeccionar explícitamente qué variables aparecen en la fórmula, cómo se combinan y qué tipo de no linealidades emergen.

Por ejemplo, si la ecuación utiliza principalmente el tiempo a vencimiento y el strike, puede interpretarse que el sustituto simbólico ha identificado esas variables como suficientes para reproducir una parte relevante del comportamiento del modelo. Si aparecen interacciones multiplicativas o cocientes entre variables, puede existir una relación no lineal aprendida por el modelo que merece ser analizada financieramente. Sin embargo, estas lecturas deben hacerse con prudencia: la presencia de una variable en la fórmula no demuestra causalidad, y la ausencia de una variable no implica necesariamente irrelevancia económica.

También es importante considerar la estabilidad de la expresión. La regresión simbólica puede encontrar fórmulas matemáticamente válidas pero sensibles a pequeñas variaciones de entrada, especialmente si incluyen divisiones, potencias o combinaciones de alto grado. Por ello, las ecuaciones deben evaluarse no solo por su métrica agregada, sino también por su comportamiento en distintas regiones de la superficie de volatilidad, especialmente en vencimientos cortos, extremos de moneyness o zonas con menor densidad de observaciones.

G.10 LIMITACIONES

La regresión simbólica aporta interpretabilidad, pero no elimina las limitaciones propias del aprendizaje estadístico. En primer lugar, la búsqueda es heurística. El algoritmo no garantiza encontrar la mejor expresión posible en términos globales, sino una buena expresión dentro del espacio explorado y bajo las restricciones impuestas.

En segundo lugar, la ecuación encontrada depende del conjunto de operadores permitido. Si se excluyen funciones relevantes, el algoritmo no podrá descubrir expresiones que las necesiten. Si se incluyen demasiados operadores, el espacio de búsqueda se vuelve más amplio y aumenta el riesgo de encontrar fórmulas difíciles de interpretar.

En tercer lugar, la regresión simbólica puede sobreajustar. Una fórmula suficientemente compleja puede aproximar muy bien una muestra concreta y generalizar peor fuera de ella. Por este motivo, en el proyecto se evalúa la fidelidad sobre predicciones de test y se penaliza la complejidad mediante un término de parsimonia.

En cuarto lugar, cuando la regresión simbólica se usa como sustituto, la explicación queda limitada por el modelo base. La fórmula explica el comportamiento del modelo original, no necesariamente el mecanismo real que genera los precios o volatilidades de mercado.

Finalmente, las expresiones simbólicas deben interpretarse dentro del dominio observado. Una fórmula puede ser razonable para los rangos de strike, subyacente, vencimiento y tipos presentes en los datos, pero comportarse de forma inestable si se extrapola fuera de ellos. Por ello, su función principal en este trabajo es explicativa y diagnóstica, no la sustitución directa de los modelos de pricing o de predicción de volatilidad en producción.

G.11 RELACIÓN CON LA EXPLICABILIDAD DEL PROYECTO

Dentro del sistema de explicabilidad desarrollado, la regresión simbólica cumple una función complementaria a otras técnicas. Mientras que SHAP ofrece atribuciones locales y agregaciones globales de importancia, la regresión simbólica intenta condensar el comportamiento del modelo en una fórmula explícita. Esta fórmula puede ayudar a detectar patrones aprendidos, dependencias no lineales, interacciones entre variables y posibles simplificaciones del modelo original.

Su valor metodológico no está en afirmar que la ecuación descubierta sea una nueva ley cerrada de volatilidad implícita, sino en proporcionar una herramienta adicional para auditar modelos complejos. Si una expresión relativamente simple reproduce con alta fidelidad las predicciones de un modelo, entonces parte del comportamiento del modelo puede resumirse de forma transparente. Si, por el contrario, solo expresiones muy complejas alcanzan una fidelidad aceptable, esto sugiere que el modelo base está utilizando relaciones más difíciles de condensar en una fórmula interpretable.

Por tanto, la regresión simbólica se incorpora en este proyecto como una técnica de explicabilidad global basada en modelos sustitutos. Su objetivo es mejorar la trazabilidad y la comprensión del modelo de volatilidad, manteniendo siempre la distinción entre fidelidad al modelo, precisión frente al mercado e interpretabilidad financiera.

ENTRENAMIENTO PROGRESSIVE ATM

El entrenamiento progressive ATM se desarrolló como estudio paralelo para intentar mejorar el aprendizaje de la superficie. Su hipótesis de partida es financiera: la región ATM suele ser más líquida y central para la formación del nivel de volatilidad, mientras que los extremos de moneyness pueden contener más ruido, menor frecuencia o mayor sensibilidad a microestructura.

El procedimiento ordena las observaciones por distancia a ATM:

$$d_{ATM} = |\log(F/K)|. \quad (66)$$

Después divide el entrenamiento en segmentos. En la configuración actual se usan tres segmentos. Los modelos no iterativos reciben pesos mayores para tramos más cercanos al ATM; los modelos iterativos entrenan de forma acumulativa, empezando por la zona central e incorporando progresivamente tramos más alejados.

Si hay S segmentos ordenados de ATM a tramos más alejados, los pesos conceptuales decrecen con el segmento:

$$w_s \propto S - s, \quad \tilde{w}_i = \frac{w_i}{\bar{w}}. \quad (67)$$

La diferencia por familia es relevante: regresión lineal y Random Forest usan pesos de muestra; XGBoost reparte estimadores entre fases acumulativas; y las redes entrenan sucesivamente sobre datasets acumulados. En los resultados de este trabajo, progressive ATM mejora Quantum Inspired pero no mejora al Random Forest seleccionado. Por tanto, se conserva como estrategia analítica útil, pero no como regla general de entrenamiento.

ANÁLISIS DE SENSIBILIDAD POR MONEYNES Y VENCIMIENTO

Este apéndice desarrolla, en formato operativo, la sensibilidad del error por regiones de superficie. Se apoya en las vistas de *Diagnosis*, donde se integran tres gráficos clave: *Residual Heatmap*, *Error by Maturity* y *Error by Moneyness*. El objetivo es localizar zonas de mayor fragilidad para valoración y control de riesgos, no solo reportar un error promedio. Funciona como ampliación técnica de la Sección 4.2.



Figura 44: Análisis de sensibilidad por moneyness y vencimiento en los dos finalistas. Cada vista de *Diagnosis* integra el residual heatmap y las descomposiciones de error por maturity y por moneyness.

La Figura 44 permite una lectura directa por capas. Primero, el *Residual Heatmap* identifica si la concentración de error se desplaza hacia extremos de moneyness o hacia tramos de vencimiento concretos. Segundo, *Error by Maturity* resume si los vencimientos cortos, medios o largos concentran más desviación. Tercero, *Error by Moneyness* separa el desempeño entre regiones ITM, ATM y OTM.

Esta lectura es especialmente útil para mesa de derivados y tesorería porque conecta calidad predictiva con decisiones operativas. Si una zona presenta error estructuralmente mayor, esa zona requiere mayor prudencia en valoración, contraste adicional con modelos internos y monitorización reforzada de cobertura. En cambio, zonas con

error bajo y patrón estable son candidatas a uso más sistemático del modelo como referencia de apoyo.

BIBLIOGRAFÍA

- Daniel W. Apley and Jingyu Zhu. Visualizing the effects of predictor variables in black box supervised learning models. *Journal of the Royal Statistical Society: Series B*, 82(4):1059–1086, 2020.
- Fischer Black. The pricing of commodity contracts. *Journal of Financial Economics*, 3(1-2):167–179, 1976.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- Jay Cao, Jacky Chen, and John Hull. A neural network approach to understanding implied volatility movements. Working paper, Joseph L. Rotman School of Management, University of Toronto, 2019.
- Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- Miles Cranmer. Interpretable machine learning for science with PySR and symboli-cregression.jl. *arXiv preprint arXiv:2305.01582*, 2023.
- Emanuel Derman and Michael B. Miller. *The Volatility Smile*. Wiley, 2016.
- Alex Goldstein, Adam Kapelner, Justin Bleich, and Emil Pitkin. Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1):44–65, 2015a. doi: 10.1080/10618600.2014.907095.
- Alex Goldstein, Adam Kapelner, Justin Bleich, and Emil Pitkin. Peeking inside the black box: Visualizing statistical learning with plots of individual conditional expectation. *Journal of Computational and Graphical Statistics*, 24(1):44–65, 2015b.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- John C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 11 edition, 2022.
- Shuaiqiang Liu, Cornelis W. Oosterlee, and Sander M. Bohte. Pricing options and computing implied volatilities using neural networks. *arXiv preprint arXiv:1901.08943*, 2019.

- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Christoph Molnar. *Interpretable Machine Learning*. 2 edition, 2022. URL <https://christophm.github.io/interpretable-ml-book/>.
- Alexander Novikov, Dmitrii Podoprikin, Anton Osokin, and Dmitry P. Vetrov. Tensorizing neural networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- Fernando Perez-Cruz, Jermy Prenio, Fernando Restoy, and Jeffery Yong. Managing explanations: How regulators can address AI explainability. Technical Report 24, Bank for International Settlements, 2025.
- Sam Solaimani and Phoebe Long. Beyond the black box: Operationalising explicability in artificial intelligence for financial institutions. *International Journal of Business Information Systems*, 49(5), 2025. doi: 10.1504/IJBIS.2025.10071822.
- E. Miles Stoudenmire and David J. Schwab. Supervised learning with tensor networks. In *Advances in Neural Information Processing Systems*, volume 29, 2016.